

lecture and hands-on

presentation

IoT & Embedded OS Security



Internet of Things & Embedded OS Security

Cristian TOMA

E-mail: cristian.toma@ie.ase.ro

Web: ism.ase.ro | acs.ase.ro | dice.ase.ro

Github: www.github.com/critoma/



Business Card



Cristian Toma

IT&C Security Master

Dorobantilor Ave., No. 15-17
010572 Bucharest - Romania

<http://ism.ase.ro>

cristian.toma@ie.ase.ro

T +40 21 319 19 00 - 310

F +40 21 319 19 00



Agenda for IoT & eMbedded OS





**IoT Clouds, Device Communications Protocols – REST/MQTT/CoAP, IoT Java DIO
Hands-on: Java DIO and MQTT/HTTP-Rest/CoAP on Raspberry Pi/ESP8266, Node-
RED/Node.js**

Internet of Things



1. IoT Overview

“Mission: 50 Billion Connected Devices by 2020”

Some Big Numbers:

- 14 bn Connected Devices | Bosch SI
- 50 bn Connected Devices | Cisco
- 309 bn IoT Supplier Revenue | Gartner
- 1.9 tn IoT Economic Value Add | Gartner
- 7.1 tn IoT Solutions Revenue | IDC

Some Small Numbers:

Peter Middleton, Gartner:
By 2020, component costs will have come down to the point that connectivity will become a standard feature, even for processors costing less than

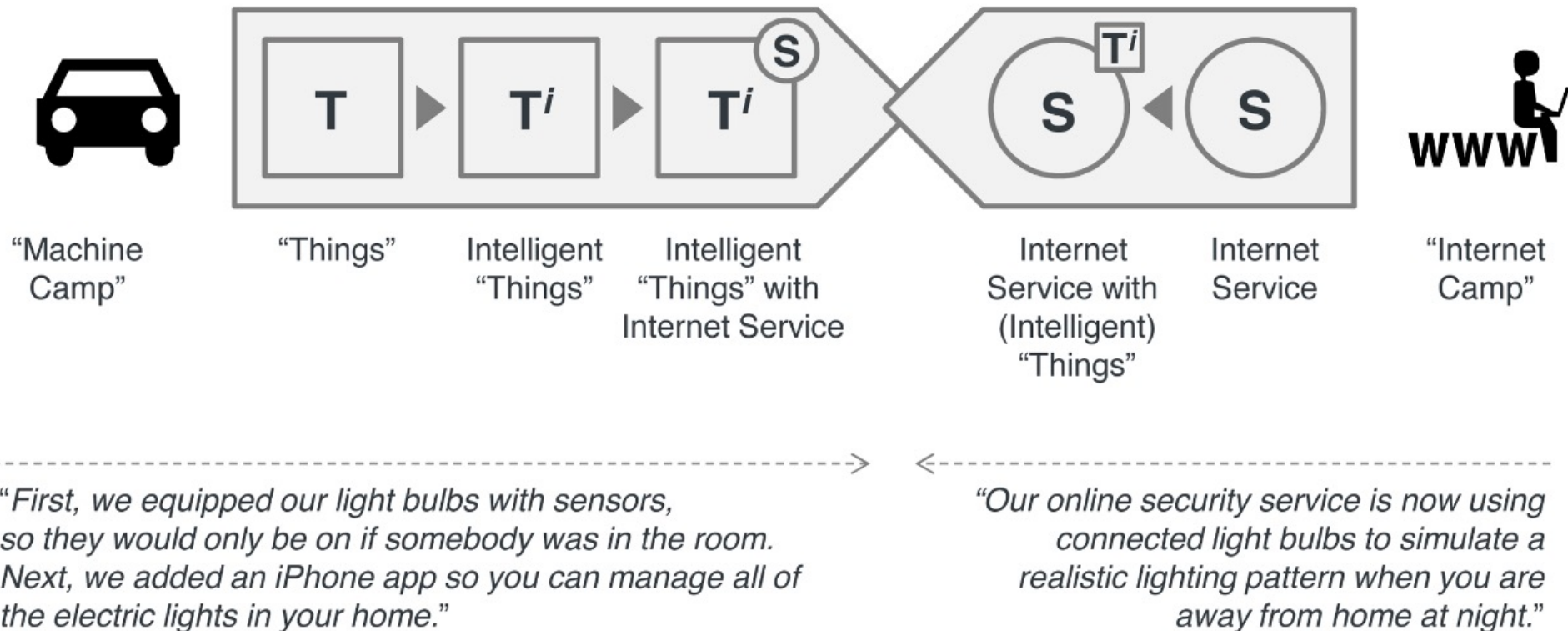
\$1

Data from <http://postscapes.com/internet-of-things-market-size>

Copyright: Excerpt From: Dirk Slama, Frank Puhlmann, Jim Morrish, and Rishi M. Bhatnagar - “Enterprise IoT”, O’Reilly Print House

1. IoT Overview

IoTS – Internet of Things Services Formula: “Difficulty of Finding the Right Service”



1. IoT Overview

IoT – Internet of Things Definition

- **Wiki:** Internet of Things (IoT) is the network of physical objects—devices, vehicles, buildings and other items embedded with electronics, software, sensors, and network connectivity—that enables these objects to collect and exchange data.^[1] The Internet of Things allows objects to be sensed and controlled remotely across existing network infrastructure,^[2] creating opportunities for more direct integration of the physical world into computer-based systems, and resulting in improved efficiency, accuracy and economic benefit;^{[3][4][5][6][7][8]} when IoT is augmented with sensors and actuators, the technology becomes an instance of the more general class of cyber-physical systems, which also encompasses technologies such as smart grids, smart homes, intelligent transportation and smart cities. Each thing is uniquely identifiable through its embedded computing system but is able to interoperate within the existing Internet infrastructure. Experts estimate that the IoT will consist of almost 50 billion objects by 2020.^[9]

1. IoT Overview

IoTS – Internet of Things Context

“Moore’s law: Ever-increasing hardware performance enables new levels of abstraction in the embedded space, which provides the basis for semantically rich embedded applications and the decoupling of on-asset hardware and software lifecycles. The app revolution for smartphones will soon be replicated in the embedded space.

Wireless technology: From ZigBee to Bluetooth LE, and from LTE/4G/5G (IoT-NB) to specialized low-power, wide-area (LPWA) IoT communication networks—the foundation for “always-on” assets and devices is either already available or in the process of being put in place.

Metcalfe’s law: Information and its value grow exponentially as the number of nodes connected to the IoT increases. With more and more remote assets being connected, it looks like we are reaching a tipping point.

Battery technology: Ever-improving battery quality enables new business models, from electric vehicles to battery-powered beacons.

Sensor technology: Ever-smaller and more energy-efficient sensors integrated into multi-axis sensors and sensor clusters, an increasing number of which are preinstalled in devices and assets.

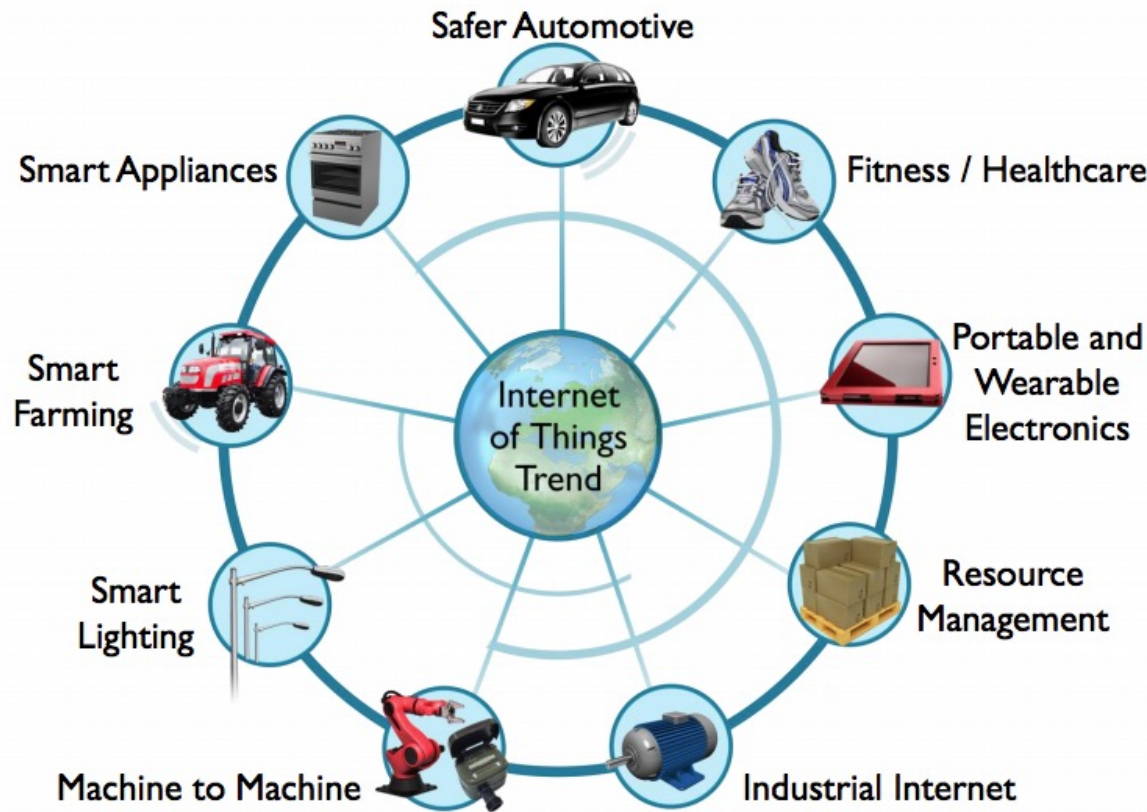
Big Data: Technology that is able to ingest, process, and analyze the massive amounts of sensor-generated data at affordable cost.

The cloud: The scalable, global platform that delivers data-centric services to enable new IoT business models.”

1. IoT Overview

IoTS – Internet of Things Intro

The “Internet of Things” Drives Opportunities



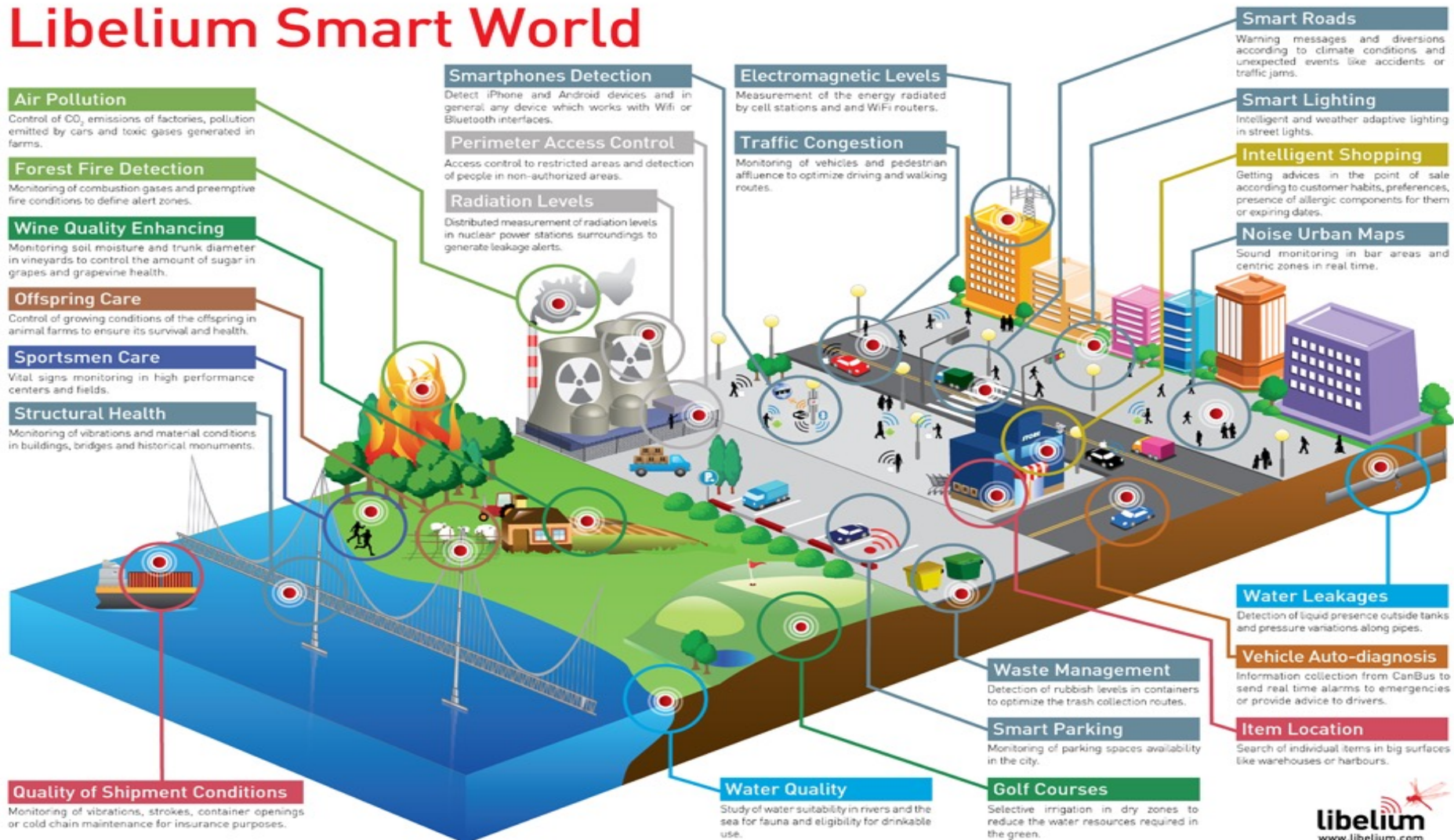
and Challenges

- ☐ Always on
- ☐ Increased connectivity
- ☐ Confidentiality
- ☐ Security
- ☐ Regulations
- ☐ New users

1. IoT Overview

IoTS – Internet of Things Smart Cities Solutions

Libelium Smart World

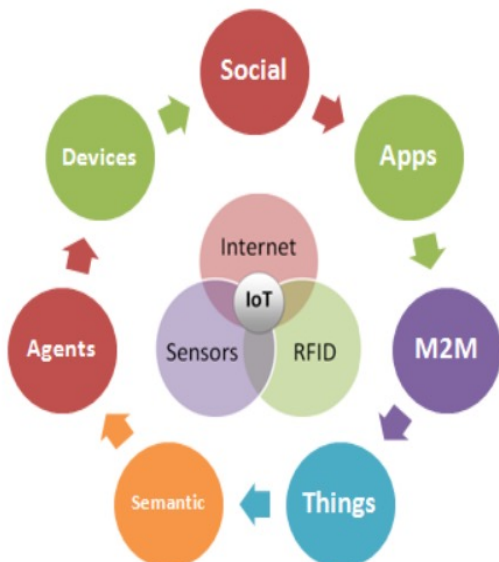
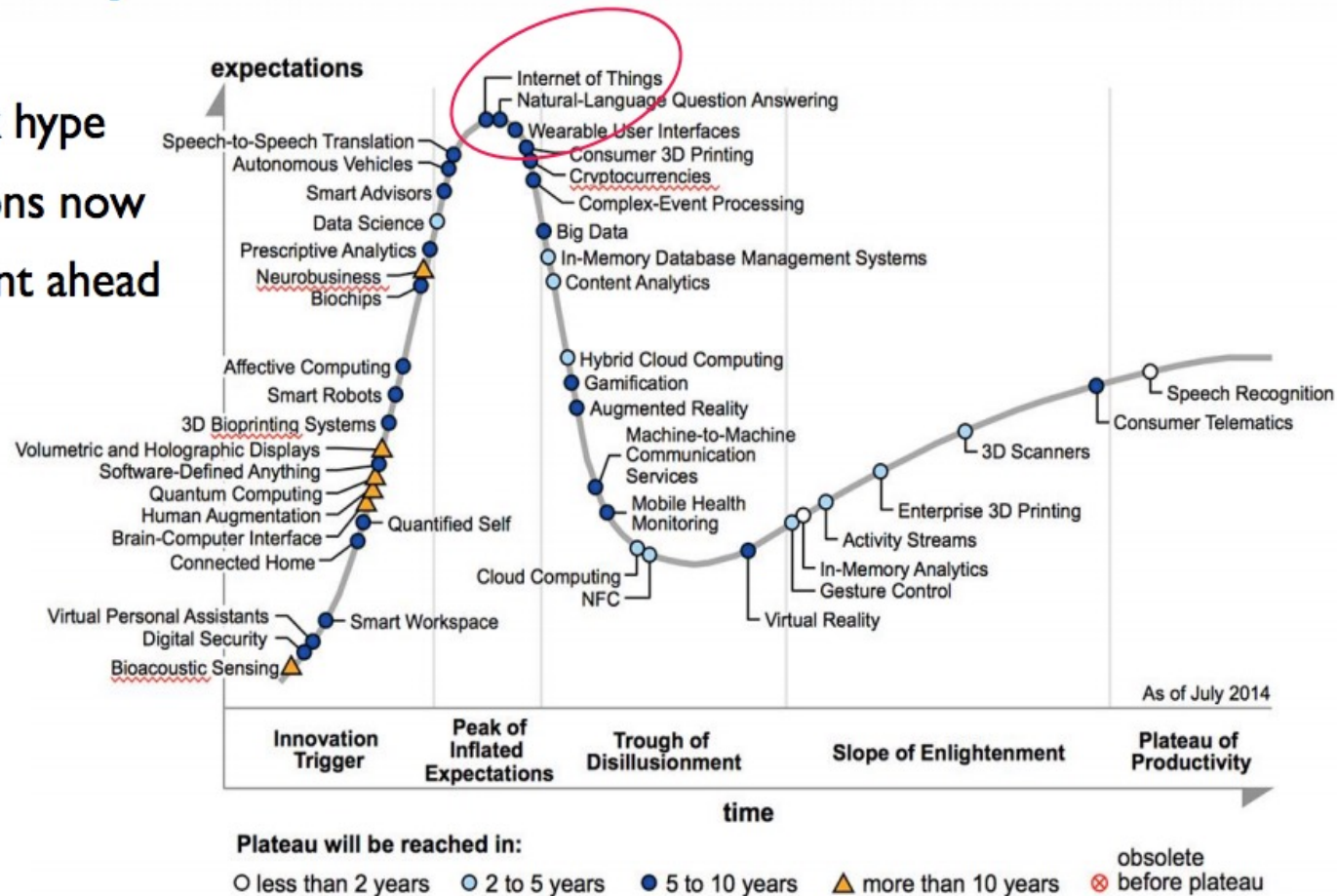


1. IoT Overview

IoT – Internet of Things Intro

IoT and the Hype Cycle

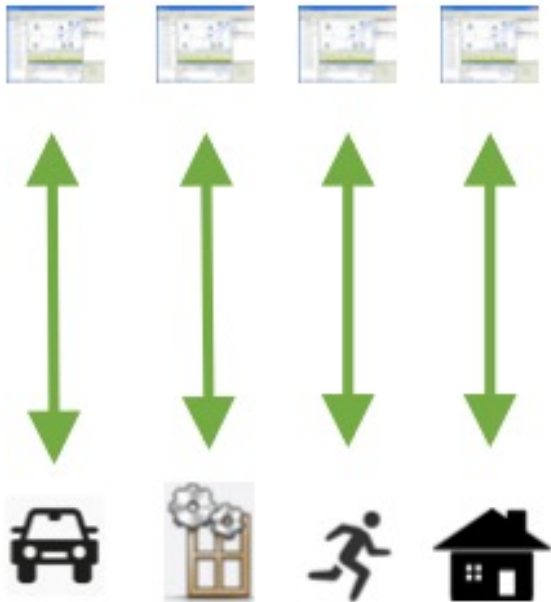
- ☐ Gartner has IoT at peak hype
- ☐ Most inflated expectations now
- ☐ Trough of Disillusionment ahead
- ☐ 5-10 years to Plateau



1. IoT Overview

IoTS – Internet of Things Evolution from M2M to IoT

M2M



Big Data
Internet of Things



IoT Cloud

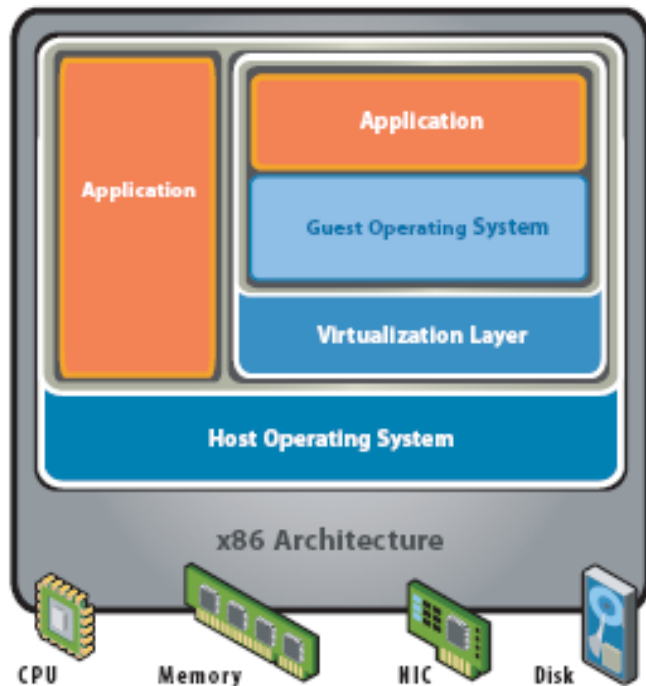
Services

The Web

Things

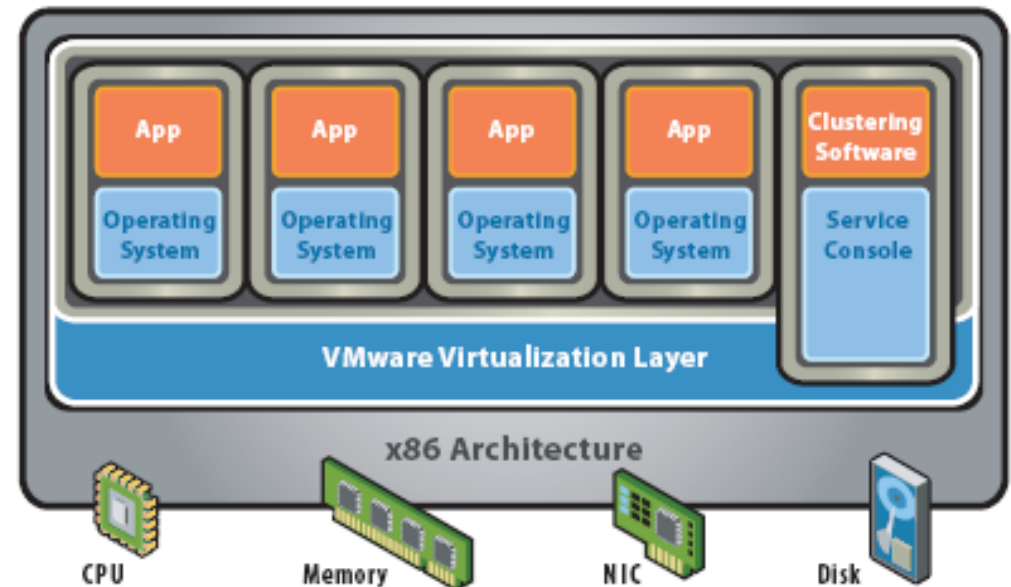
Little Data

RECAP: Cloud Concepts – Intro – Virtualization



Hosted Architecture

- Installs and runs as an application
- Relies on host OS for device support and physical resource management

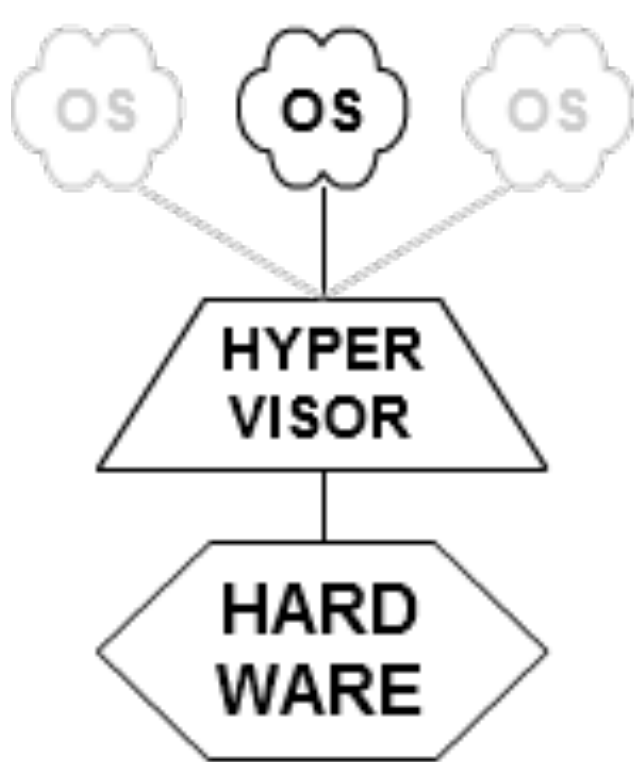


Bare-Metal (Hypervisor) Architecture

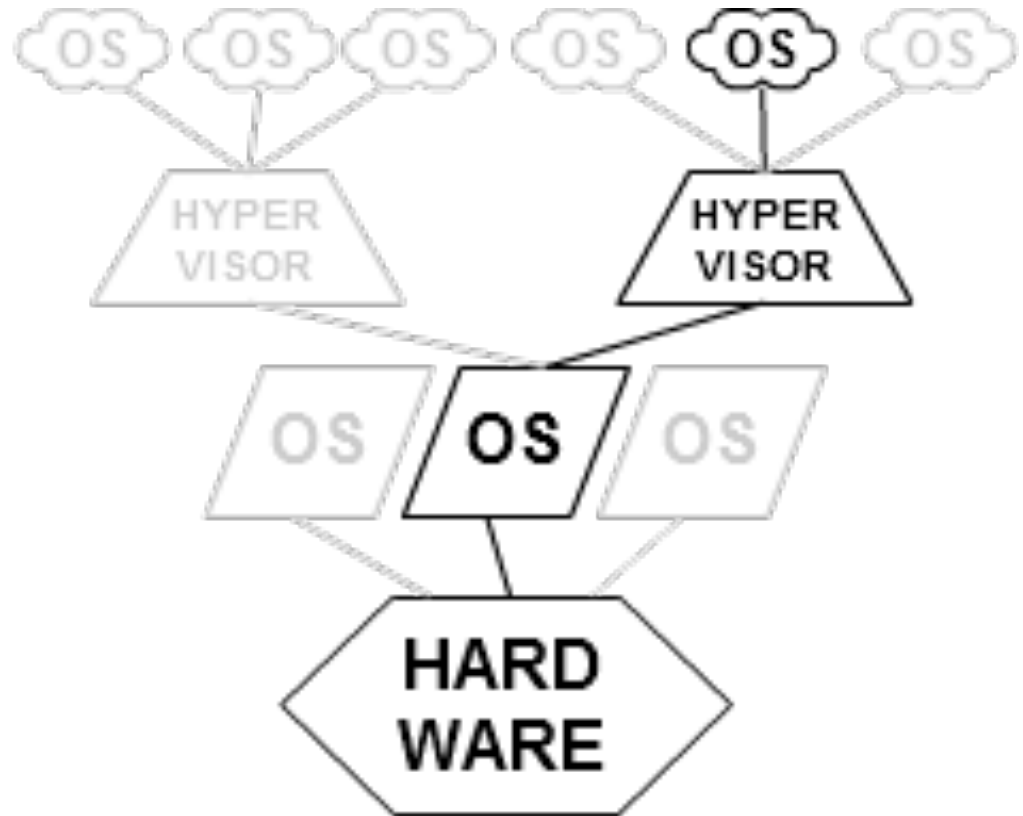
- Lean virtualization-centric kernel
- Service Console for agents and helper applications

Figure 2: Virtualization Architectures

RECAP: Cloud Concepts – Intro – Virtualization Overview



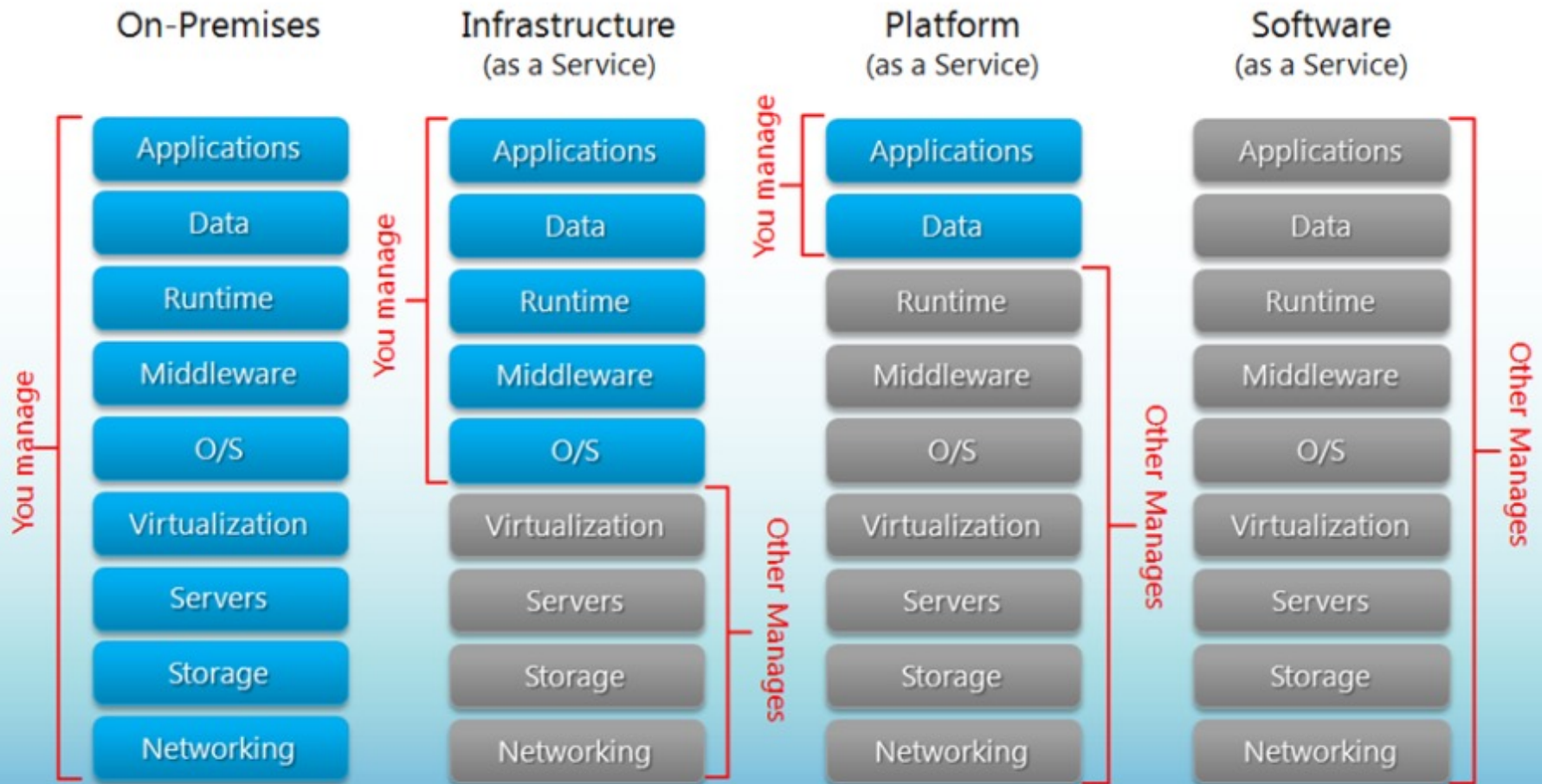
TYPE 1
native
(bare metal)



TYPE 2
hosted

RECAP: Cloud Concepts – Intro – IaaS, PaaS, SaaS

Separation of Responsibilities

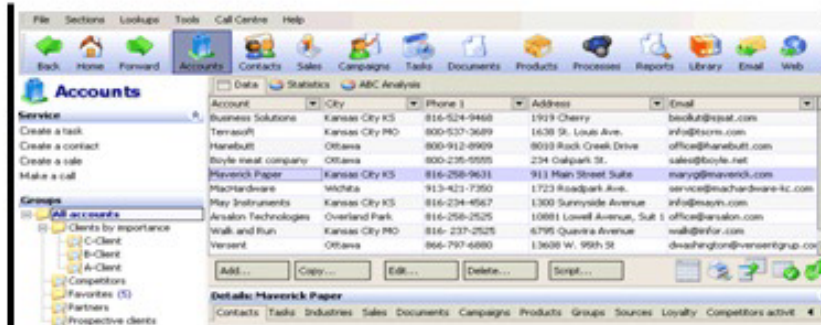


Copyright to

<http://blogs.technet.com/b/kevinremde/archive/2011/04/03/saas-paas-and-iaas-oh-my-quot-cloudy-april-quot-part-3.aspx>

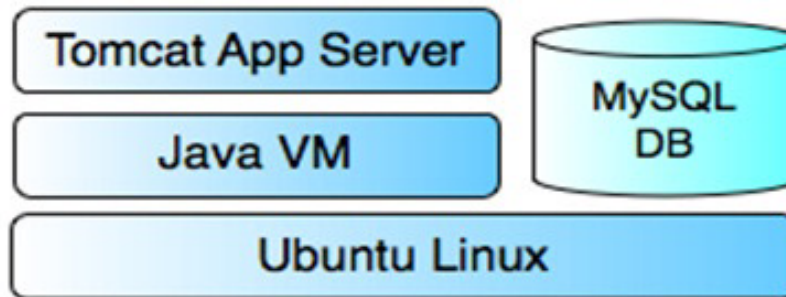
RECAP: Cloud Concepts – Intro – IaaS, PaaS, SaaS

SaaS



SalesForce.com, Google Apps

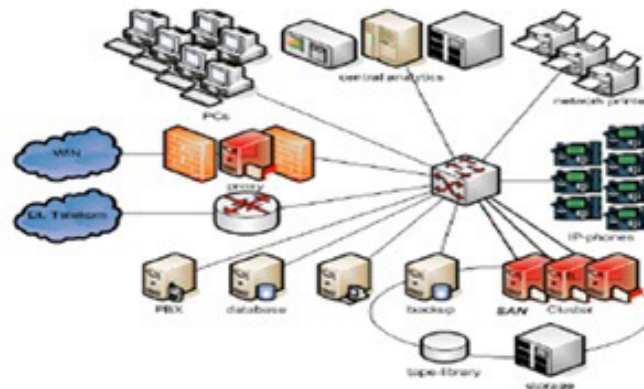
PaaS



Google App Engine for:
Java, Ruby, Python & GO

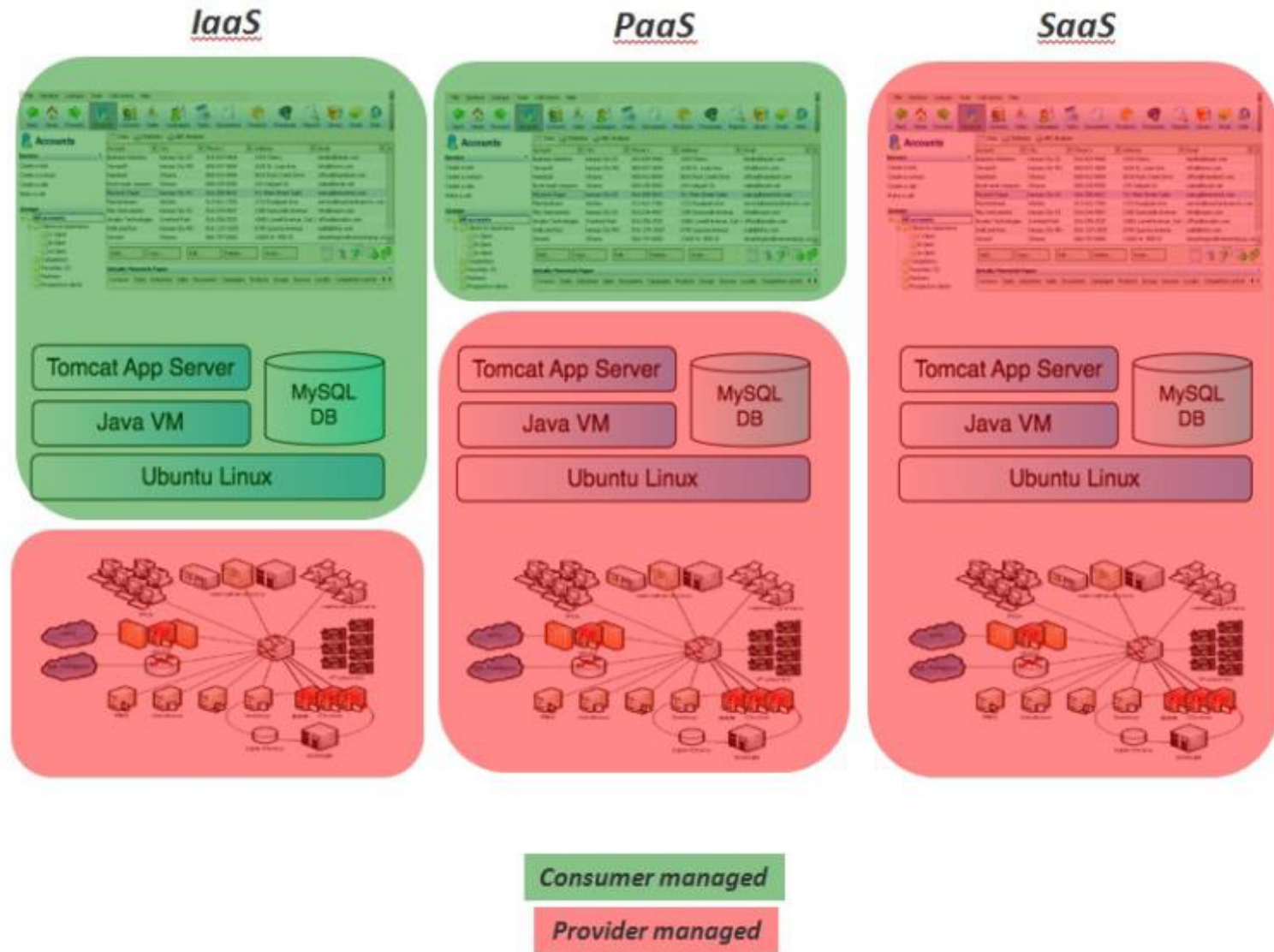
VMForce.com, MS Azure

IaaS



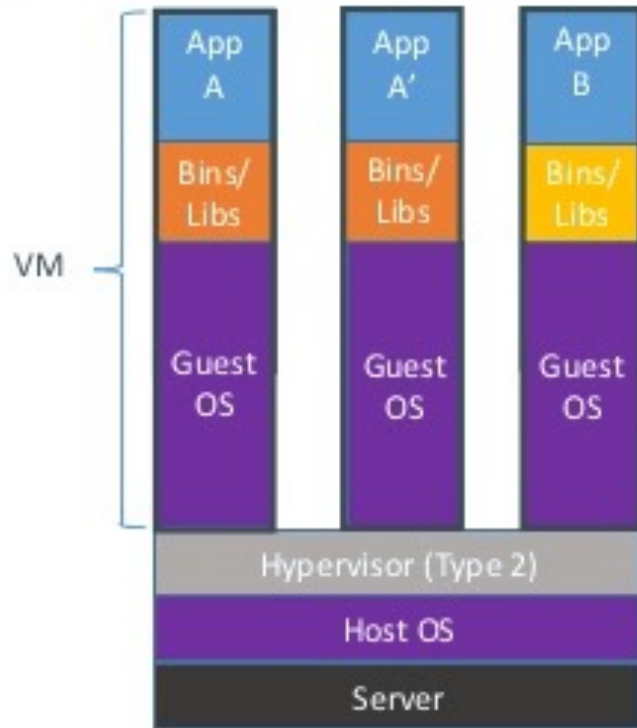
vCloud Express/Datacenter,
Amazon EC2

RECAP: Cloud Concepts – Intro – IaaS, PaaS, SaaS

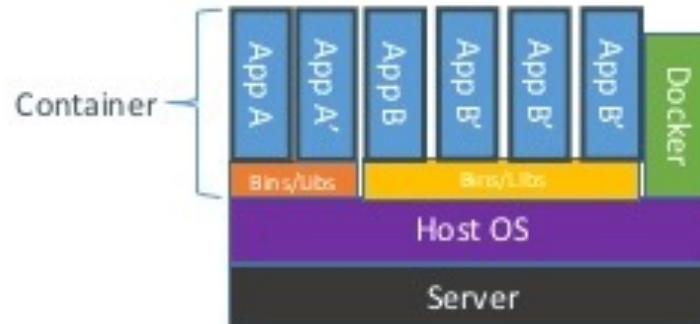


RECAP: Cloud vs. Micro-Services/Containers Concepts

Containers vs. VMs

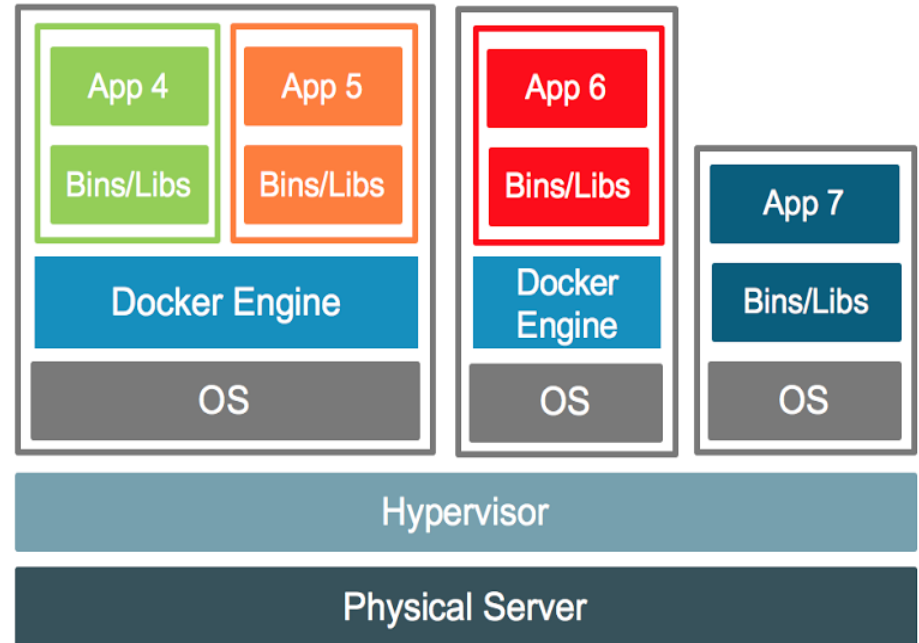
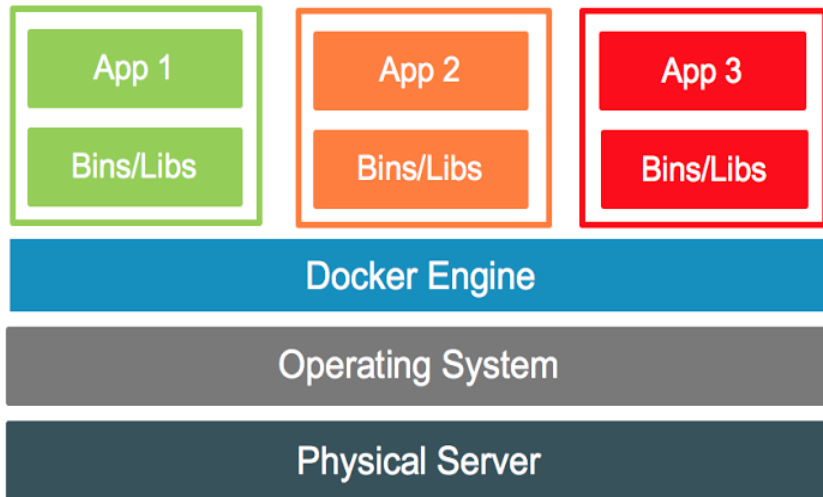


Containers are isolated, but share OS and, where appropriate, bins/libraries



RECAP: Cloud vs. Micro-Services/Containers Concepts

Your Datacenter or VPC



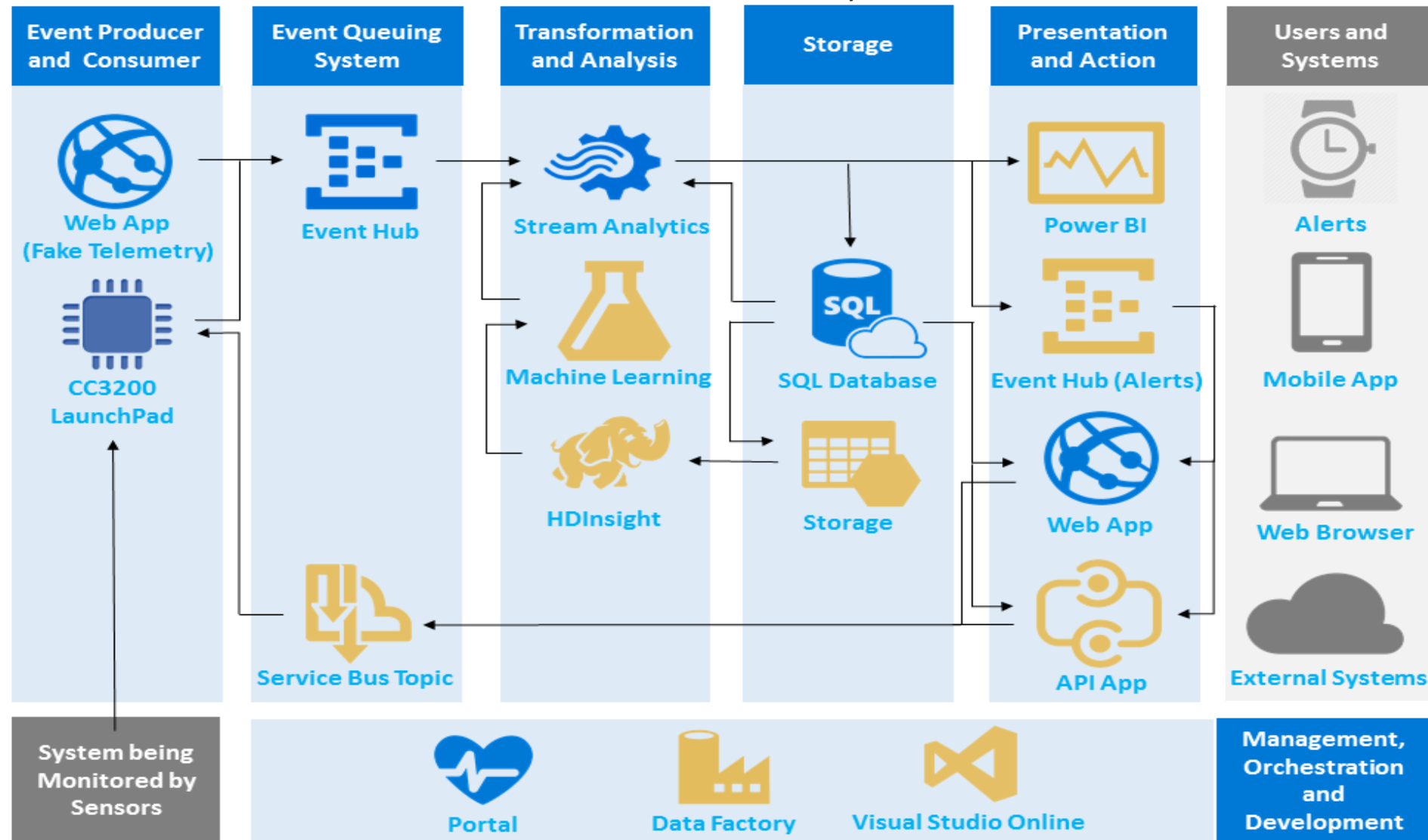
1. Embedded OS & IoT Architecture + Security

IoT Cloud Service Architecture + Security



1. Embedded OS & IoT Architecture + Security

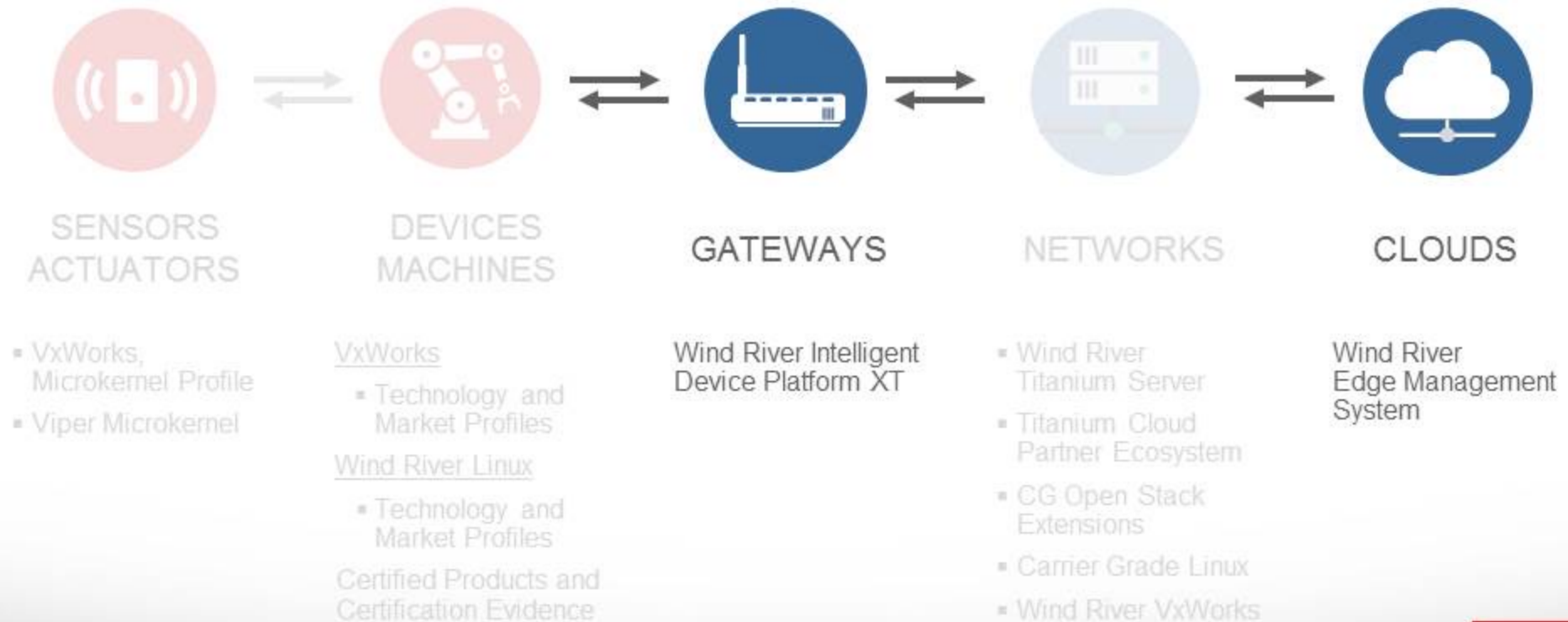
Embedded Hardware & OS within IoT Architecture + Security



1. Embedded OS & IoT Architecture + Security

IoT Cloud Service Architecture + Security – Intel WindRiver

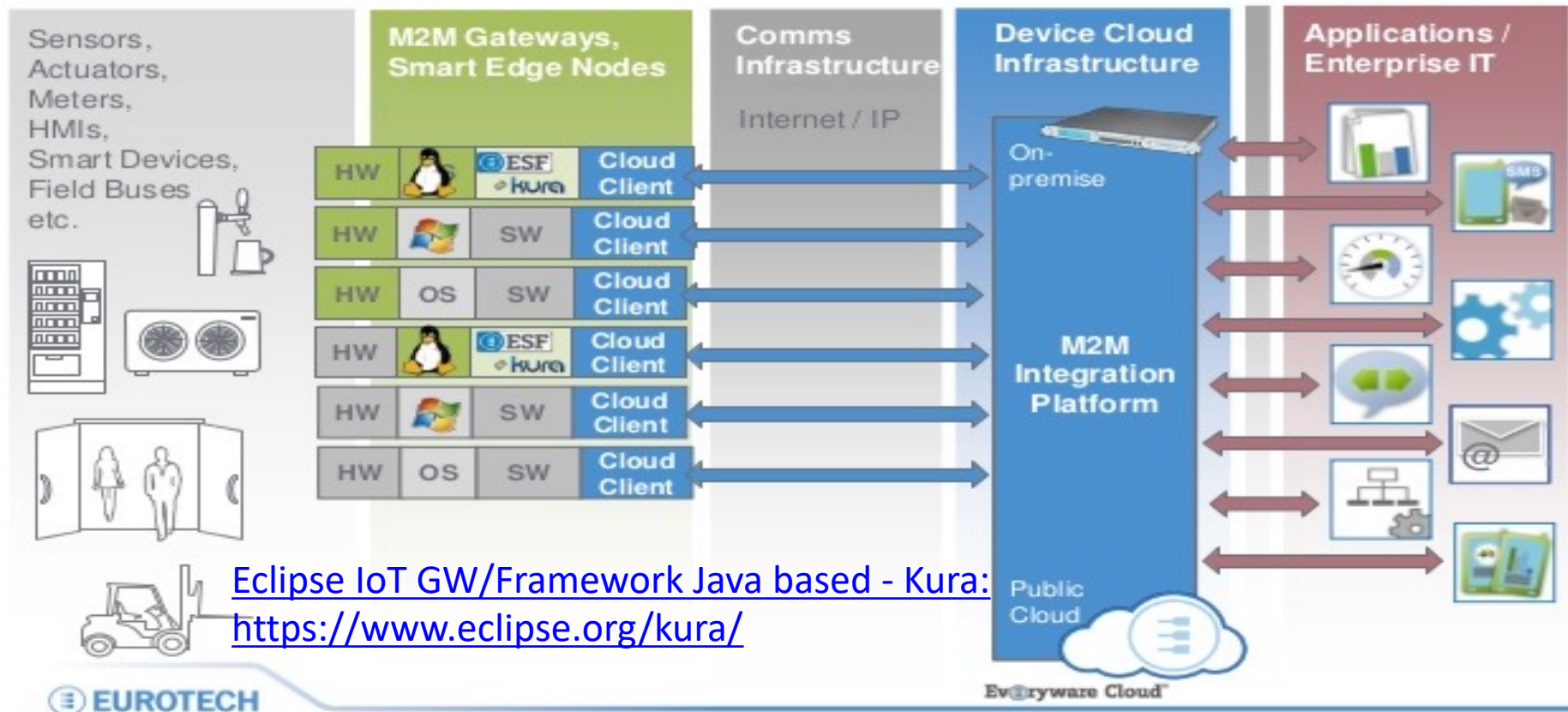
Wind River Helix Product Portfolio Applied to Topology



1. Embedded OS & IoT Architecture + Security

IoT Cloud Service Architecture + Security – Eurotech

IoT Architecture Typical Gateway Scenarios



1. Embedded OS & IoT Architecture + Security

IoT Cloud Service Architecture + Security – Cisco

Cisco Internet of Things Portfolio



Manufacturing



Mining



Energy-Utility



Oil and Gas



Transportation



City



Defense



SP/M2M

Connected Factory • Connected Train • City Safety and Security • Energy Distribution Automation • Connected Well

Industrial Switching



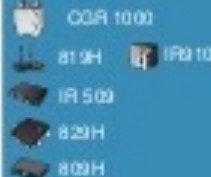
Industrial Routing



Industrial Wireless



Field Network



Embedded Networks



Connected Safety & Security



Digital Media



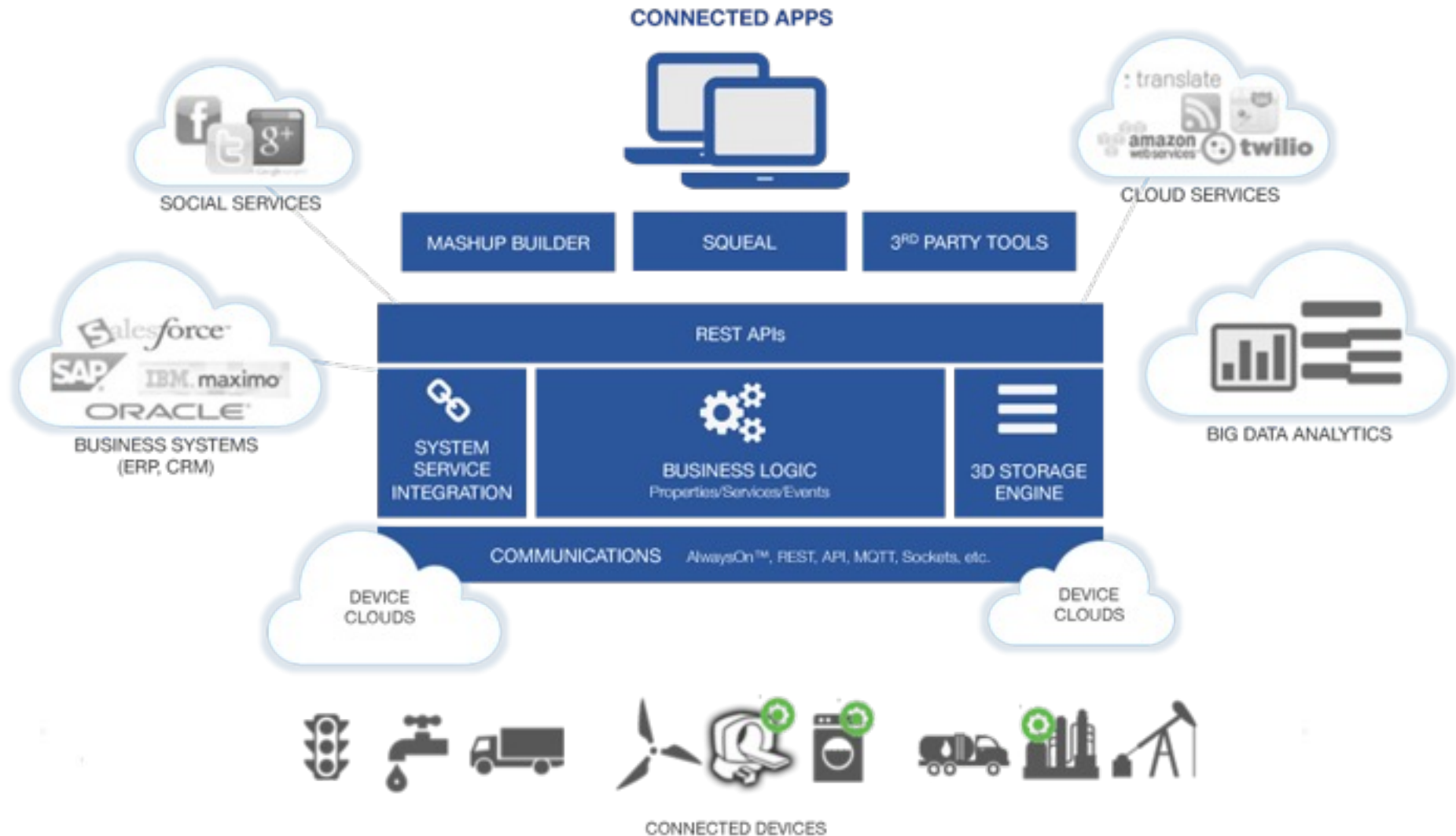
IoT Security

Application Enablement [Fog Computing/IOx]

Management

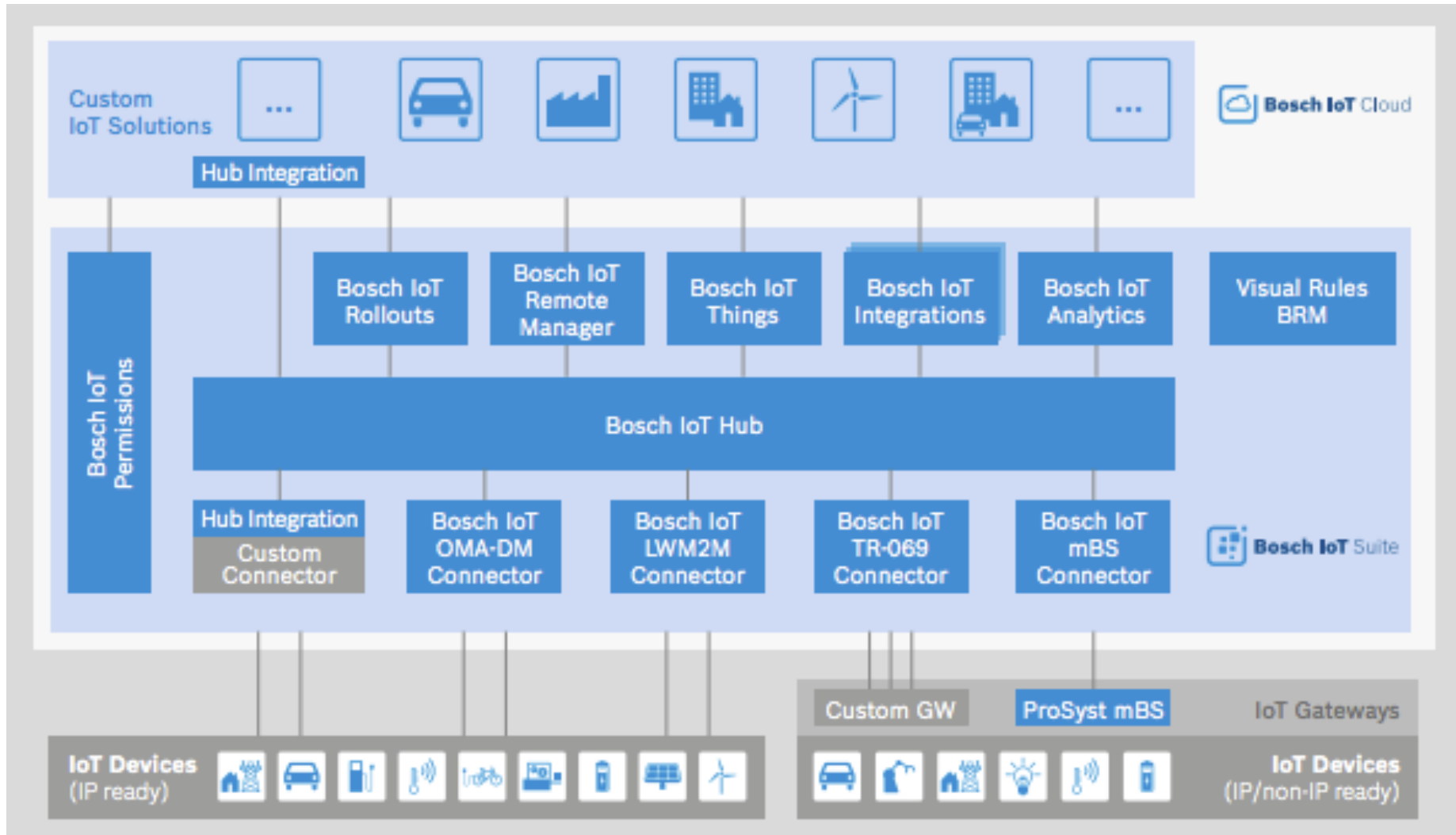
1. Embedded OS & IoT Architecture + Security

IoT Cloud Service Architecture + Security – Salesforce/c9.io - ThingsWorx



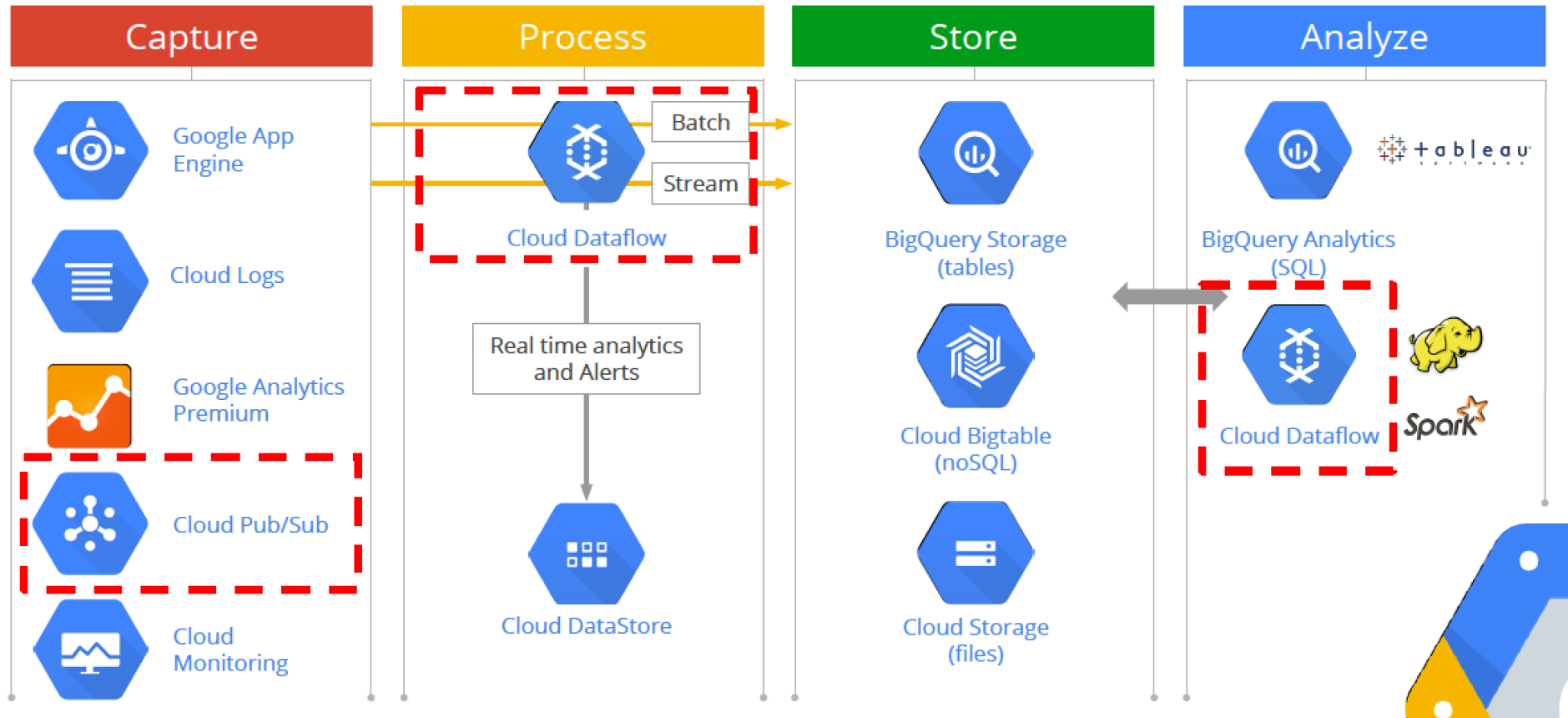
1. Embedded OS & IoT Architecture + Security

IoT Cloud Service Architecture + Security – Bosch



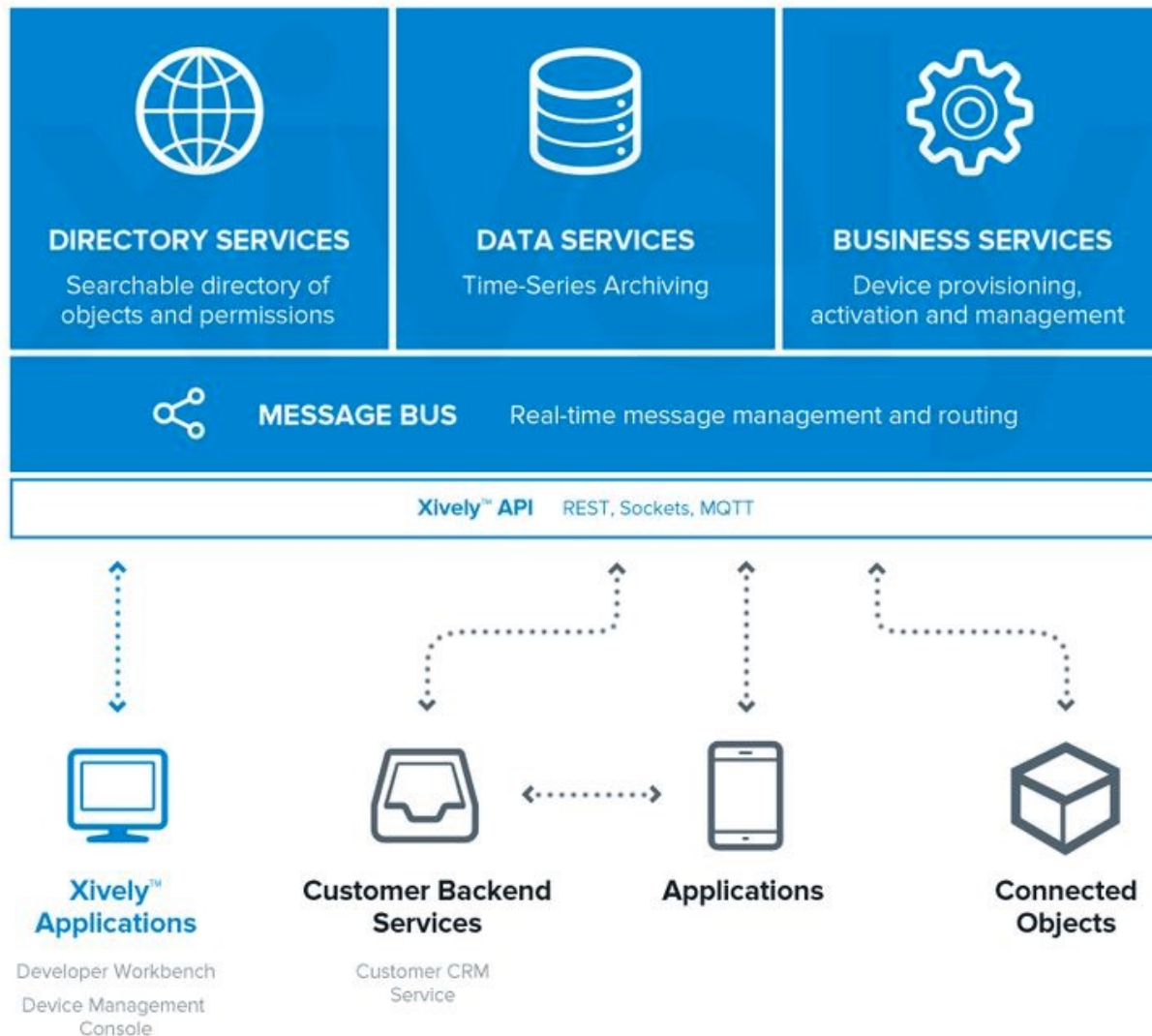
1. Embedded OS & IoT Architecture + Security

IoT Cloud Service Architecture + Security – Google



1. Embedded OS & IoT Architecture + Security

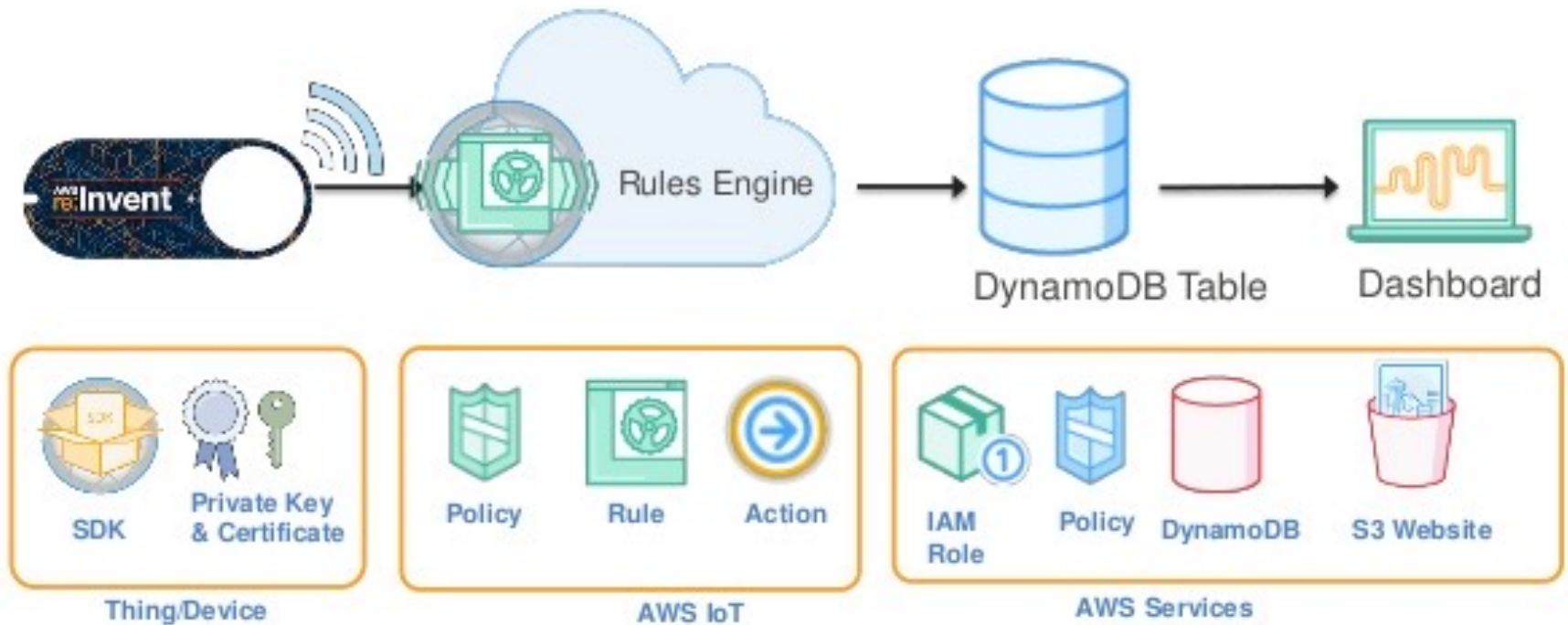
IoT Cloud Service Architecture + Security – Xively



1. Amazon IoT Architecture

IoT Cloud Service Architecture + Security – Amazon AWS IoT Cloud

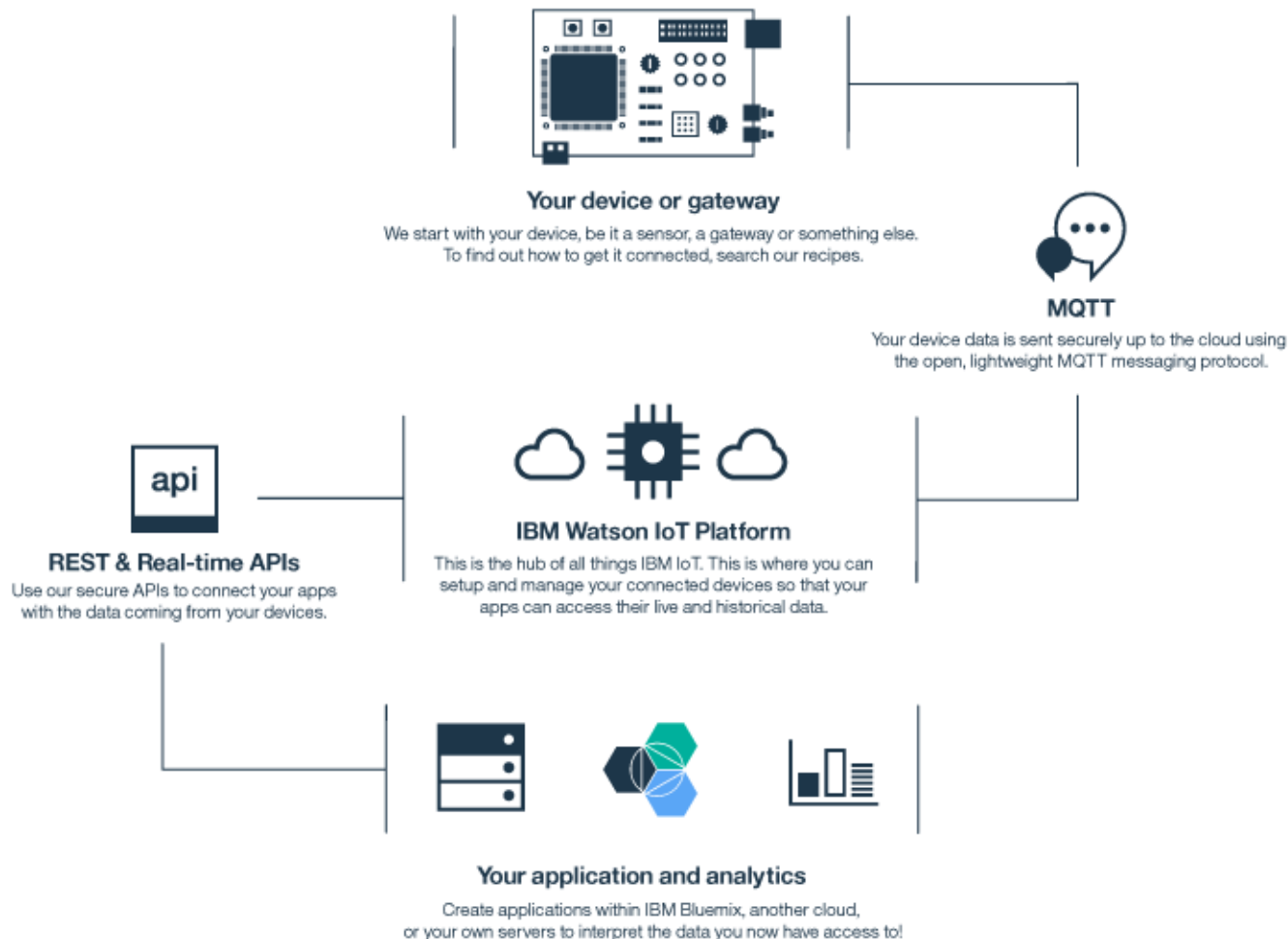
AWS IoT to Amazon DynamoDB to Dashboard



Select * from 'iotbutton/+'

1. IBM IoT Architecture

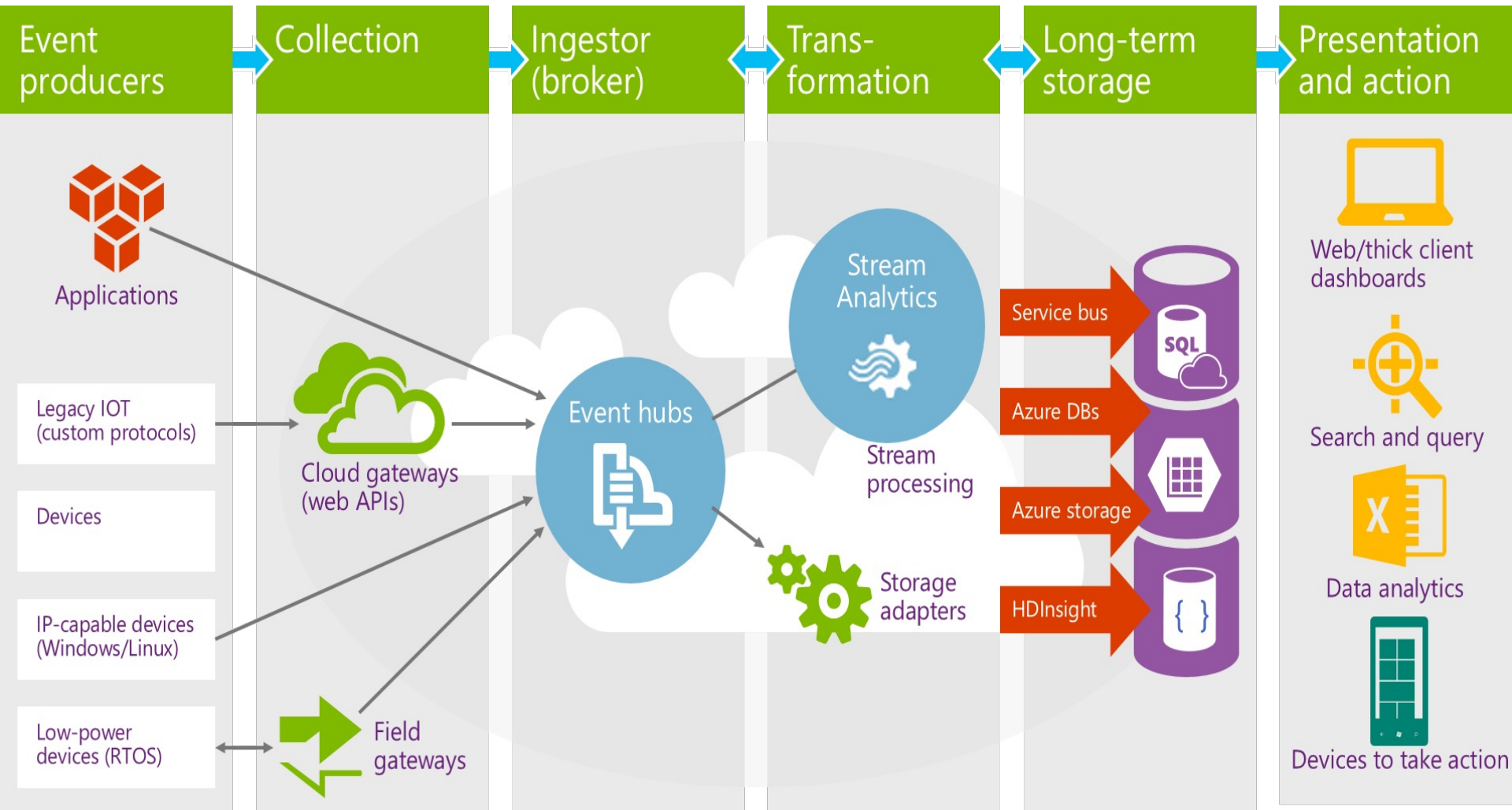
IoT Cloud Service Architecture + Security – IBM Watson IoT Platform



[Eclipse IoT GW/Framework Java based - Kura:](https://www.eclipse.org/kura/)
<https://www.eclipse.org/kura/>

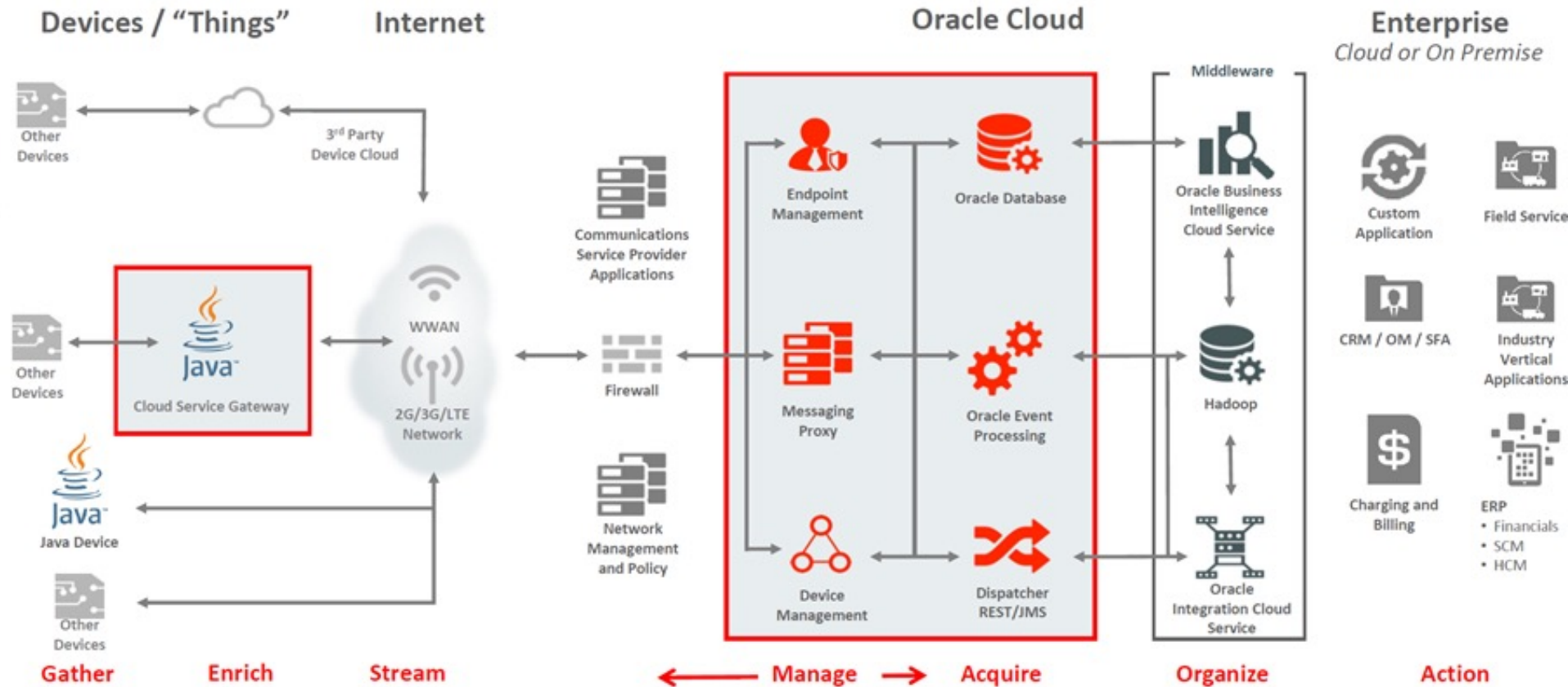
1. Microsoft IoT Architecture

IoT Cloud Service Architecture + Security – Microsoft Azure IoT



1. Oracle IoT Architecture

IoT Cloud Service Architecture + Security – Oracle IoT Cloud Service



1. Cloud Vendor Emphasis Comparison (Y 2013)

Gartner Cloud Comparison (Y 2013)

Summary of Major Vendor Emphasis

	Build Private Services		Services Delivered*			Private Offerings	
	Build Private Services	Deliver Services	IaaS	PaaS	SaaS	Enabling Tech.	Packaged Cloud
Amazon						— None —	— None —
salesforce.com						— None —	— None —
Google						— None —	— None —
Microsoft						— ● —	— ● —
IBM						— ● —	— ● —
VMware						— ● —	— ● —
Oracle						— ● —	— ● —
SAP						— None —	— None —
HP						— ● —	— ● —

Note: This is not an evaluation of capabilities, but rather of emphasis.



* The provider may offer public, community or virtual private services

1. IoT Overview

IoTS – Internet of Things Intro

IoT - Internet of Things

Applications: Smart Cities (e.g. Waste Collection Management), Distributed Computing in IoT Cloud/Silos, Semantic Sensor Network, Health Care App, E-Payment Solutions, Social Sensing Applications, Big Data / NoSQL processing, Crypto Blockchain Solutions.

**IoT
Sensors & Actuators**

Temperature,
Humidity,
Motion,
Camera,
Pollution - CO,
Noise, Infrared
– Actuators:
Engines, Plugs,
Boilers, etc.

Digital Serials
(UART, SPI,
I2C,...)
Radio -
ISO 14443 A/B
(prox. card/tag)
NFC / ISO 15693
& 18000
vicinity card/tag
Z-Wave,
ZigBee,...

Embedded Devices - Smart Objects

Embedded IoT GWs and
Nodes
PoC: Raspberry PI,
Arduino, BeagleBone
/Ninja blocks/Nitrogen..

Production: Eurotech,
Cisco/Tehnicolor, HMS-
Netbiter, Honeywell, etc.

Smart Objects
API
&
Device
Models/Types

Gateway
Services

**IoT Middleware
(M2M - Machine to Machine /
Internet Protocols)
Over GSM, Wi-Fi, etc....**

**REST & WS-
SOA**

**Web
Semantic
(Web 2.0 &
3.0) -
OWL/RDF**

CoAP

**MQTT

M2M –
MoM
Protocols,
Agents
based
Middleware**

**IoT Cloud / Back-End
Systems**

Oracle IoT CS

IBM BlueMix

Microsoft Azure

Amazon AWS
IoT

Public / Private
PaaS or IaaS
Clouds
(ThingsWorks,
Xively)

Open Source
Clouds / GRID /
Distributed and
Parallel Systems

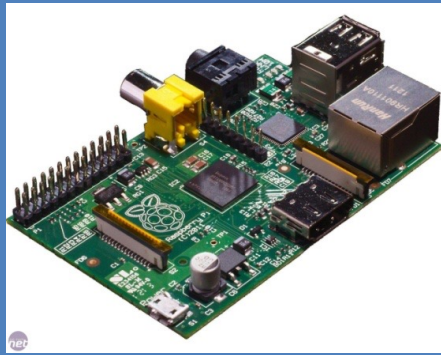
1. IoT Boards

IoTS – Internet of Things Smart Objects: IoT Nodes and Gateways (Dev Boards – Non-production)

Smart Object

Tools + Developers Web Page

Raspberry Pi



Micro-processor / Micro-controller: ARM | Linux based OS

Programming Languages / Dev Platforms:
Python, C/C++, ARM ASM or Java Embedded

Web (UK):

<http://www.raspberrypi.org/>

Arduino



Micro-processor / Micro-controller: ATmega

Programming Languages / Dev Platforms:
Arduino Programming Language (based on Wiring), Atmel ASM, C/C++

Web (Italy):

<http://www.arduino.cc/>

1. IoT Boards

IoTS – Internet of Things Smart Objects: IoT Nodes and Gateways (Dev Boards – Non-production)

Smart Object	Tools + Developers Web Page
Waspote 	Micro-processor / Micro-controller: ATmega Programming Languages / Dev Platforms: Waspote Scripts (looks like Atmel C/C++ combined with Java) Web (Spain): http://www.libelium.com/development/
BeagleBone Blocks / Ninja 	Micro-processor / Micro-controller: ARM Linux based OS: Ubuntu or Android 4.0 Programming Languages / Dev Platforms: C/C++, Java for Android, Ubuntu Programming Languages, ASM ARM Web (US): http://beagleboard.org/Products/BeagleBone
ESP-8266 ESP-01/ESP-12 / NodeMcu Lua ESP8266 CH340G WIFI 	Micro-processor / Micro-controller: ESP8266EX HAL/OS: Native Firmware with AT commands or Lua interpreter capabilities Programming Languages / Dev Platforms: Lua or AT commands Web: https://benlo.com/esp8266/esp8266QuickStart.html

1. IoT Mobile Smart Devices as Gateways

IoT – Internet of Things Smart Objects: Mobile Smart Devices (Dev & Production)

Mobile Convergence for M-App Development

HTML 5 / CSS 3 / JavaScript

WebKIT Engine / Similar Engine

Mobile IoT: Android/Java, iOS, C-Posix, JavaScript

ANDROID



BlackBerry



Windows
8/10 IoT
Core



Intel Tizen



SailfishOS & Ubuntu
Touch



Firefox
OS



Java

C/C++

Swift /
Objective-C

Java &
Native
C/C++

Web:
HTML
5

C#.NET

C/C++

Web:
HTML
5

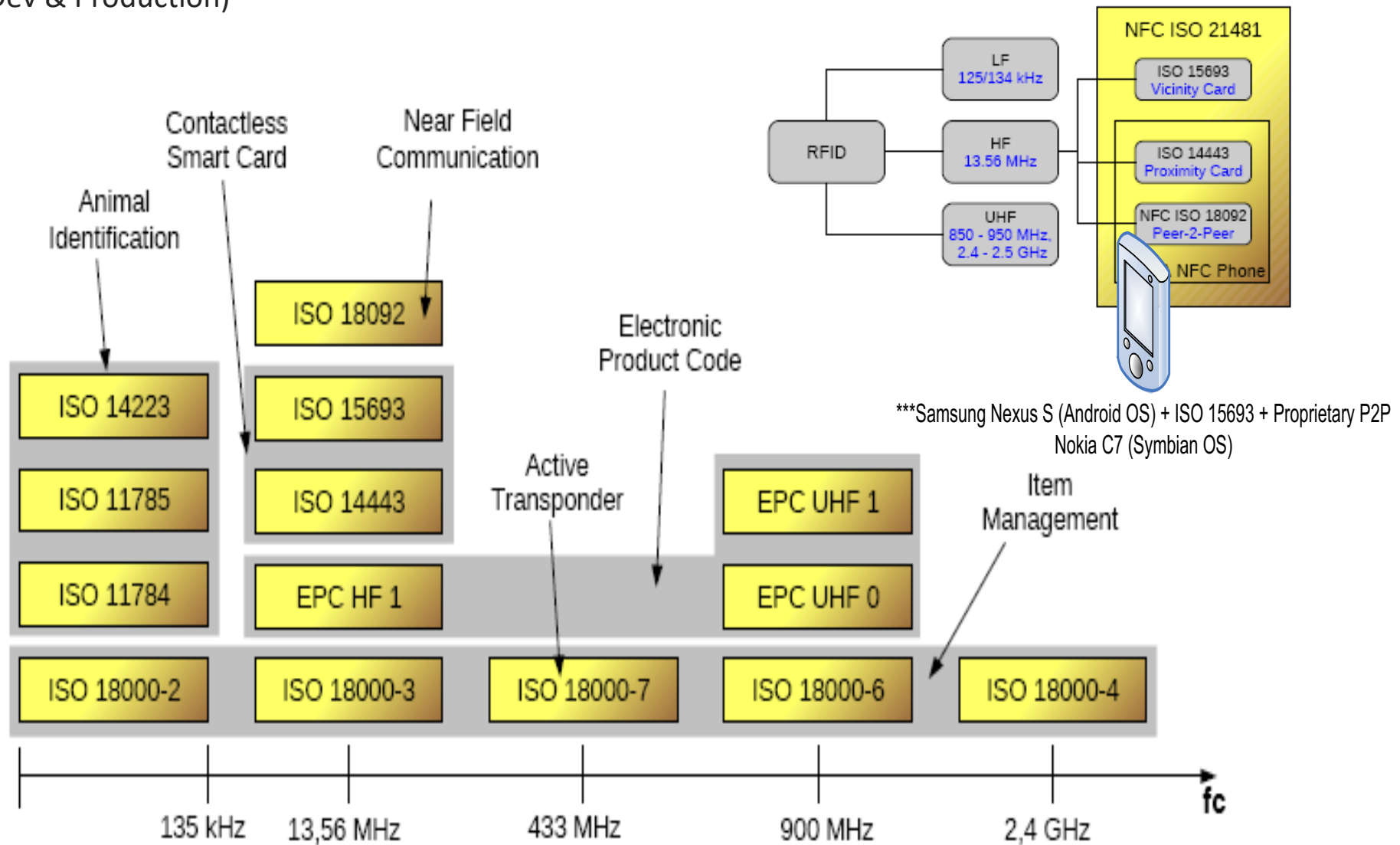
Native:
C++ Qt

Web:
QML
or
HTML
5

Web:
HML 5

1. IoT Mobile Vicinity and Proximity Techs

IoTS – Internet of Things Smart Objects: Mobile NFC | RFID Sensors & Standards – Vicinity vs. Proximity
(Dev & Production)



Embedded OS HW Details (for IoT GW) – Real Devices vs. Development Boards

No	Vendor/Provider	Model	OS / VM	CPU	RAM (MB)	Flash/SD/SSD (GB)	Main Promoted Dev Platforms/SDKs
1	Eurotech						
1.1		DynaGATE 15-10	Yocto Linux, Intel Gateway Solutions for IoT ready; It may support Java SE	Intel Quark SoC X1020D, 400Mhz	512	8 GB	1. Everyware Software Framework (ESF) support to extend capabilities further, including remote device management and Java programmability 2. POSIX C 3. Java SE-e
1.2		ReliaGATE 20-11	Wind River Linux (intel); it may support Java SE	Intel ATOM, E620 (T) 600MHz, E640(T) 1GHz, E660(T) 1.3GHz, E680(T) 1.6GHz	512	8 GB	1. Everyware Software Framework (ESF) Ready compatible with Eurotech's Everyware Device Cloud (EDC) 2. POSIX C 3. Java SE-e
1.3		ReliaGATE 15-10	Yocto Linux, Intel Gateway Solutions for IoT ready; It may support Java SE	Intel Quark SoC X1020D, 400Mhz	512	4 GB	1. Everyware Software Framework (ESF) support to extend capabilities further, including remote device management and Java programmability 2. POSIX C 3. Java SE-e

Embedded OS HW Details

2	IBM	it supports including Raspberry Pi and Arduino					
2.1		Intel IoT GW D200 (D50/100/200)	Wind River Linux (Intel) - IoT Ready (including MQTT distro); it may support Java SE	Intel Quark SoC X1020D	512	8 GB	1. Python with MQTT Paho lib connected to IBM Bluemix Cloud 2. POSIX C 3. Java SE-e 4. OSGi - Eclipse Kura / Lua
3	Netbiter / HMS						
3.1		Netbiter EC350	Linux BusyBox, Optional activation via Argos Cloud and it is Java SE and C enabled	ARM Cortex	1024	0,5 (512 MB - with restricted partitions))	1. HMS GW SW Stack -with ZeroMQ, Native SNMP, Native Protocol Agents (with samples for Java and C) and Netbiter Argos Cloud 2. POSIX C 3. Java SE-e
3.2		Netbiter EC310	Linux BusyBox, Optional activation via Argos Cloud and it is Java SE and C enabled	ARM Cortex	1024	0,5 (512 MB - with restricted partitions))	1. HMS GW SW Stack -with ZeroMQ, Native SNMP, Native Protocol Agents (with samples for Java and C) and Netbiter Argos Cloud 2. POSIX C 3. Java SE-e

Embedded OS HW Details

4	Thales (Gemalto)						
4.1		Cinterion Concept Board	Proprietary OS? With Java ME 3.2 embedded on EHS6 chip	ARM 11	10	0,01 GB (10 MB user space for JME Midlets)	1. Java ME 3.2
5	Microsoft (boards promoted by Microsoft)						
5.1		MinnowBoard MAX / Rpi 2 Model B+	Windows 10 IoT Core, Debian GNU/Linux, Yocto Project Compatible, Android 4.4 System	Intel Atom x64 E38xx Series SoC	1024 / 2048	8 GB	1. Microsoft C# .NET / C++ 2. POSIX C 3. Java SE-e 4. JavaScript on Node.js
5.2		Sharks Cove	Windows 8.1	Intel ATOM Z3735G	1024	16 GB	1. Microsoft C# .NET / C++
5.3		Qualcomm DragonBoard 410C	Linux based on Ubuntu, and planned support for Windows 10 IoT Core RT	ARM Cortex A7	1024	8 GB	1. POSIX C 2. C++ 3. Java SE-e 4. JavaScript - Node.js / Microsoft C#

Embedded OS HW Details

6	BoundaryDevice						
6.1		Nitrogen6 SabreLite	Android or Linux Ubuntu 14 Trusty T	ARM Cortex A9 dual-core	512	4 GB	1. POSIX C 2. C++ 3. Java SE-e 4. JavaScript - Node.js / Python
6.2		Nitrogen6 MAX	Android or Linux Ubuntu 14 Trusty T	ARM Cortex A9 quad-core	1024	8 GB	1. POSIX C 2. C++ 3. Java SE-e 4. JavaScript - Node.js / Python
7	Intel (GW HW provider - please see the above models)						
7.1		Intel Edison + Arduino Breakout Board	Yocto Linux, Arduino, Node.js (nodeRED)	Intel ATOM SoC	1024	8 GB	1. C-Arduino 2. Gnu C/C++
7.2		Galileo Gen 2	Yocto Linux	Intel Quark SoC X1000	256	8 GB	1. C-Arduino 2. Python 3. JavaScript - Node.js 4. POSIX C
7.3		Intel Galileo	Yocto Linux	Intel ATOM x64	256	8 GB	1. C-Arduino 2. Python 3. JavaScript - Node.js

Embedded OS HW Details

8	Beagle Boards						
8.1		BeagleBone Black	Debian Android Ubuntu Cloud9 IDE on Node.js w/ BoneScript library	ARM Cortex A8 Sitara	512	4 GB	1. Java SE-e 2. C/C++ 3. JavaScript - Node.js 4. Python
9	Raspberry						
9.1		Rpi Modelb / Raspberry Pi 2 Model B+	Raspbian Wheezy OS/Debian Linux Embedded	Broadcom BCM2835 SoC/BCM2836 SoC, single- Core/Quad- core ARM Cortex-A7	512 / 1024	4GB	1. Python 2. C/C++ 3. JavaScript - Node.js + node- Red 4. Java 8 SE-e 5. Java 8 ME-e
10	Arduino						
10.1		Arduino Yun	Linux eMbedded OS + Arduino Environment	Atheros AR9331 MIPS/ARM + ATmega32u4	64	2 GB	1. C-Arduino + Bridge Library - HTTP and Python

Embedded OS HW Details (for IoT Nodes)

1	ESP						
1.1		ESP8266 ESP-01 board	Proprietary OS/Firmware with Interpreters	106micro Diamond Standard core (LX3)	64 KB Instruction 96 KB data	64 KB	1. AT/HTTP Commands 2. Lua (with NodeMCU Lua)
1.2		NodeMcu Lua WIFI IoT dev board based ESP8266 module	Proprietary OS/Firmware with Interpreters	106micro Diamond Standard core (LX3)	64 KB Instruction 96 KB data	64 KB	1. AT/HTTP Commands 2. Lua (with NodeMCU Lua)
1.2		ESP8266 ESP-12 board	Proprietary OS/Firmware with Interpreters	106micro Diamond Standard core (LX3)	?	?	1. HTTP Commands 2. Lua (with NodeMCU Lua)
2	NXP (NXP acquired Freescale & Qualcomm acquired NXP)						
2.1		ARM mbed NXP LPC1768 Development Board	?	ARM Cortex-M3	32 KB	512 KB	1. ARM mbed C and Library
2.2		Freescale FRDM-K64	?	ARM Cortex-M3	256 KB	1024 KB	1. ARM mbed C

1. IoT Communications Protocols HTTP-REST, CoAP, MQTT

1.Infrastructure (e.g.: 6LowPAN, IPv4/IPv6, RPL, ZigBee, Z-Wave, SigFox, LoraWAN, GSM-3G/4G, WiMAX, GSM 5G IoT-NB)

2.Identification (e.g.: EPC, uCode, IPv6, URIs)

3.Comms / Transport (e.g.: Wi-Fi, Bluetooth, BLE, LPWAN, ZigBee, Z-Wave, SigFox, LoraWAN, GSM-3G/4G. WiMAX)

4.Discovery (e.g.: Physical Web, mDNS, DNS-SD)

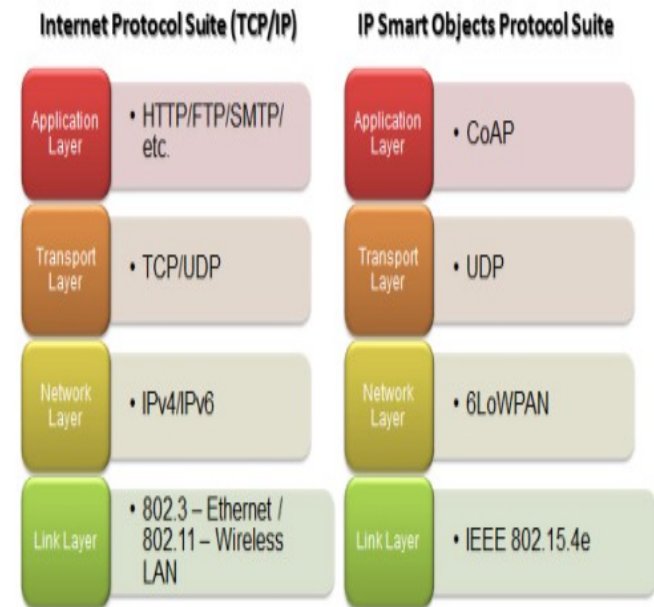
5.Data Protocols (e.g.: MQTT, CoAP/HTTP-REST, AMQP, WebSocket, LWM2M, ModBus)

6.Device Management (e.g.: TR-069, OMA-DM)

7.Semantic (e.g.: JSON-LD, Web Thing Model)

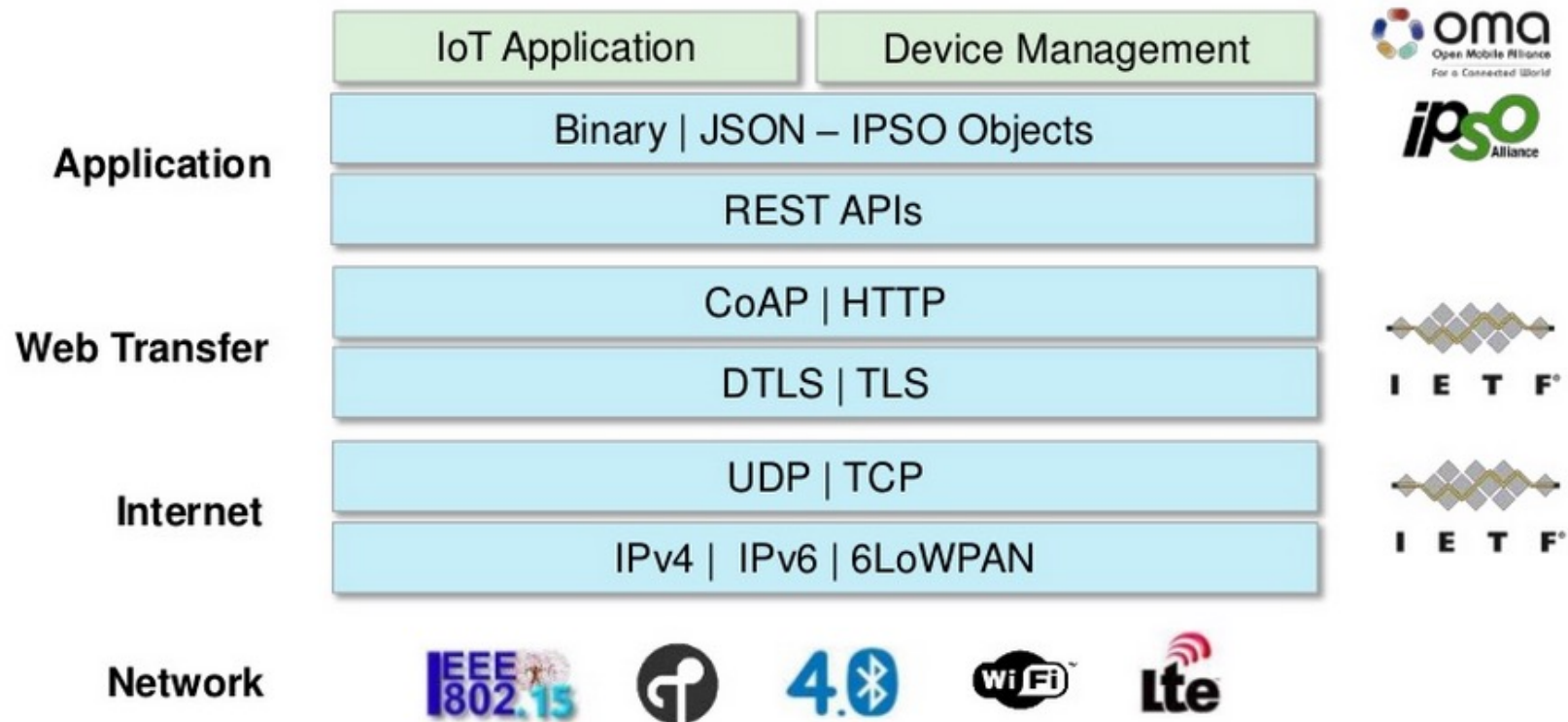
8.Multi-layer Frameworks (e.g.: Alljoyn, IoTivity, Weave, Homekit, etc.) – e.g. SmartHouse: Amazon Alexa, Google Home, Apple HomeKit,...

9. Security – Transversal (e.g. Global Platform, [Open Trust Protocol](#) (OTrP) in TEE, X509, IETF, IEEE, W3C ... - **ASN.1 DER vs. COSE/CBOR**)



1. IoT Communications Protocols HTTP-REST, CoAP, MQTT

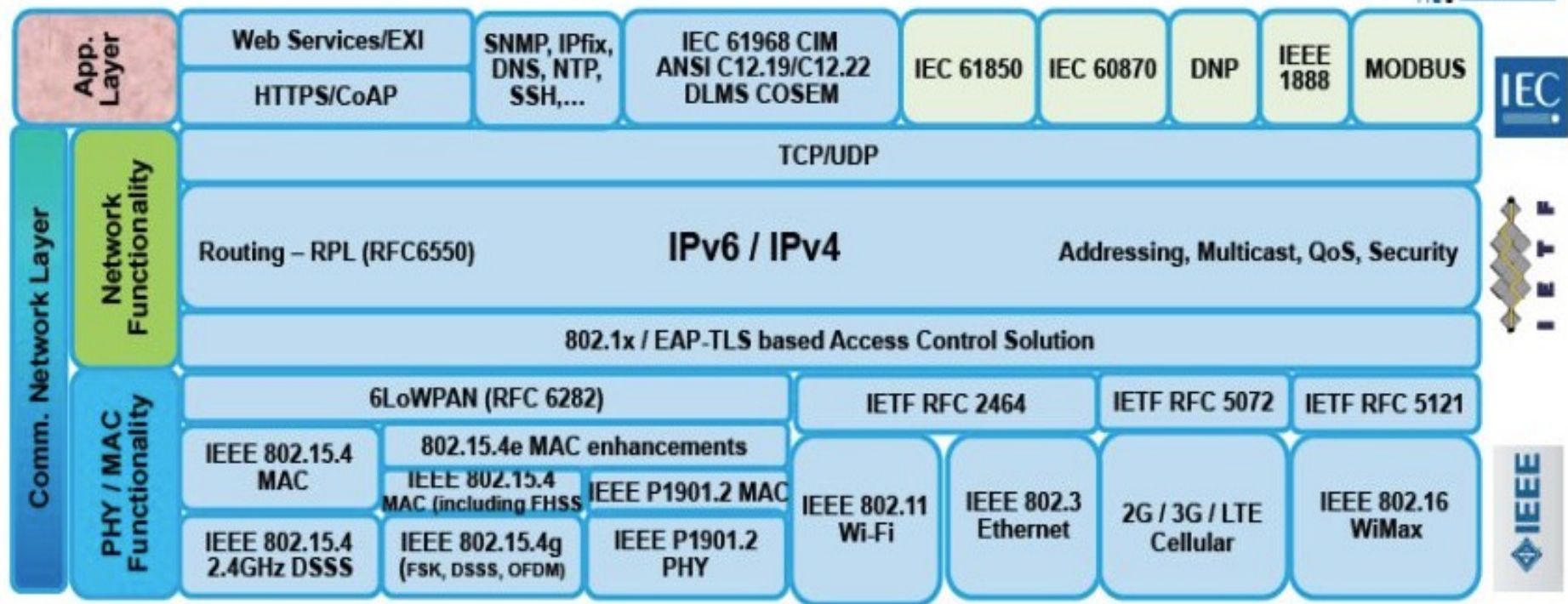
Remember the I in IoT!



ARM

1. IoT Communications Protocols HTTP-REST, CoAP, MQTT

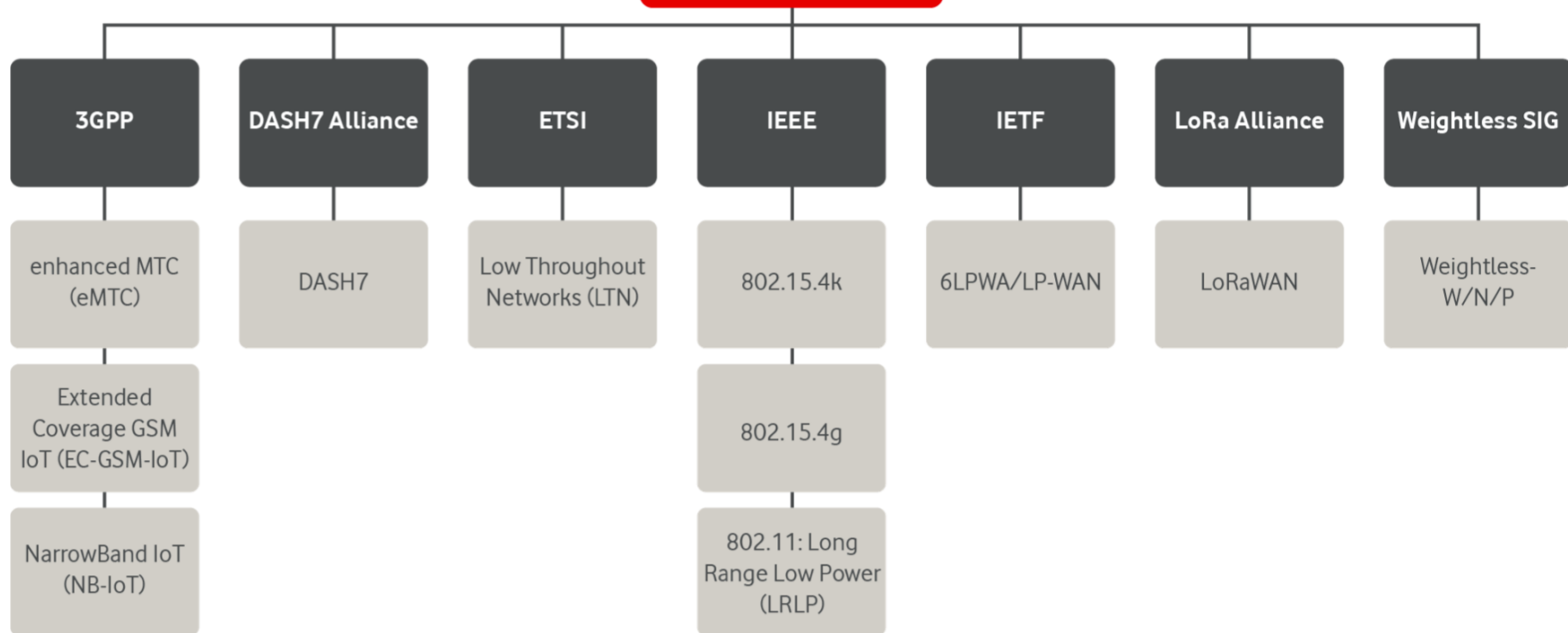
Open Standards Reference Model



David E Culler Open Standards Reference Model

IOT COMMUNICATIONS

LPWA standards

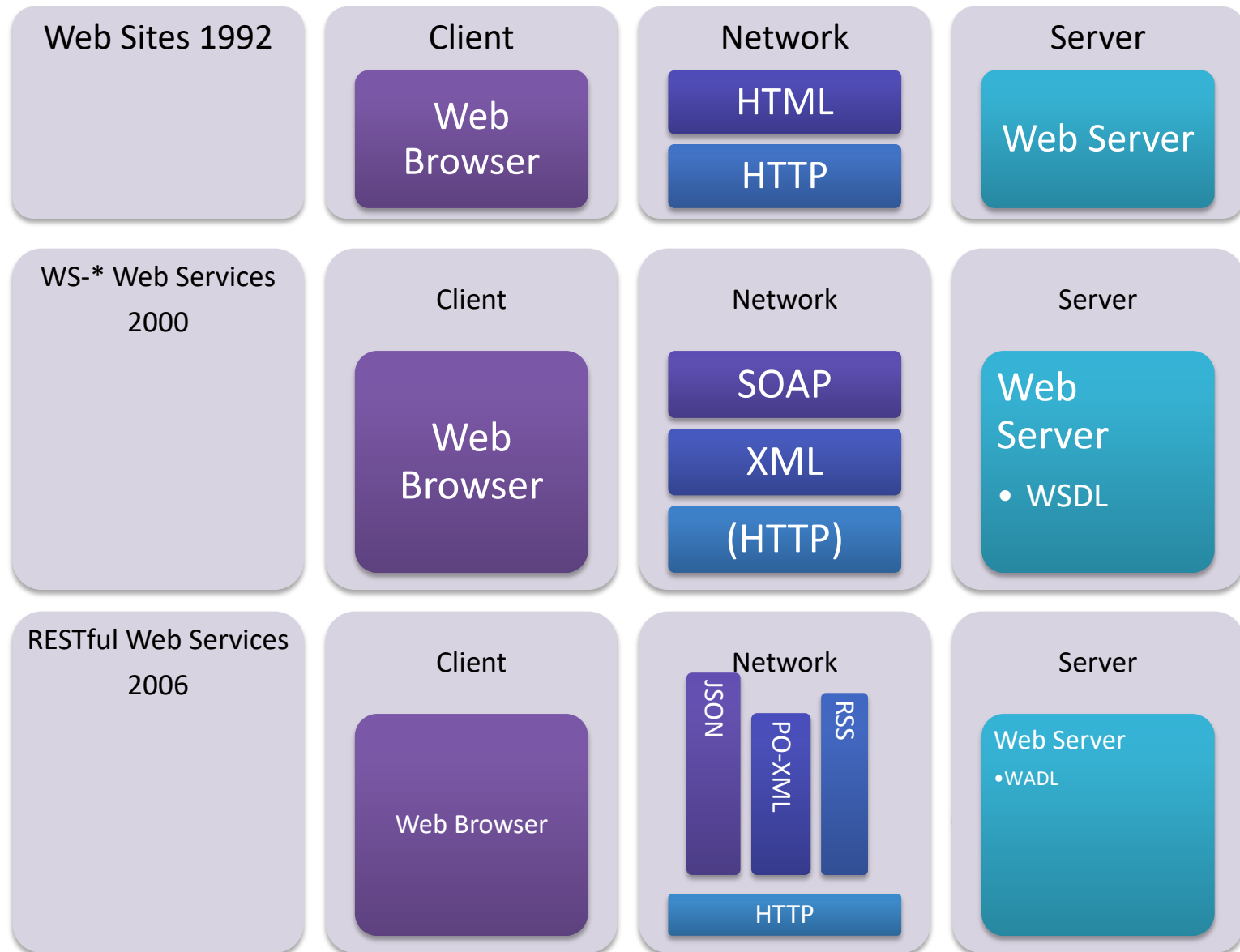


IOT COMMUNICATIONS

	LoRa	Sigfox	NB-IoT
Coverage	160dB	157dB	164dB
Technology	Proprietary	Proprietary	Open LTE
Spectrum	Unlicensed	Unlicensed	Licensed (LTE/any)
Duty Cycle restrictions	Yes	Yes	No
Output power restrictions	Yes (14dBm = 25mW)	Yes (14dBm = 25mW)	No (23dBm = 200mW)
Downlink data rate	<0.1kbps	<10kbps	0.5 – 200kbps
Uplink data rate	<0.1kbps	<10kbps	0.3 – 180kbps
Battery life (200b/day)	10+ years	10+ years	15+ years
Module cost	<\$10 (2016)	<\$10 (2016)	\$6 (2017) to <\$2 (2020)
Security	Low	Low	Very high

Fig 1. Key technical specifications for NB-IoT (from R1-157741, Summary of NB-IoT evaluations results, 3GPP RAN1#83, Nov 2015), Sigfox, and LoRa (from LoRaWAN: a technical overview of LoRa and LoRaWAN, LoRa Alliance, Nov 2015).

1. IoT Communications Protocols HTTP-REST, CoAP, MQTT



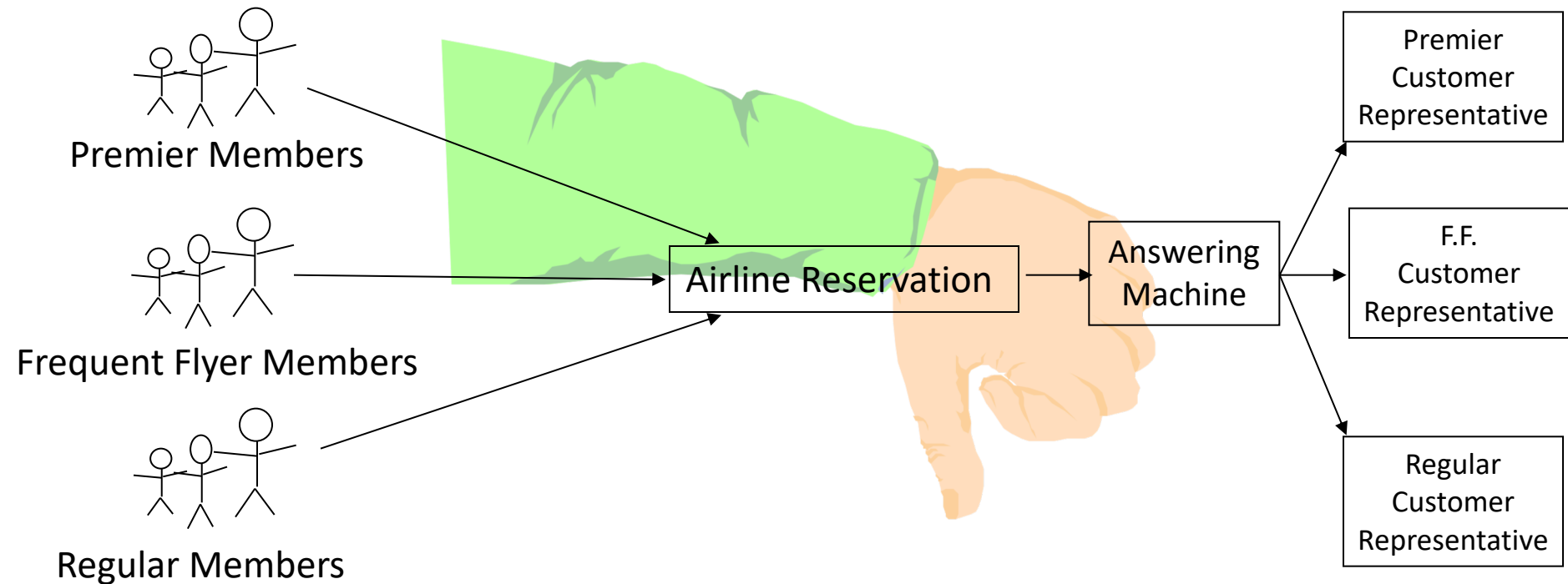
1. IoT Communications Protocols HTTP-REST, CoAP, MQTT

WS-* vs. REST Comparison



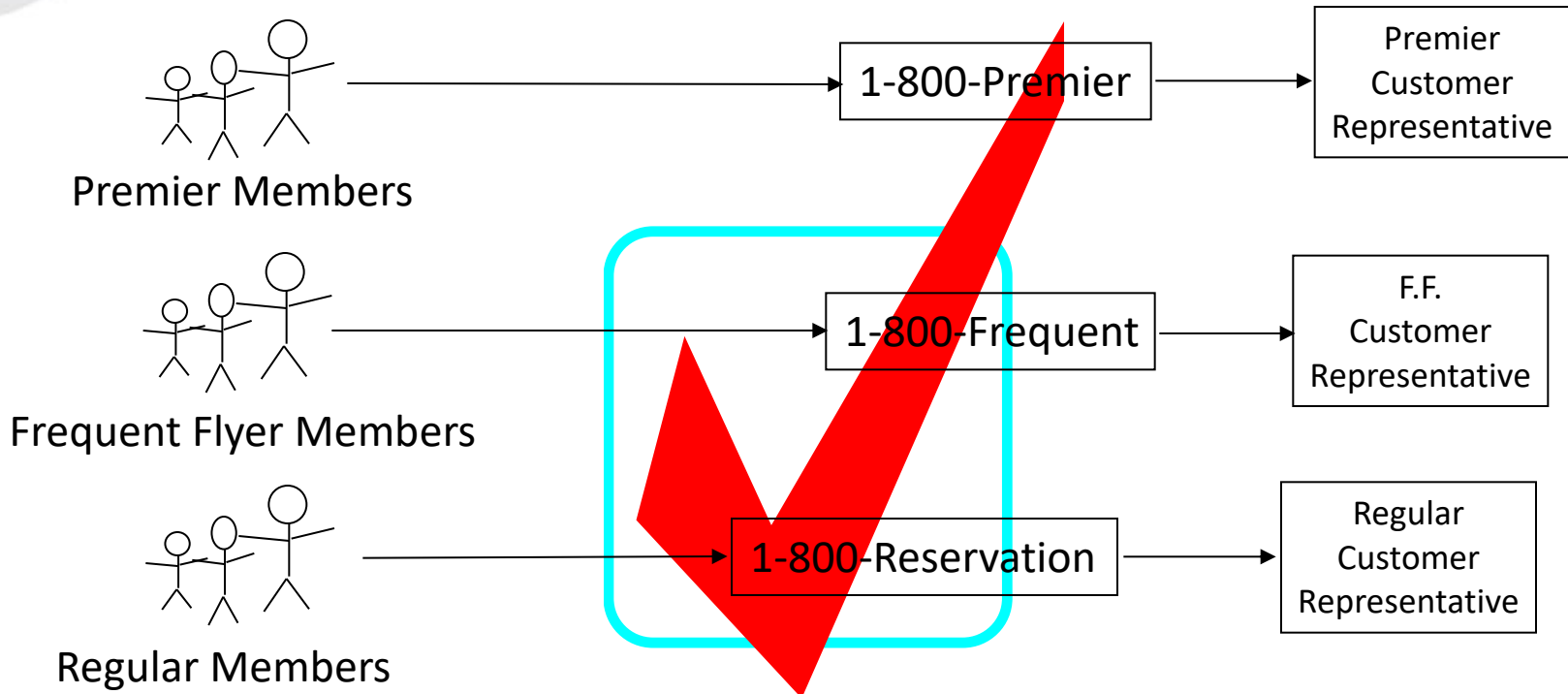
1. IoT Communications Protocols HTTP-REST vs. WS Recap

This Ain't the REST Design Pattern



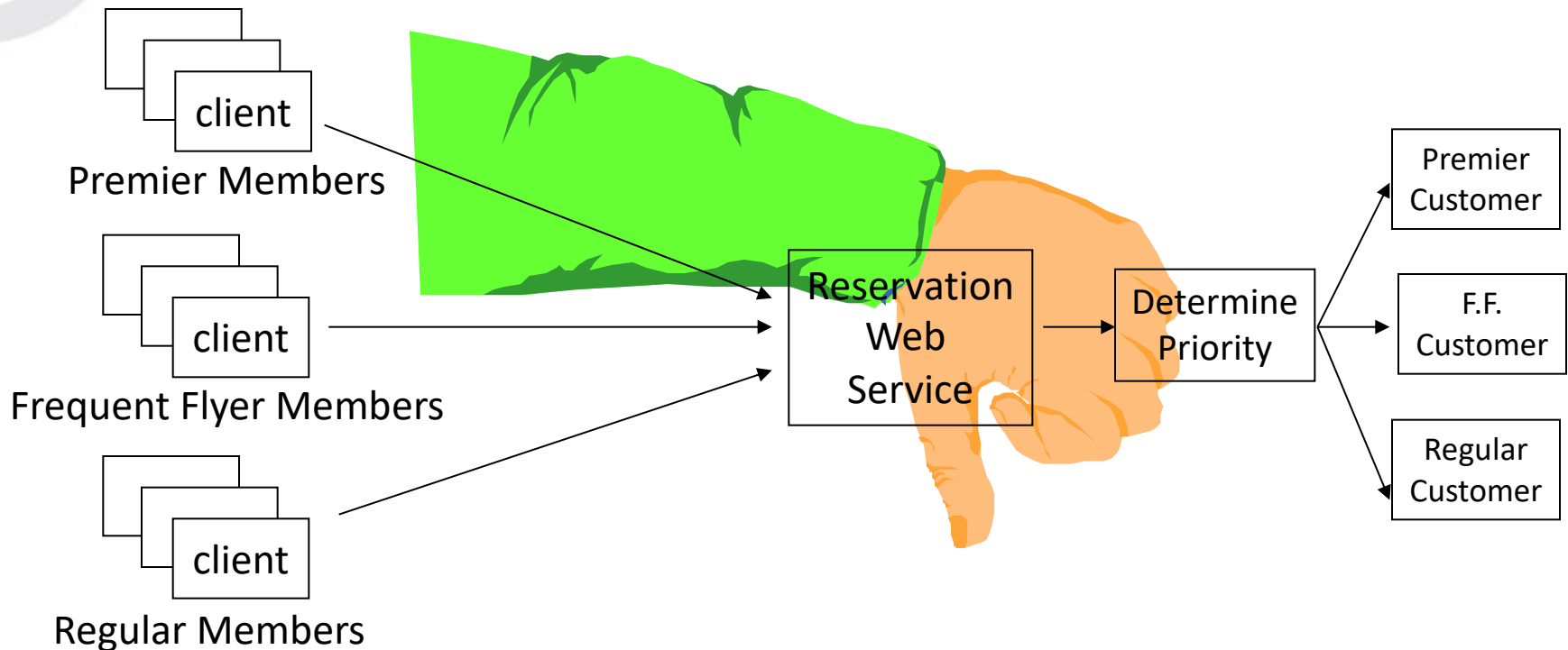
1. IoT Communications Protocols HTTP-REST vs. WS Recap

This is the REST Design Pattern



1. IoT Communications Protocols HTTP-REST vs. WS Recap

This ain't the REST Design Pattern

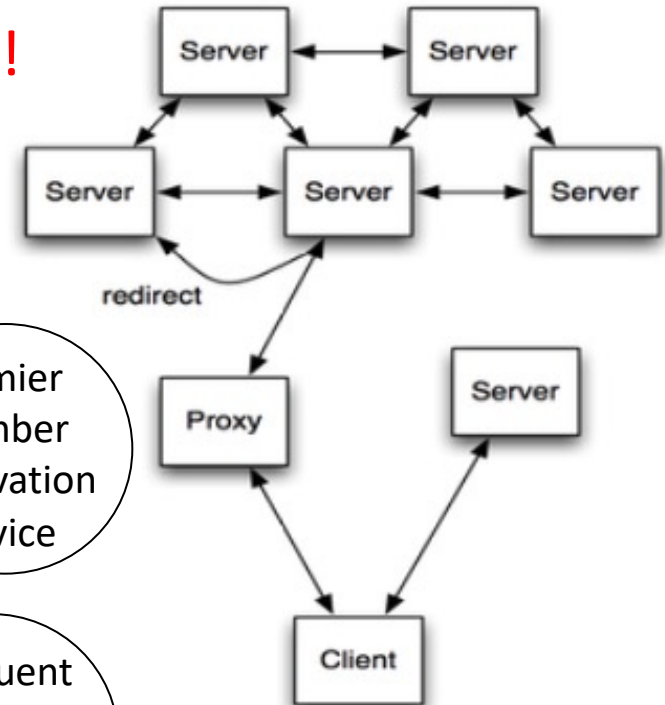
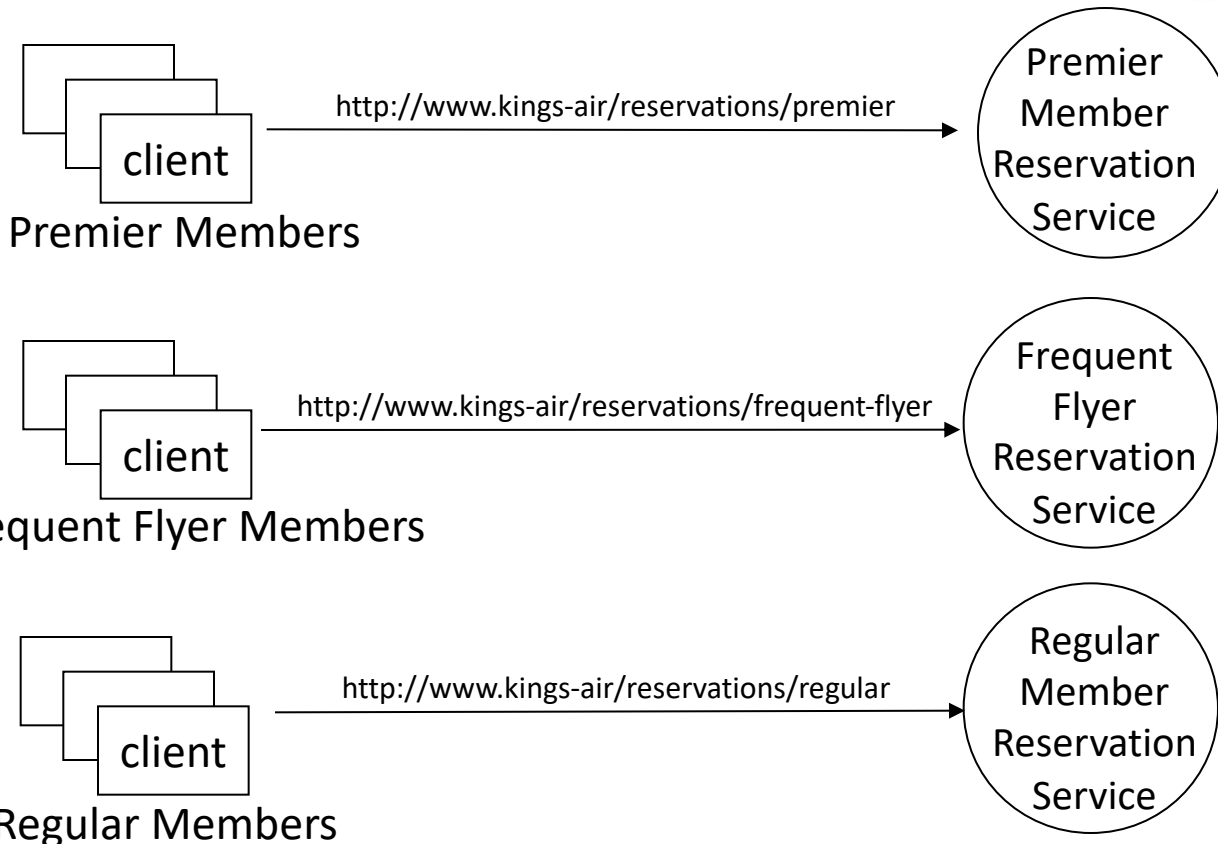


1. IoT Communications Protocols HTTP-REST vs. WS Recap

REST Approach in Web Architecture:

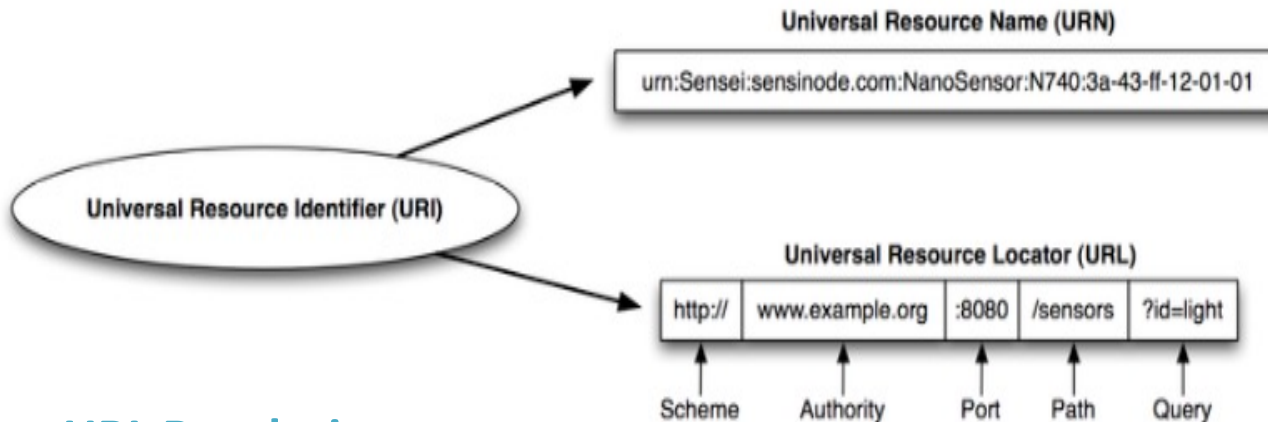
URLs are Cheap! Use Them!

The airline provides several URLs - one URL for premier members, a different URL for frequent flyers, and still another for regular customers.

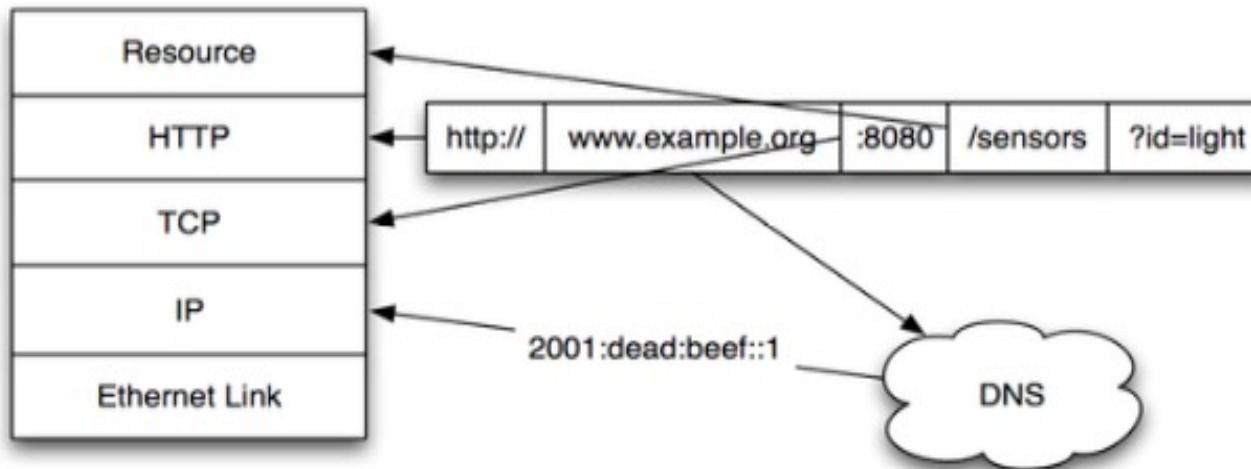


1. IoT Communications Protocols HTTP-REST Recap

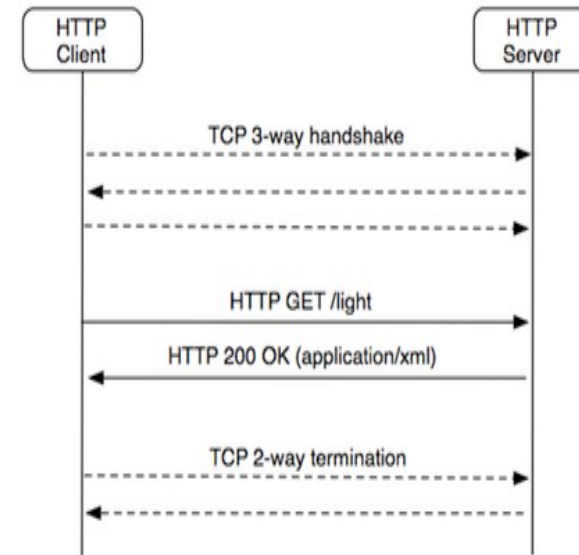
Web Naming



URL Resolution



An HTTP Request



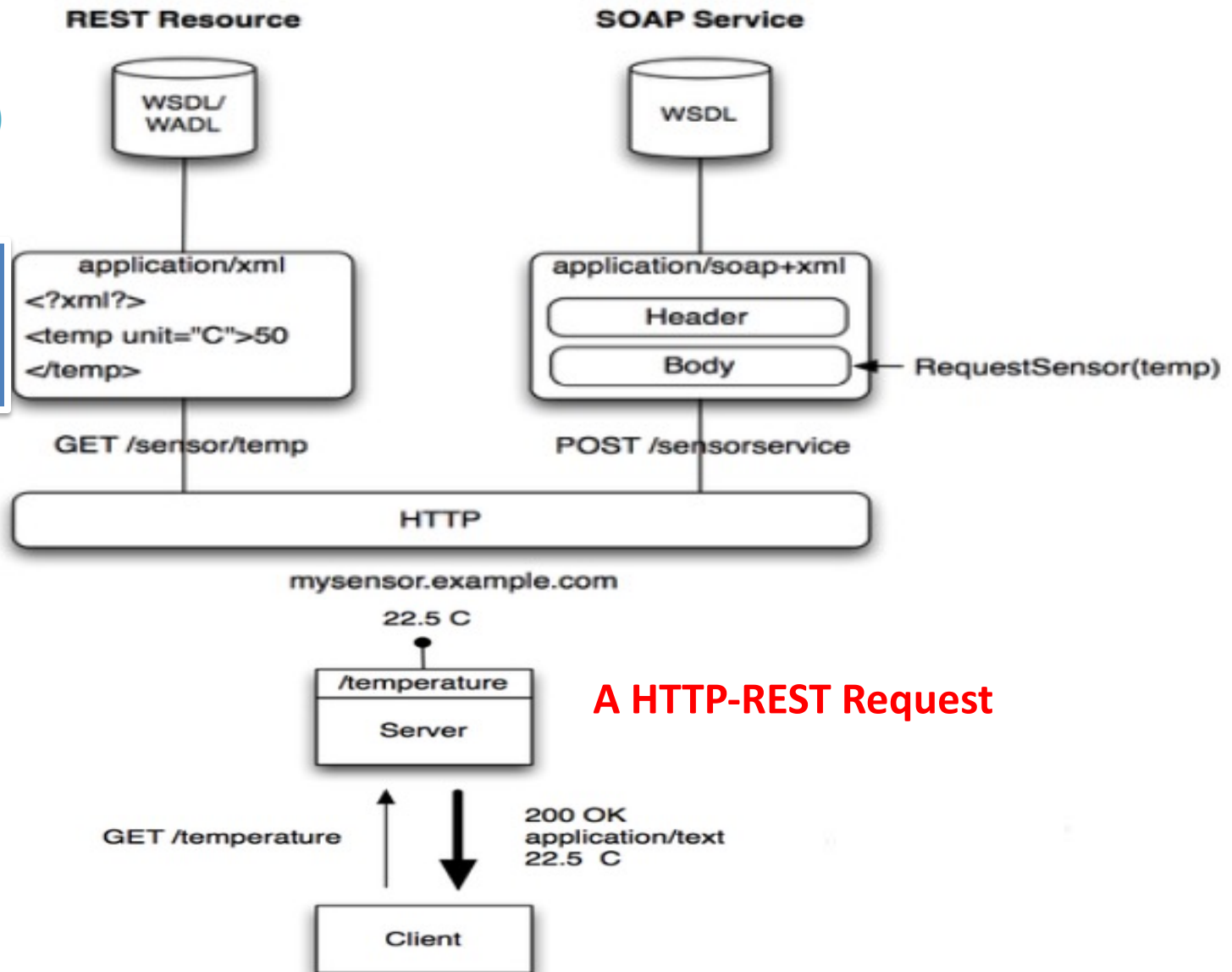
See RFC2616 - Hypertext Transfer Protocol v1.1

1. IoT Communications Protocols HTTP-REST Recap

Web Paradigms:

REST Resource vs. SOAP Service (WS-*)

```
application/json
{
  "temp":50, "unit": "C"
}
```



1. IoT Communications Protocols HTTP-REST vs. WS Recap

- Simple **web service** as an example: querying a phonebook application for the details of a given user
- Using Web Services and SOAP, the request would look something like this:

```
<?xml version="1.0"?>
```

```
<soap:Envelope
```

```
  xmlns:soap="http://www.w3.org/2001/12/soap-envelope"
```

```
  soap:encodingStyle="http://www.w3.org/2001/12/soap-encoding">
```

```
    <soap:body
```

```
      pb="http://www.acme.com/phonebook">
```

```
        <pb:GetUserDetails>
```

```
          <pb:UserID>12345</pb:UserID>
```

```
        </pb:GetUserDetails>
```

```
      </soap:Body>
```

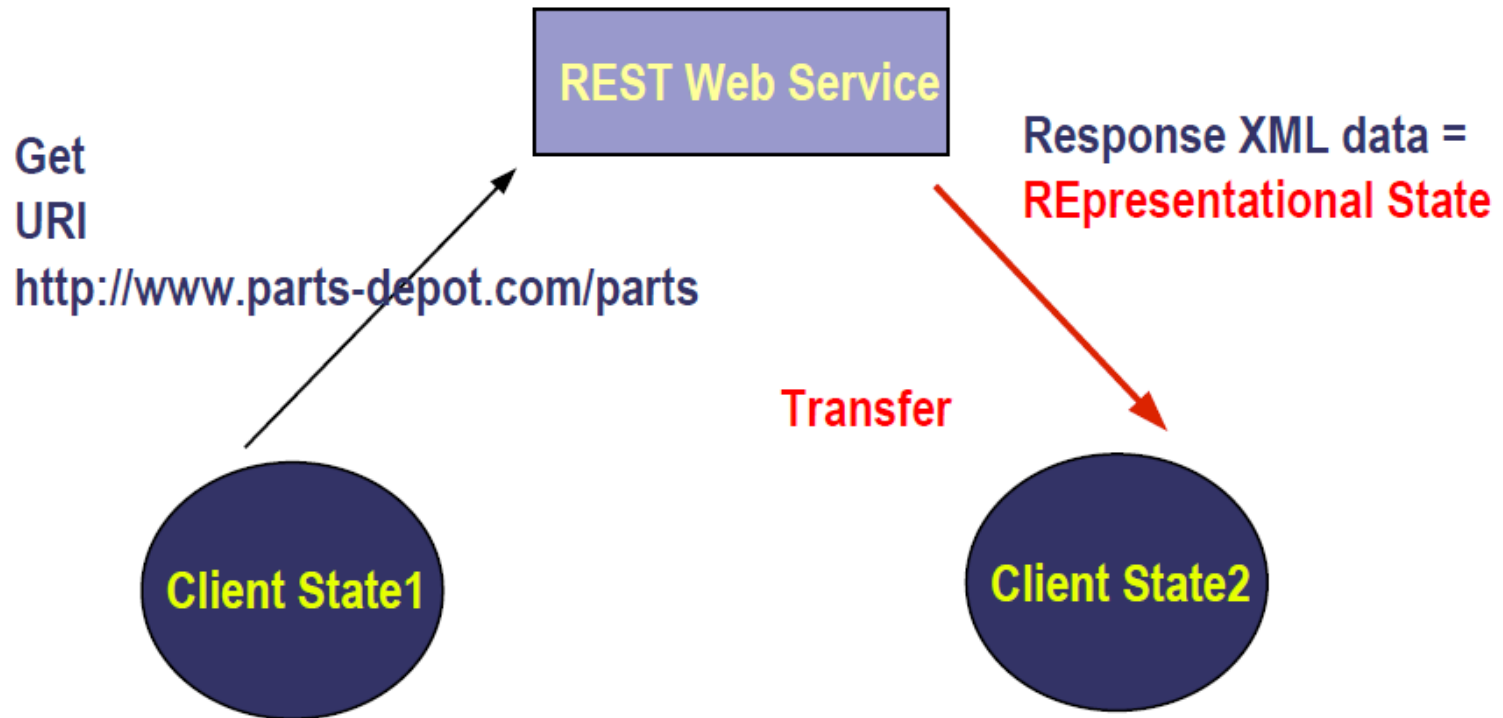
```
</soap:Envelope>
```

- Simple **REST service** as an example
- And with REST? The query will probably look like this:
<http://www.acme.com/phonebook/UserDetails/12345>
- GET [/phonebook/UserDetails/12345](http://www.acme.com/phonebook/UserDetails/12345) HTTP/1.1
Host: www.acme.com
Accept: application/xml
- **Complex query:**
<http://www.acme.com/phonebook/UserDetails?firstName=John&lastName=Doe>

1. IoT Communications Protocols HTTP-REST Recap

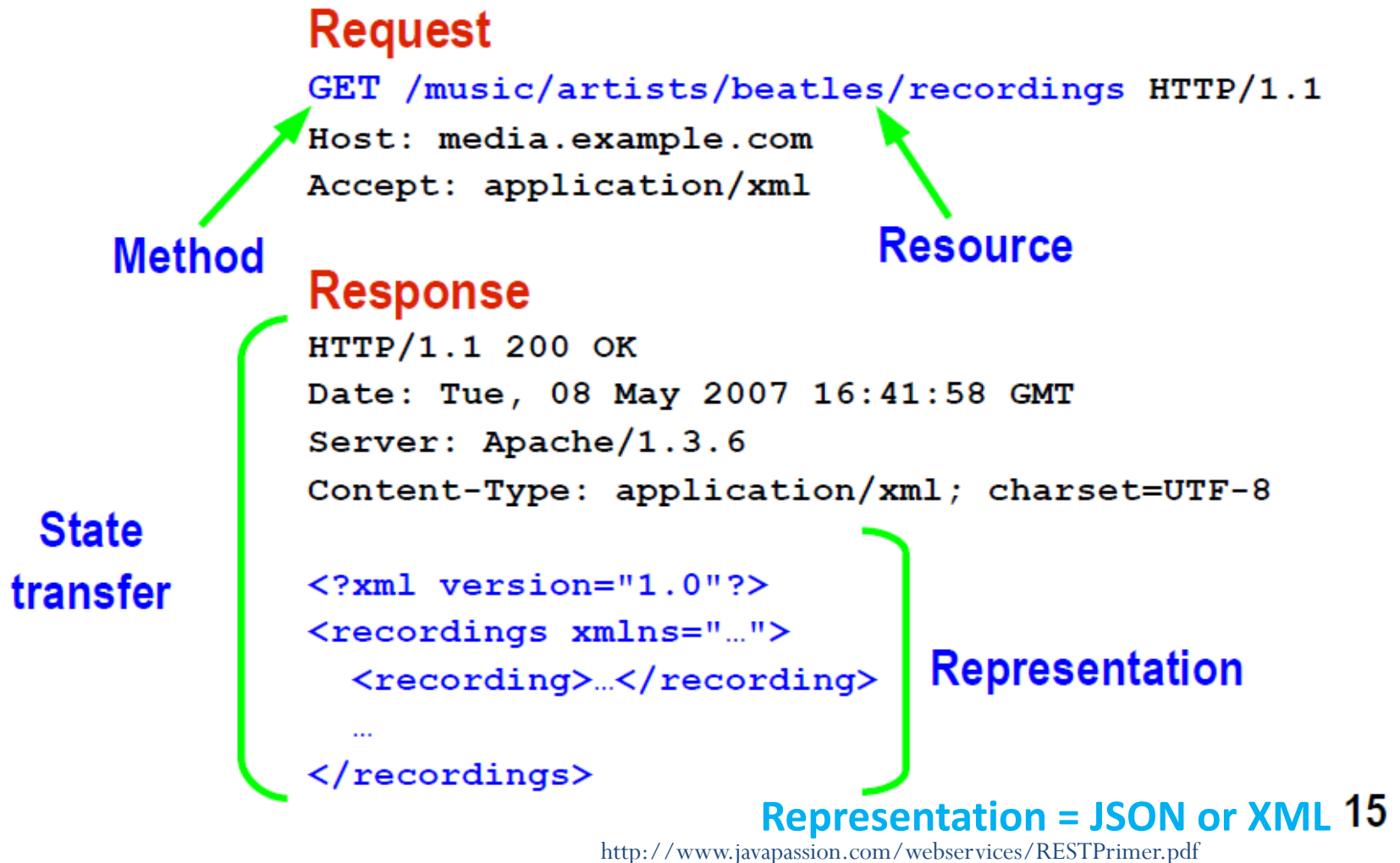
What is REST?

REpresentational **S**tate **T**ransfer



1. IoT Communications Protocols HTTP-REST Recap

HTTP Request/Response As REST



1. IoT Communications Protocols HTTP-REST Recap

REST over HTTP – Uniform interface

- **CRUD** operations on resources
 - Create, Read, Update, Delete
- Performed through **HTTP methods + URI**



CRUD Operations

4 main HTTP methods

	Verb	Noun
Create (Single)	POST	Collection URI
Read (Multiple)	GET	Collection URI
Read (Single)	GET	Entry URI
Update (Single)	PUT	Entry URI
Delete (Single)	DELETE	Entry URI

Operation	SQL	HTTP
Create	INSERT	PUT / POST
Read (Retrieve)	SELECT	GET
Update (Modify)	UPDATE	PUT / PATCH
Delete (Destroy)	DELETE	DELETE

POST	=	Create
GET	=	Read
PUT	=	Update
DELETE	=	Delete

1. IoT Communications Protocols CoAP vs. MQTT

HTTP + HTML – The Web Protocol

What was the element of success for the Web?

HTML

Uniform representation of documents;

URIs

Uniform Referents for Data and Services on the Web;

HTTP

Universal transfer protocol;

Enables a Distribution System of Proxies and Reverse Proxies

1. IoT Communications Protocols CoAP vs. MQTT

REST and Web Architecture

REpresentational State Transfer

Relies on a stateless, client-server, cacheable communication protocol

Instead of using complex mechanisms to connect between machines, simple HTTP is used to make call between machines

RESTful applications use HTTP requests to post data (create and/or update), read data (e.g., make queries), and delete data.

Thus, REST uses HTTP for all four CRUD (Create/Read/Update/Delete) operations.

Do Not Forget: REST is not a protocol nor a standard, but an **ARCHITECTURAL STYLE**

1. IoT Communications Protocols CoAP vs. MQTT

HTTP: Why not in IoT? – although possible to be in IoT GWs but rare in IoT Nodes



8/16-bit Microcontrollers, with limited RAM and ROM;

Constrained networks such as 6LoWPAN support the fragmentation of IPv6 packets into small link- layer frames, however incurring significant reduction in packet delivery probability;

TCP as the Transport Protocol, too heavy for LLN motes;

SSL/TLS for security: too heavy;

1. IoT Communications Protocols CoAP

CoAP – Default UDP Port 5683

IETF CoRE, designed to ensure interoperability with the WEB (GET, PUT, POST,DELETE).

Last Update: v18, 28 June 2013, Link:

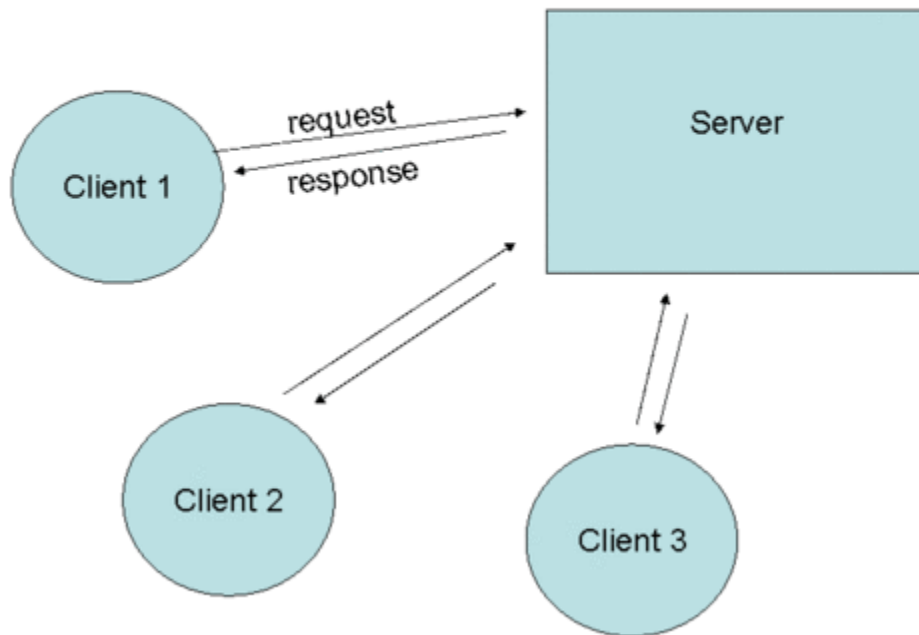
<http://tools.ietf.org/html/draft-ietf-core-coap-18>

- Document-Centric.
- Request/Response mode, with the Observe flag.
- UDP binding, with optional reliability supporting unicast and multicast request (5683 UDP Port)
- Asynchronous Messages Exchanges
- Low Header Overhead and Parsing Complexity
- Simple proxying and Caching Capabilities
- Security binding to DTLS

1. IoT Communications Protocols CoAP

CoAP – Interaction Model

Client/Server Architecture



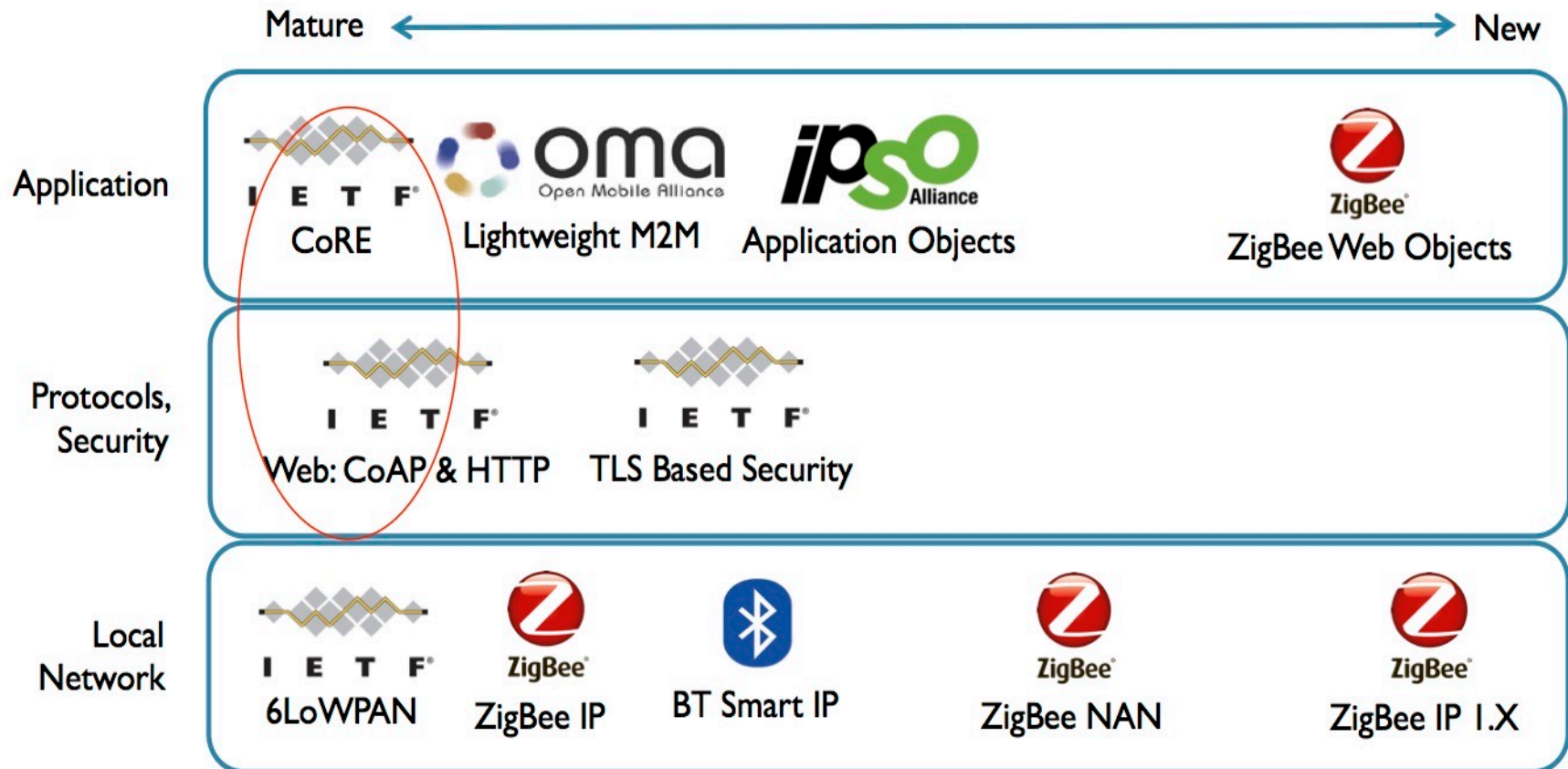
A CoAP implementation acts both in client and server role

Response Code;

Asynchronous Exchange

1. IoT Communications Protocols CoAP

CoAP is One Key IoT Standard

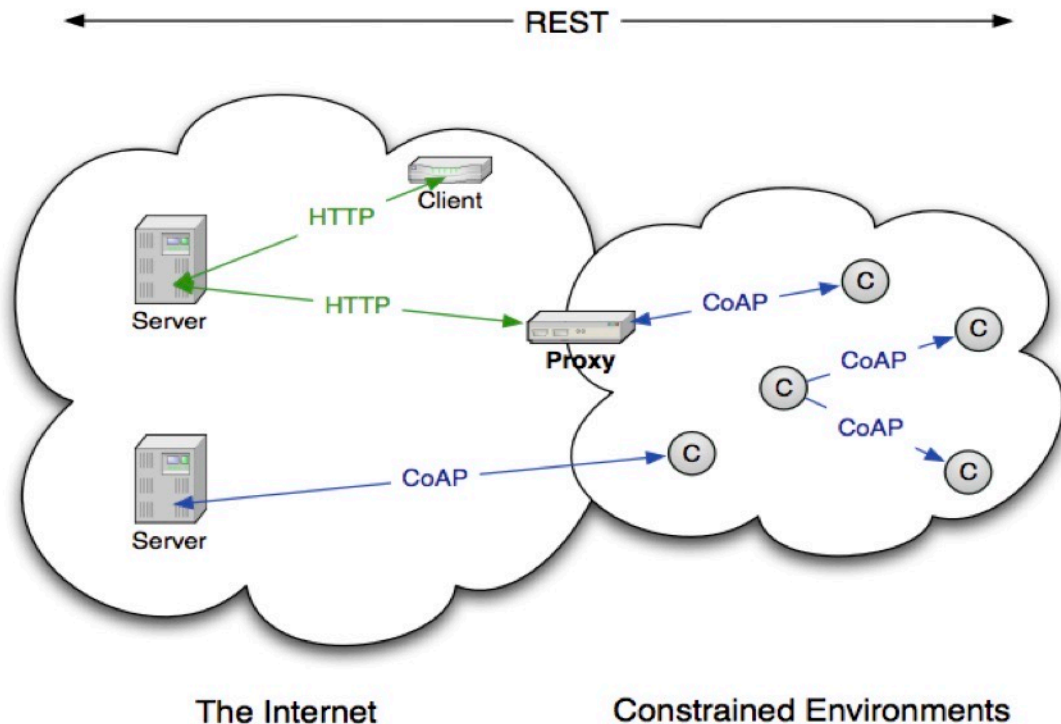
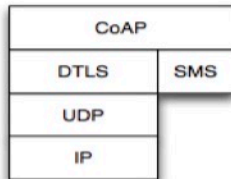


ARM

1. IoT Communications Protocols CoAP

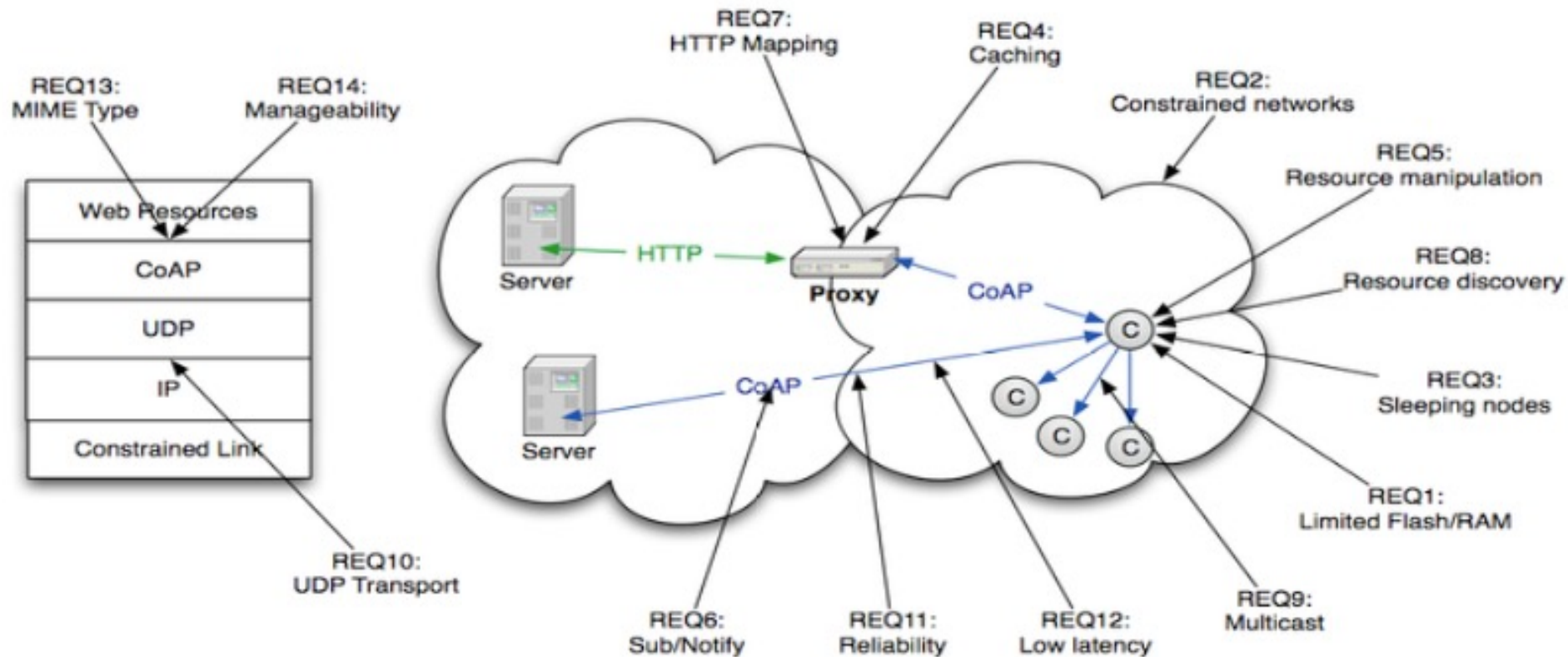
CoAP: The Web of Things Protocol

- Open IETF Standard
- Compact 4-byte Header
- UDP, SMS, (TCP) Support
- Strong DTLS Security
- Asynchronous Subscription
- Built-in Discovery



1. IoT Communications Protocols CoAP

CoAP Design Requirements



1. IoT Communications Protocols CoAP

What CoAP is (and is not)

- Sure, CoAP is
 - A very efficient RESTful protocol
 - Ideal for constrained devices and networks
 - Specialized for M2M applications
 - Easy to proxy to/from HTTP
- But hey, CoAP is not
 - A general replacement for HTTP
 - HTTP compression
 - Restricted to isolated “automation” networks

CoAP Features

- Embedded web transfer protocol (coap://)
- Asynchronous transaction model
- UDP binding with reliability and multicast support
- GET, POST, PUT, DELETE methods
- URI support
- Small, simple 4 byte header
- DTLS based PSK, RPK and Certificate security
- Subset of MIME types and HTTP response codes
- Built-in discovery
- Optional observation and block transfer

1. IoT Communications Protocols CoAP

Transaction Model

■ Transport

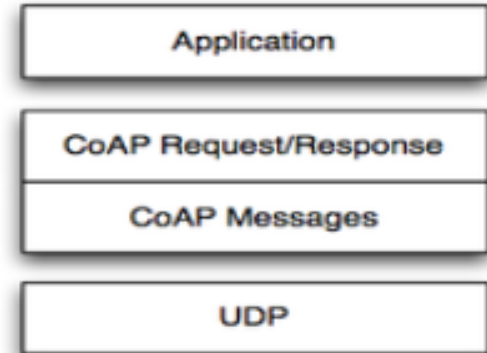
- CoAP currently defines:
- UDP binding with DTLS security
- CoAP over SMS or TCP possible

■ Base Messaging

- Simple message exchange between endpoints
- Confirmable or Non-Confirmable Message answered by Acknowledgement or Reset Message

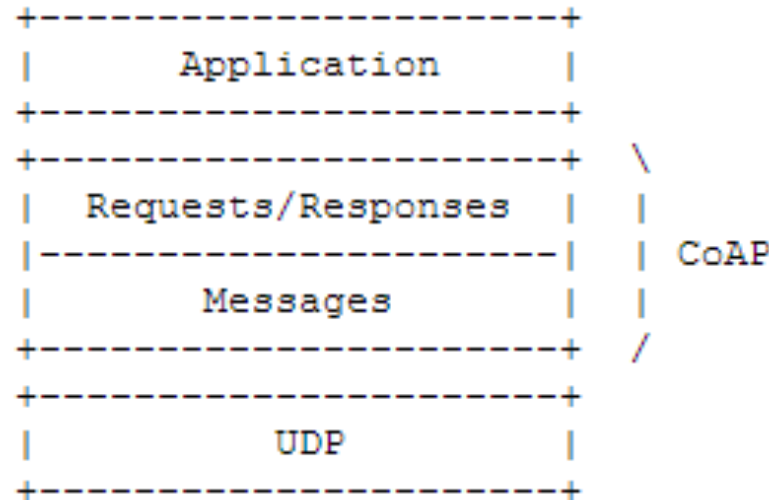
■ REST Semantics

- REST Request/Response piggybacked on CoAP Messages
- Method, Response Code and Options (URI, content-type etc.)



1. IoT Communications Protocols CoAP

CoAP – Two Layer Approach



Messages Layer: deal with UDP and the asynchronous nature of the interactions

Request Response Layer: Method and Response Codes

CoAP is however a single protocol, with messages and request/response just features of the CoAP header

1. IoT Communications Protocols CoAP

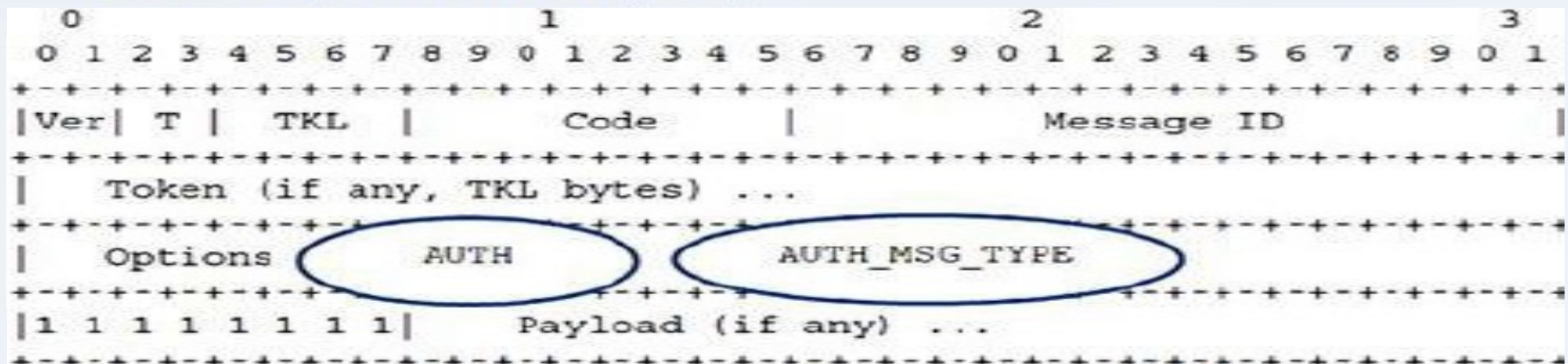


Message Format

CoAP messages are encoded in a simple binary format.

The Message Header (4 bytes).

The variable-length token value 0 and 8 bytes long.



Ver - Version (1) → 2 bit unsigned integer. Implementations of this field to 1 (01 binary).

T - Message Type → 2-bit unsigned integer. (Confirmable, Non-Confirmable, Acknowledgement, Reset).

TKL - Token Length → 4-bit unsigned integer. Indicates the length of the variable-length Token field (0-8 bytes).

Code - 8-bit unsighted integer. 3 bit class(most signification bits). 5 bits detail (least significant bits).

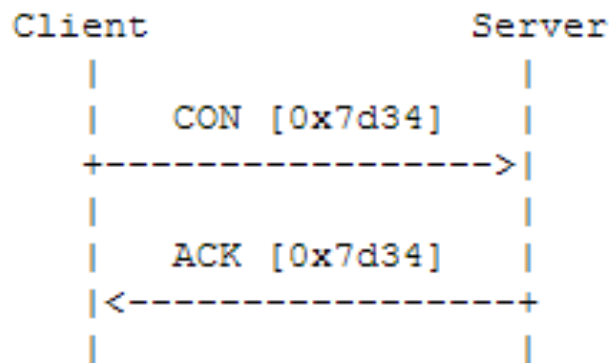
Request Method (1-10) or Response Code (40-255)

Message ID - 16-bit identifier for matching responses

Token - Optional response matching token

1. IoT Communications Protocols CoAP

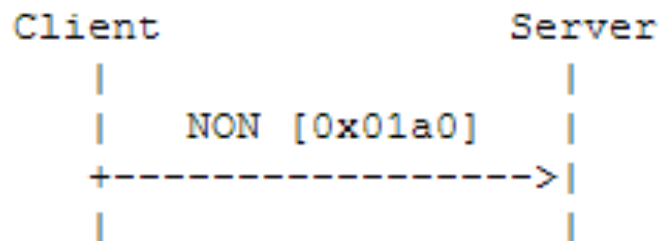
CoAP – Messaging Model



Confirmable (CON):

Default Timeout and Exponential Backoff, ACK with the same Message ID.

Reset option: if the server cannot support confirmable mode.



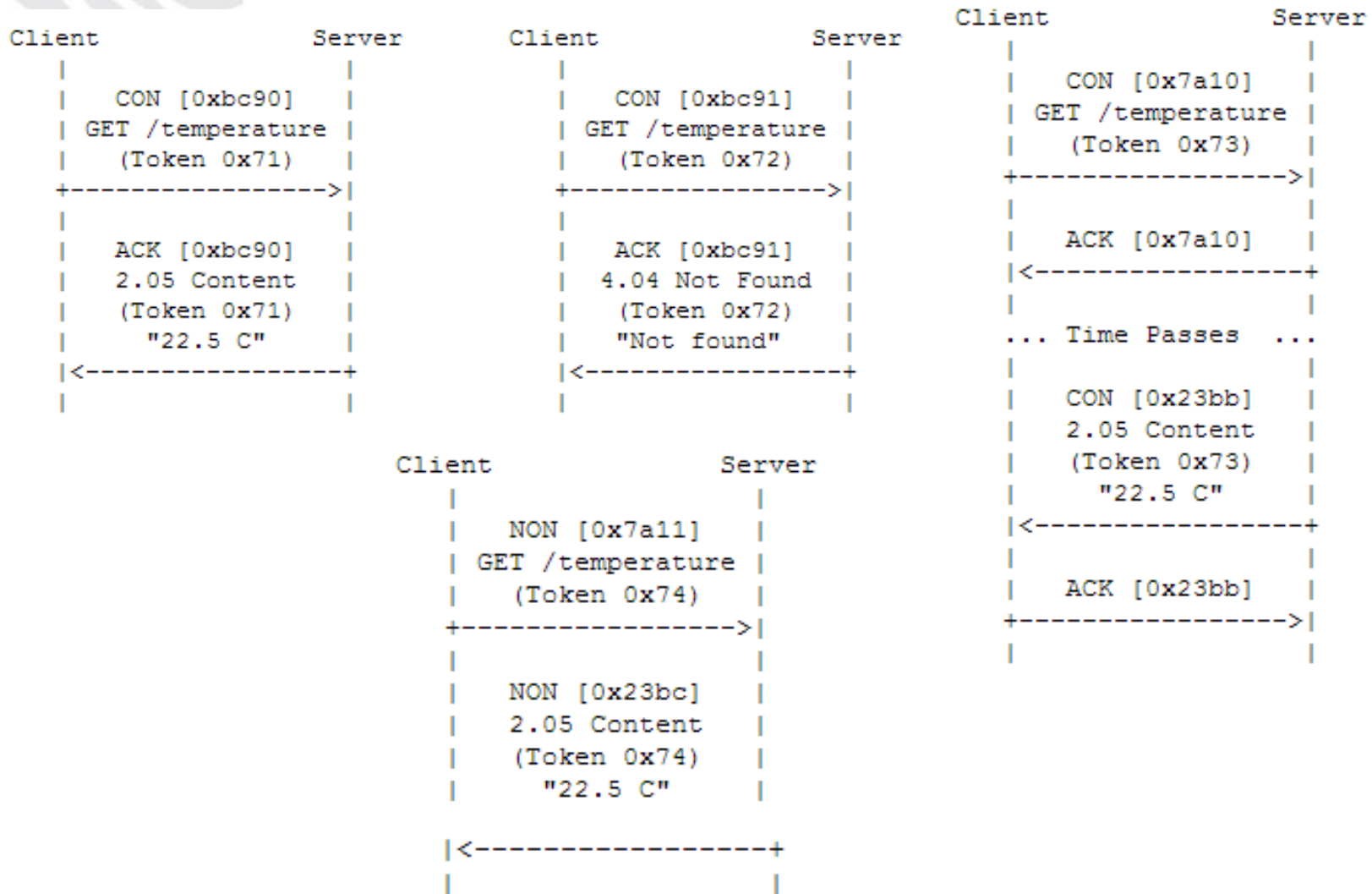
Non-Confirmable (NON):

Simple data, Message ID for duplicate detection

Reset Option: if the server cannot accept NON messages.

1. IoT Communications Protocols CoAP

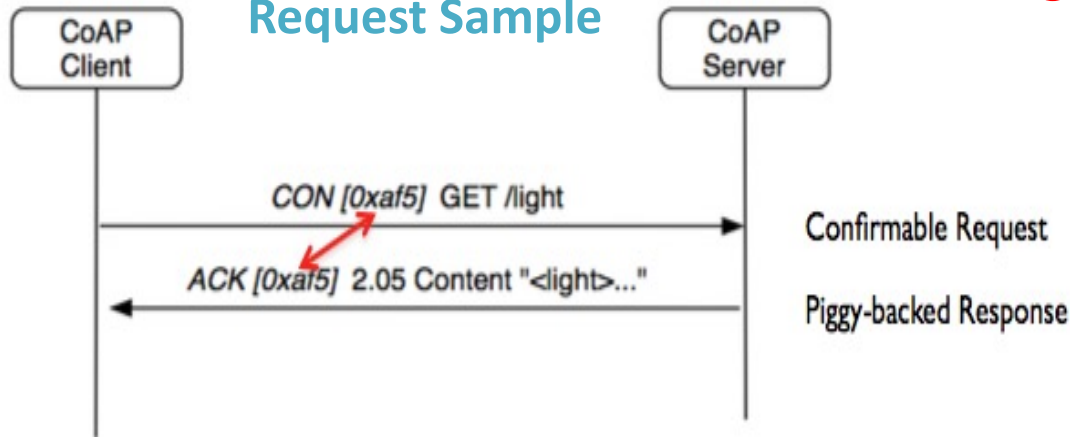
CoAP – Interaction Model



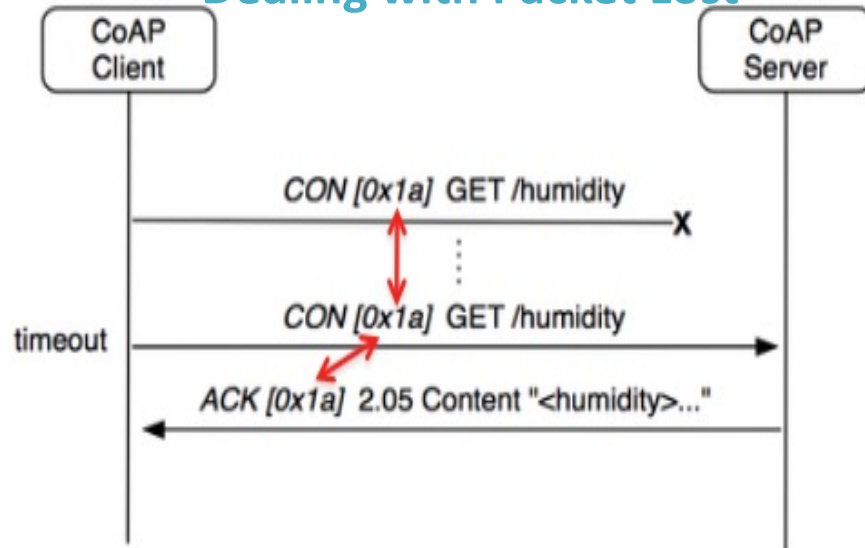
1. IoT Communications Protocols CoAP

CoAP – Interaction Model

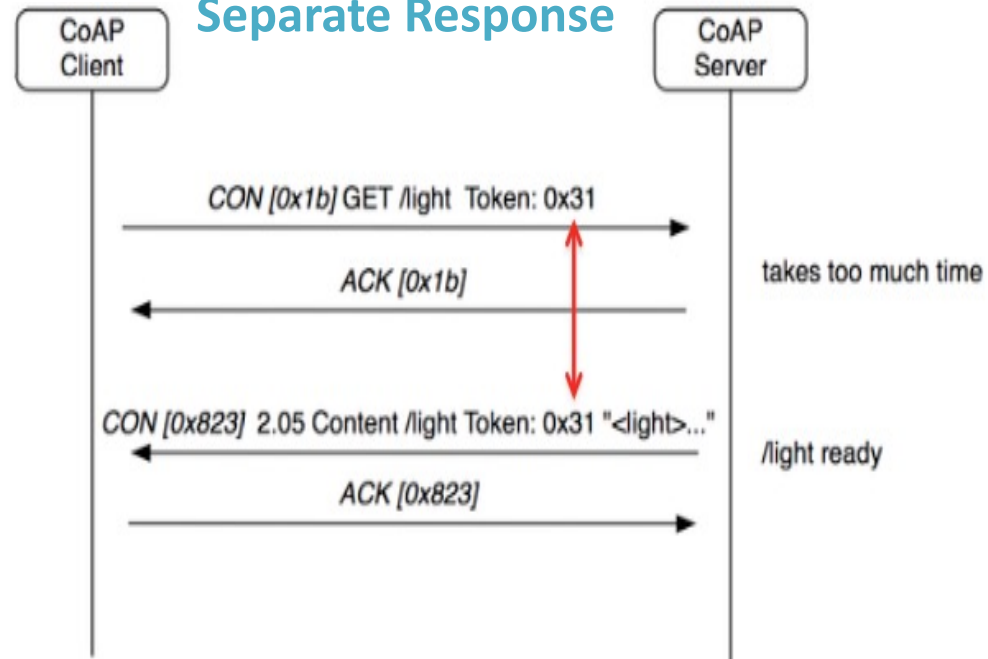
Request Sample



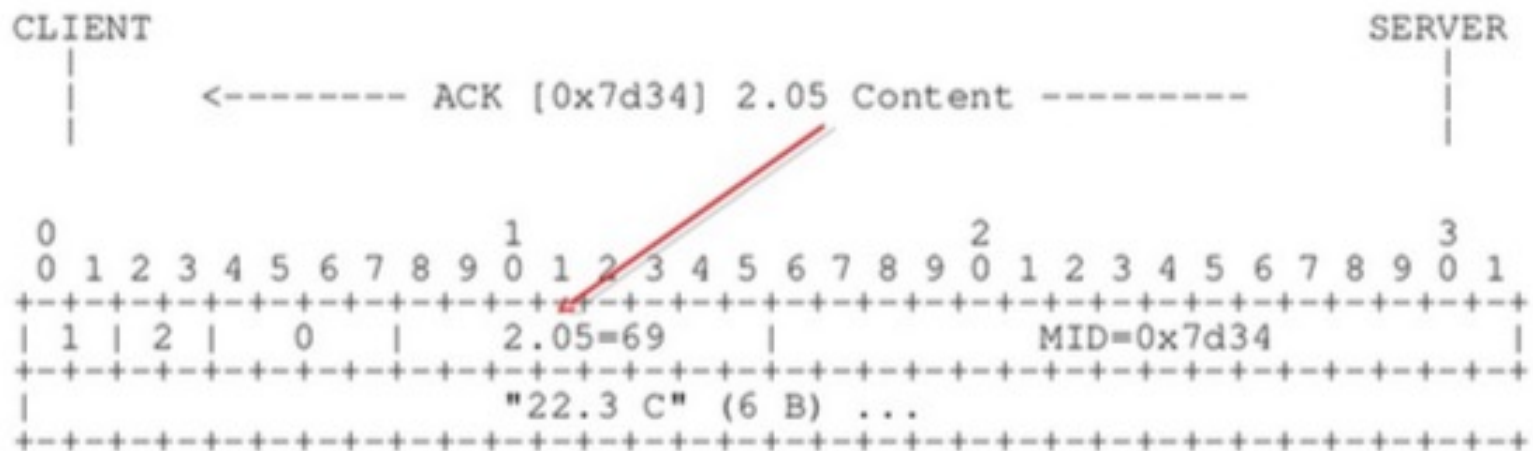
Dealing with Packet Lost



Separate Response



Bits and bytes...



1. IoT Communications Protocols CoAP

CoAP – Other Features

- Caching
- CoAP supports caching of responses to efficiently fulfill requests. Simple Caches is particularly useful in constrained networks for several reasons, including traffic limiting, performance improving, resources accessing times and security.
- Resource Discovery
- CoAP Multicast: “All CoAP Nodes”
GET ./well-known/core

1. IoT Communications Protocols CoAP

CoAP – COnstrained Application Protocol getting started

- There are many open source implementations available:
 - [mbed](#) includes CoAP support
 - Java CoAP Library [Californium](#) / [org.ws4d.coap \(ws4d-jcoap.jar\)](#)
 - C CoAP Library [Erbium](#)
 - [libCoAP](#) C Library
 - [jCoAP](#) Java Library
 - [OpenCoAP](#) C Library
 - TinyOS and Contiki include CoAP support
- CoAP is already part of many commercial products/systems
 - ARM Sensinode [NanoService](#)
 - [RTX 4100](#) WiFi Module
- Firefox has a CoAP [plugin called Copper](#)
- Wireshark has CoAP dissector support
- Implement CoAP yourself, it is not that hard! – if time available!

1. IoT Communications Protocols MQTT

Message Queueing Telemetry Transport – MQTT – Default TCP Port 1883



1998, Dave Locke & Ian Craggs, IBM. From March, 2013, start of standardization process at OASIS. Now v3.1 (2013)

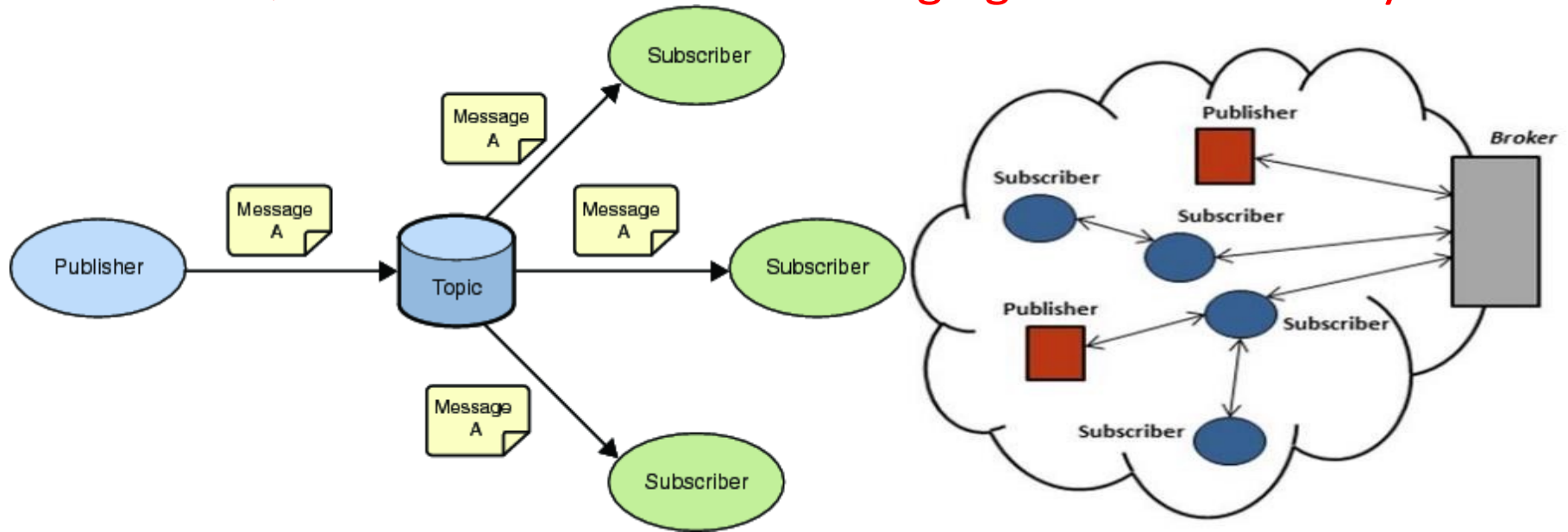
<http://public.dhe.ibm.com/software/dw/webservices/ws-mqtt/mqtt-v3r1.html>

Main Features:

- Publish / Subscribe message pattern → one-to-many messaging distribution, applications decoupling;
- Message transport payload-agnostic;
- Assumes the use of the TCP/IP protocol stack;
- 3 QoS Levels: At Most Once, At Least Once, Exactly Once;
- Small Transport Overhead, minimal messages exchanges;
- Will Mechanism, to indicate to the other part an abnormal disconnection

1. IoT Communications Protocols MQTT

MQTT - Publish Subscribe Messaging aka One to Many



A Publish Subscribe messaging protocol allowing a message to be published once and multiple consumers (applications / devices) to receive the message providing decoupling between the producer and consumer(s)

A producer sends (publishes) a message (publication) on a topic (subject)

A consumer subscribes (makes a subscription) for messages on a topic (subject)

A topic is managed within a MQTT Broker

A message server / broker matches publications to subscriptions

- If no matches the message is discarded
- If one or more matches the message is delivered to each matching subscriber/consumer

MQTT

MQ TELEMETRY TRANSPORT

AN INTRODUCTION TO MQTT, A PROTOCOL FOR
M2M AND IoT APPLICATIONS

Peter R. Egli
INDIGOO.COM

Rev. 1.80

Contents

1. What is MQTT?
2. MQTT characteristics
3. Origins and future of MQTT standard
4. MQTT model
5. MQTT message format
6. MQTT QoS
7. CONNECT and SUBSCRIBE message sequence
8. PUBLISH message flows
9. Keep alive timer, breath of live with PINGREQ
10. MQTT will message
11. Topic wildcards
12. MQTT-SN

1. What is MQTT?

MQTT is a lightweight message queueing and transport protocol.

MQTT, as its name implies, is suited for the transport of telemetry data (sensor and actor data).

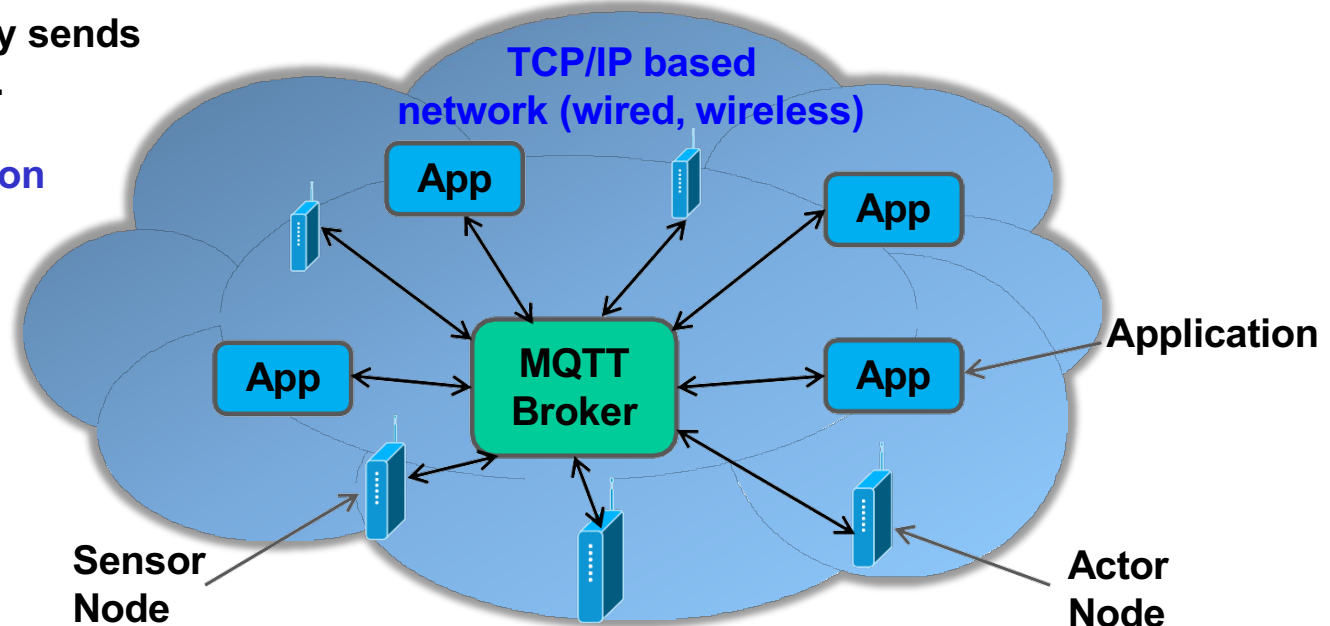
MQTT is very lightweight and thus suited for M2M (Machine to Machine / Mobile to Mobile), WSN (Wireless Sensor Networks) and ultimately IoT (Internet of Things) scenarios where sensor and actor nodes communicate with applications through the MQTT message broker.

Example:

Light sensor continuously sends sensor data to the broker.

Building control application receives sensor data from the broker and decides to activate the blinds.

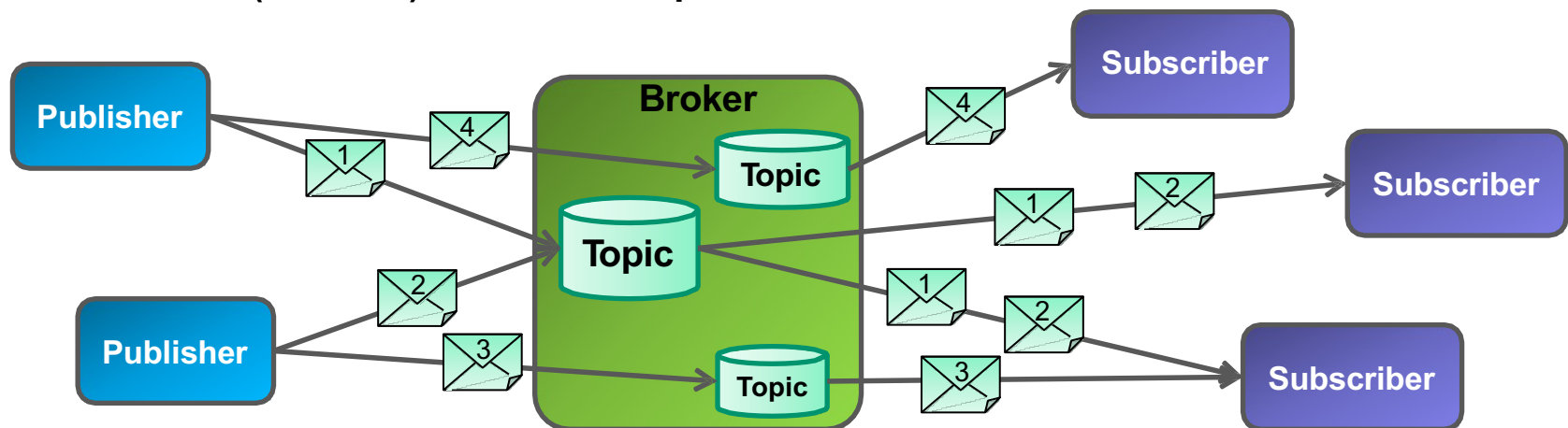
Application sends a blind activation message to the **blind actor node** through the broker.



2. MQTT characteristics

MQTT Key features:

- Lightweight message queueing and transport protocol
- Asynchronous communication model with messages (events)
- Low overhead (2 bytes header) for low network bandwidth applications
- Publish / Subscribe (PubSub) model
- Decoupling of data producer (publisher) and data consumer (subscriber) through topics (message queues)
- Simple protocol, aimed at low complexity, low power and low footprint implementations (e.g. WSN - Wireless Sensor Networks)
- Runs on connection-oriented transport (TCP). To be used in conjunction with 6LoWPAN (TCP header compression)
- MQTT caters for (wireless) network disruptions



1. IoT Communications Protocols MQTT

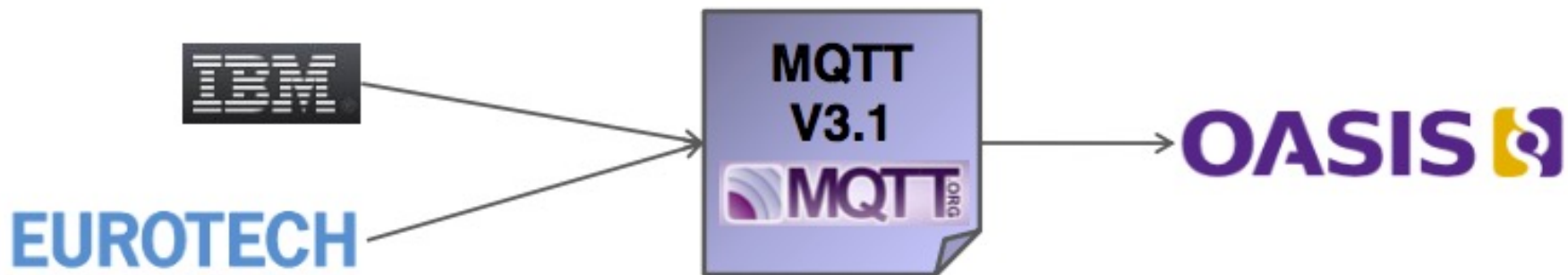
- The past, present and future of MQTT:
 - MQTT was developed by IBM and Eurotech.
 - The current version 3.1 is available from <http://mqtt.org/>

Eventually, MQTT version 3.1 is to be adopted and published as an official standard by OASIS (process ongoing).

As such, OASIS becomes the new home for the development of MQTT.

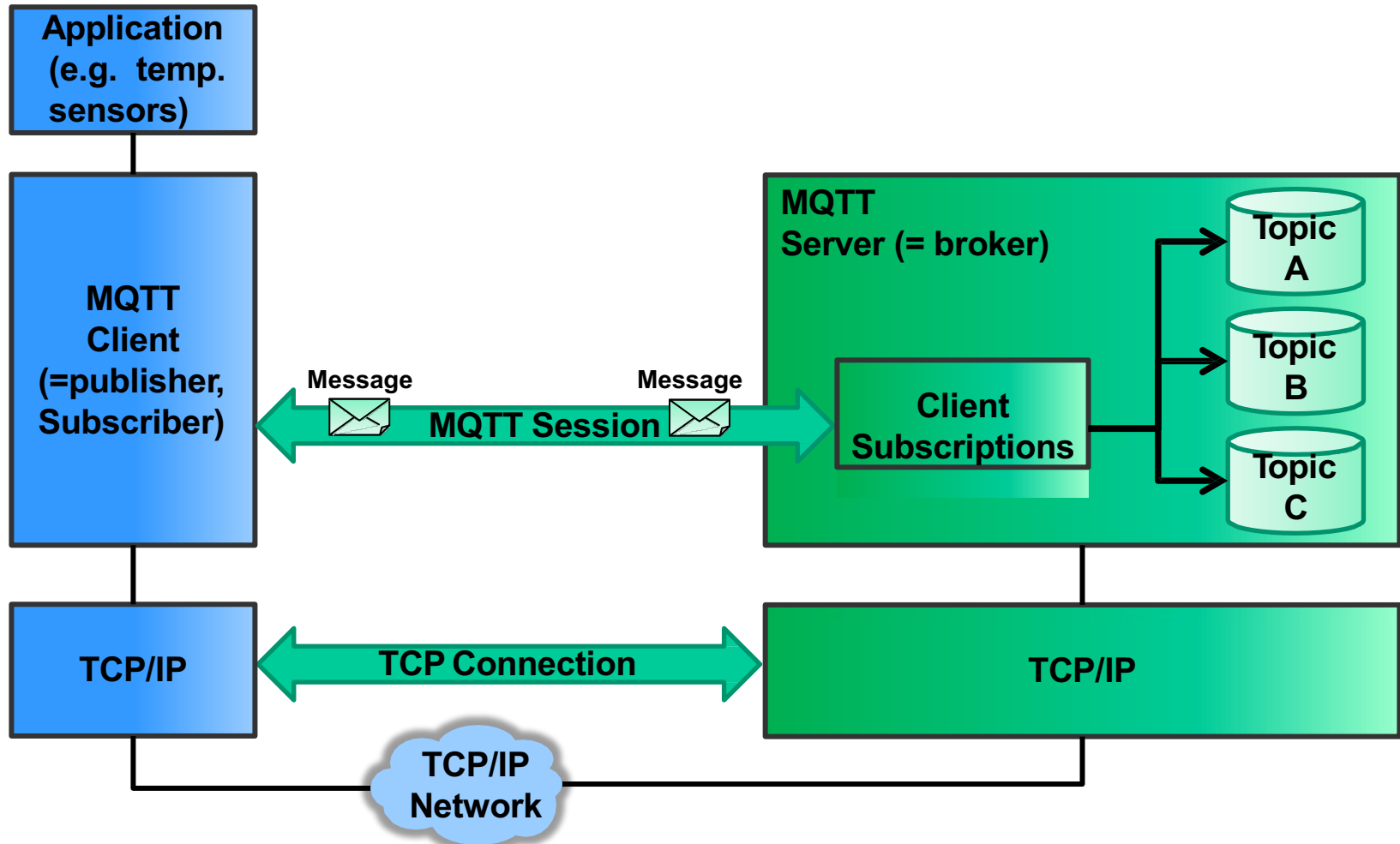
The OASIS TC (Technical Committee) tasked with the further development of MQTT commits to the following:

- Backward compatibility of forthcoming OASIS MQTT standard with MQTT V3.1
- Changes restricted to the CONNECT message
- Clarification of existing version V3.1 (mostly editorial changes)



4. MQTT model (1/3)

The core elements of MQTT are clients, servers (=brokers), sessions, subscriptions and topics.

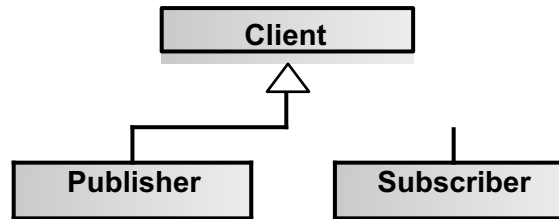


4. MQTT model (2/3)

MQTT client (=publisher, subscriber):

Clients subscribe to topics to publish and receive messages.

Thus subscriber and publisher are special roles of a client.



MQTT server (=broker):

Servers run topics, i.e. receive subscriptions from clients on topics, receive messages from clients and forward these, based on client's subscriptions, to interested clients.

Topic:

Technically, topics are message queues. Topics support the publish/subscribe pattern for clients.

Logically, topics allow clients to exchange information with defined semantics.

Example topic: Temperature sensor data of a building.



4. MQTT model (3/3)

Session:

A session identifies a (possibly temporary) attachment of a client to a server. All communication between client and server takes place as part of a session.

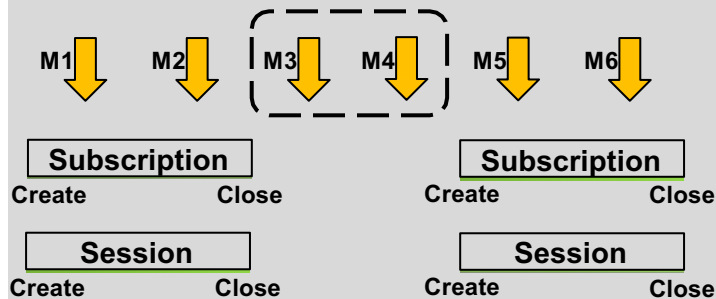
Subscription:

Unlike sessions, a subscription logically attaches a client to a topic. When subscribed to a topic, a client can exchange messages with a topic.

Subscriptions can be «transient» or «durable», depending on the clean session flag in the CONNECT message:

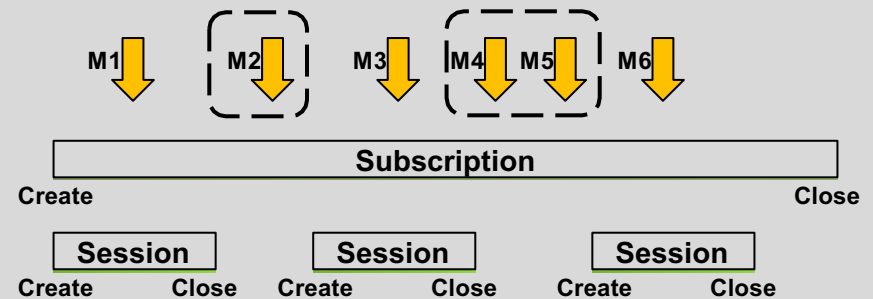
«Transient» subscription ends with session:

Messages M3 and M4 are not received by the client



«Durable» subscription:

Messages M2, M4 and M5 are not lost but will be received by the client as soon as it creates / opens a new session.



Message:

Messages are the units of data exchange between topic clients.

MQTT is agnostic to the internal structure of messages.

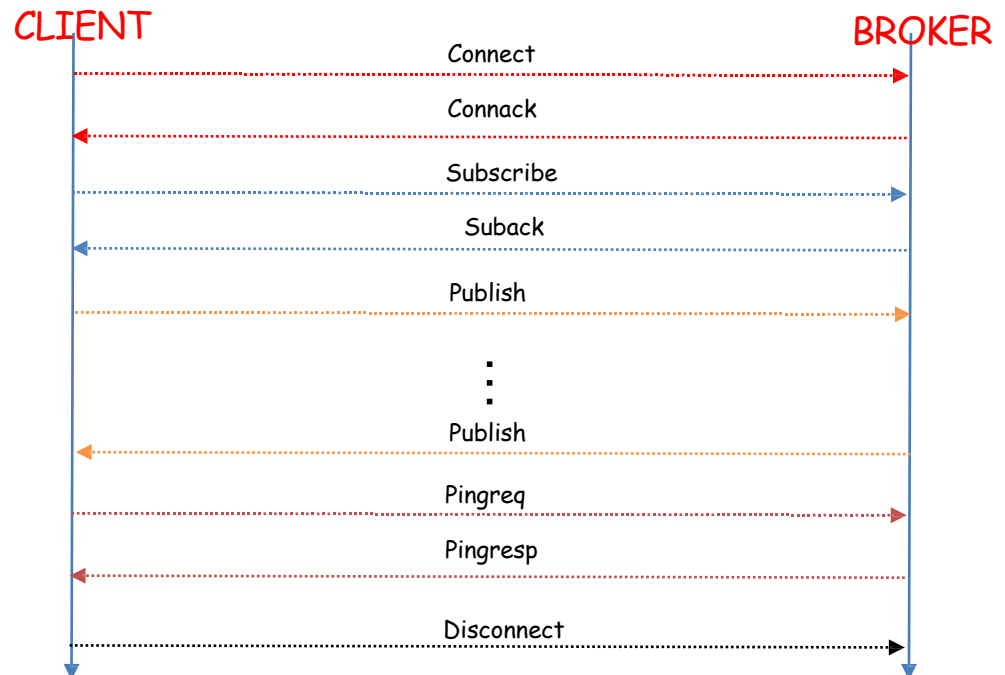
IoT Communications Protocols MQTT

MQTT – Message Type and 2 bytes Mandatory Header
Client – Broker General Messages Flow

bit	7	6	5	4	3	2	1	0
byte 1	Message Type				DUP flag	QoS level		RETAIN
byte 2	Remaining Length							

Message Type

1 CONNECT	8 SUBSCRIBE
2 CONNACK	9 SUBACK
3 PUBLISH	10 UNSUBSCRIBE
4 PUBACK	11 UNSUBACK
5 PUBREC	12 PINGREQ
6 PUBREL	13 PINGRESP
7 PUBCOMP	14 DISCONNECT



5. MQTT message format (1/14)

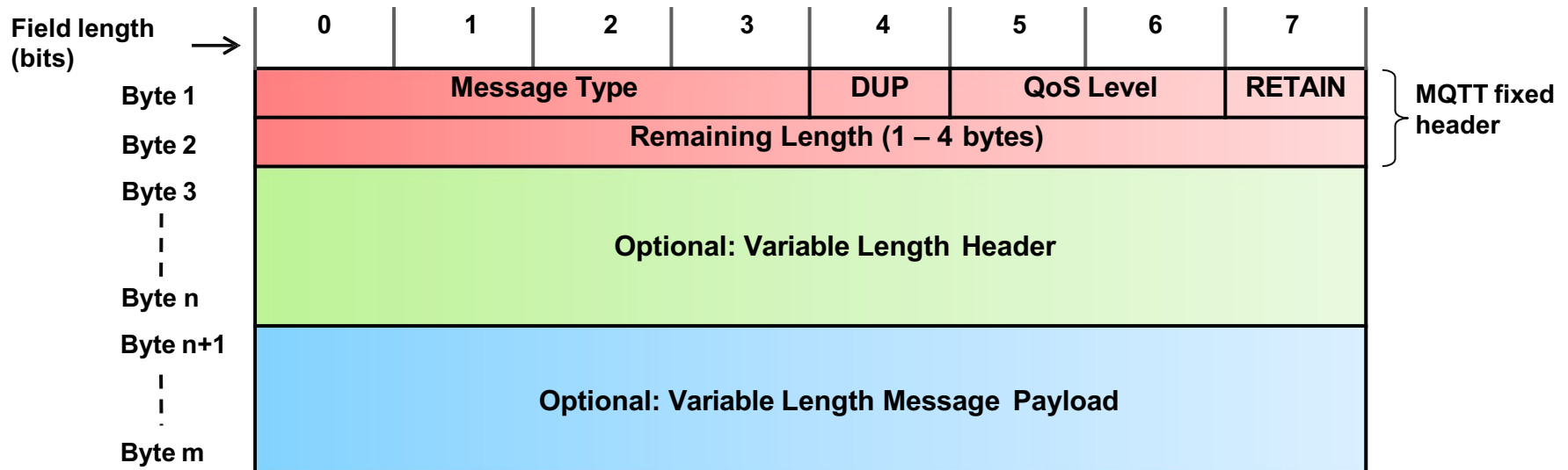
Message format:

MQTT messages contain a mandatory fixed-length header (2 bytes) and an optional message-specific variable length header and message payload.

Optional fields usually complicate protocol processing.

However, MQTT is optimized for bandwidth constrained and unreliable networks (typically wireless networks), so optional fields are used to reduce data transmissions as much as possible.

MQTT uses network byte and bit ordering.



5. MQTT message format (2/14)

Overview of fixed header fields:

Message fixed header field	Description / Values	
Message Type	0: Reserved	8: SUBSCRIBE
	1: CONNECT	9: SUBACK
	2: CONNACK	10: UNSUBSCRIBE
	3: PUBLISH	11: UNSUBACK
	4: PUBACK	12: PINGREQ
	5: PUBREC	13: PINGRESP
	6: PUBREL	14: DISCONNECT
	7: PUBCOMP	15: Reserved
DUP	Duplicate message flag. Indicates to the receiver that this message may have already been received. 1: Client or server (broker) re-delivers a PUBLISH, PUBREL, SUBSCRIBE or UNSUBSCRIBE message (duplicate message).	
QoS Level	Indicates the level of delivery assurance of a PUBLISH message. 0: At-most-once delivery, no guarantees, «Fire and Forget». 1: At-least-once delivery, acknowledged delivery. 2: Exactly-once delivery. Further details see MQTT QoS .	
RETAIN	1: Instructs the server to retain the last received PUBLISH message and deliver it as a first message to new subscriptions. Further details see RETAIN (keep last message) .	
Remaining Length	Indicates the number of remaining bytes in the message, i.e. the length of the (optional) variable length header and (optional) payload. Further details see Remaining length (RL) .	

5. MQTT message format (3/14)

RETAIN (keep last message):

RETAIN=1 in a PUBLISH message instructs the server to keep the message for this topic. When a new client subscribes to the topic, the server sends the retained message.

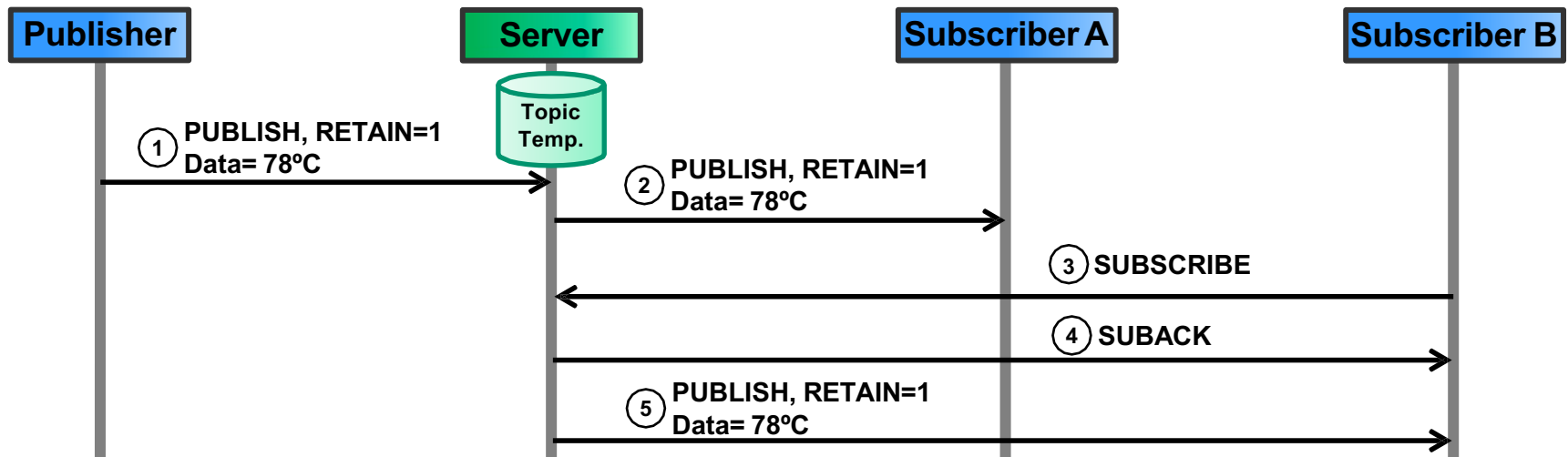
Typical application scenarios:

Clients publish only changes in data, so subscribers receive the **last known good value**.

Example:

Subscribers receive last known temperature value from the temperature data topic.

RETAIN=1 indicates to subscriber B that the message may have been published some time ago.



5. MQTT message format (4/14)

Remaining length (RL):

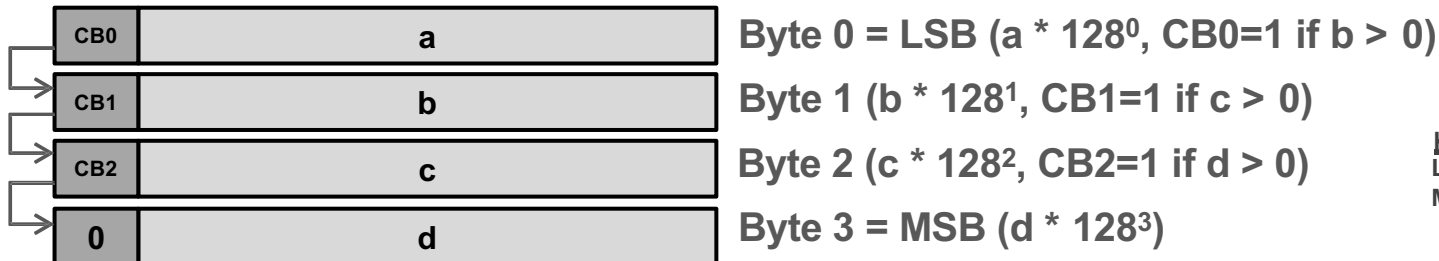
The remaining length field encodes the sum of the lengths of:

- (Optional) variable length header
- (Optional) payload

To save bits, remaining length is a variable length field with 1...4 bytes.

The most significant bit of a length field byte has the meaning «continuation bit» (CB). If more bytes follow, it is set to 1.

Remaining length is encoded as $a * 128^0 + b * 128^1 + c * 128^2 + d * 128^3$ and placed into the RL field bytes as follows:



Key:
LSB: Least Significant Byte
MSB: Most Significant Byte

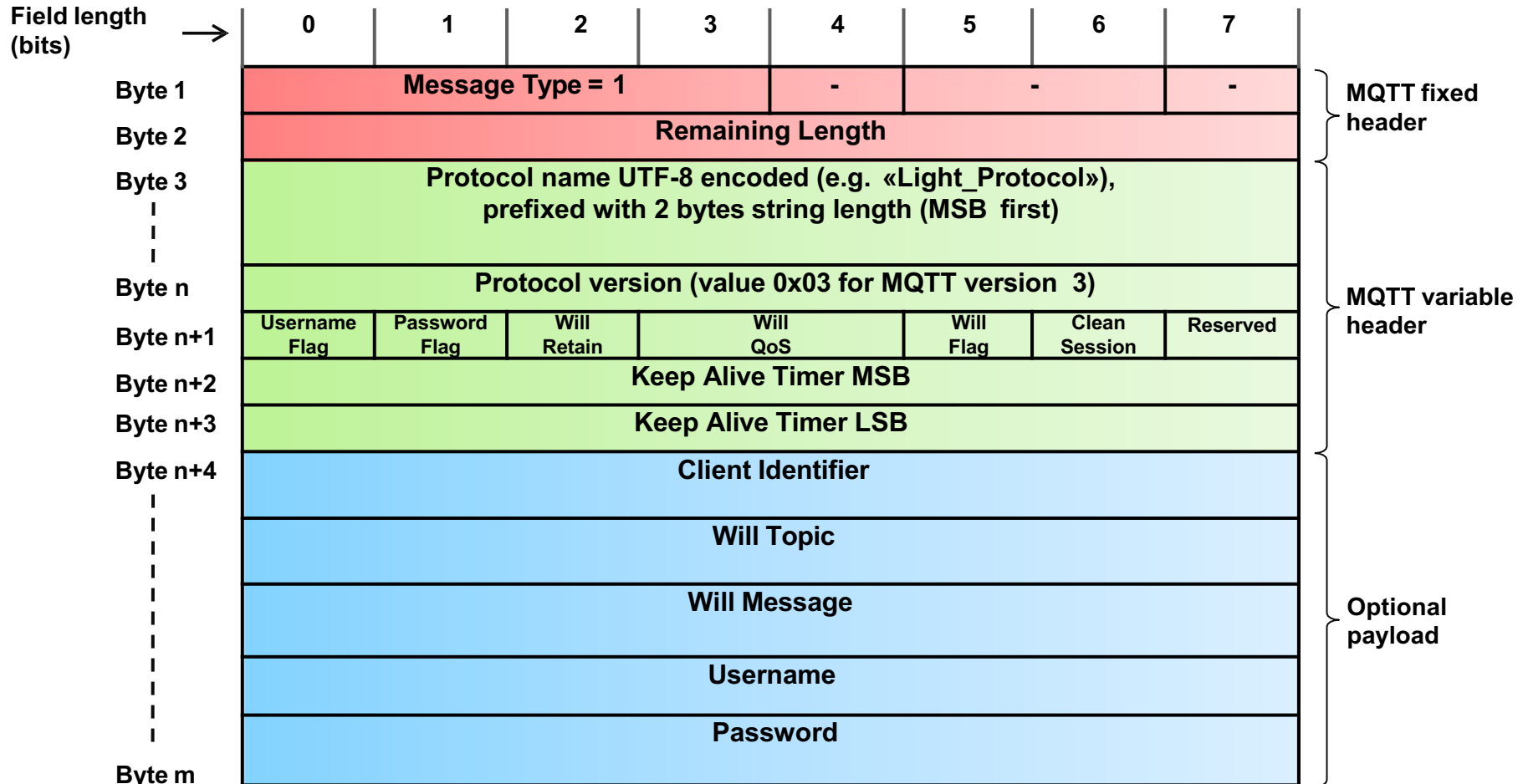
Example 1: $RL = 364 = 108 * 128^0 + 2 * 128^1 \rightarrow a=108, CB0=1, b=2, CB1=0, c=0, d=0, CB2=0$

Example 2: $RL = 25'897 = 41 * 128^0 + 74 * 128^1 + 1 * 128^2 \rightarrow a=41, CB0=1, b=74, CB1=1, c=1, CB2=0, d=0$

5. MQTT message format (5/14)

CONNECT message format:

The CONNECT message contains many session-related information as optional header fields.



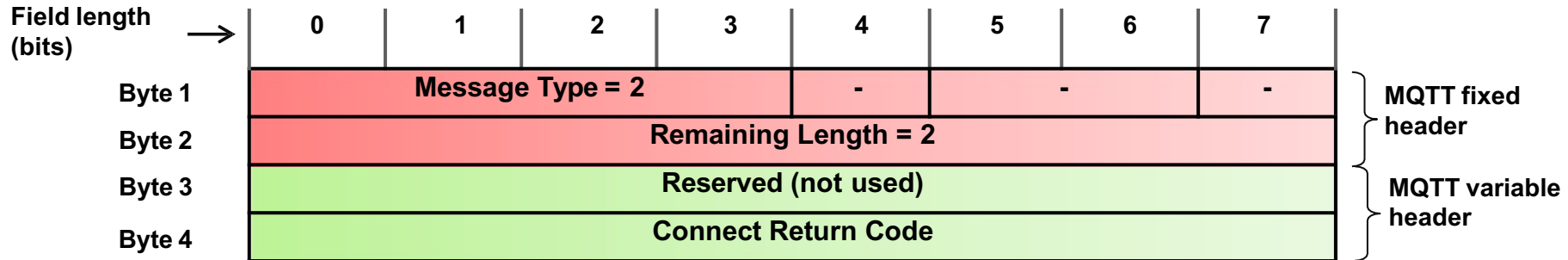
5. MQTT message format (6/14)

Overview CONNECT message fields:

CONNECT message field	Description / Values
Protocol Name	UTF-8 encoded protocol name string. Example: «Light Protocol»
Protocol Version	Value 3 for MQTT V3.
Username Flag	If set to 1 indicates that payload contains a username.
Password Flag	If set to 1 indicates that payload contains a password. If username flag is set, password flag and password must be set as well.
Will Retain	If set to 1 indicates to server that it should retain a Will message for the client which is published in case the client disconnects unexpectedly.
Will QoS	Specifies the QoS level for a Will message.
Will Flag	Indicates that the message contains a Will message in the payload along with Will retain and Will QoS flags. More details see MQTT will message .
Clean Session	If set to 1, the server discards any previous information about the (re)-connecting client (clean new session). If set to 0, the server keeps the subscriptions of a disconnecting client including storing QoS level 1 and 2 messages for this client. When the client reconnects, the server publishes the stored messages to the client.
Keep Alive Timer	Used by the server to detect broken connections to the client. More details see Keepalive timer .
Client Identifier	The client identifier (between 1 and 23 characters) uniquely identifies the client to the server. The client identifier must be unique across all clients connecting to a server.
Will Topic	Will topic to which a will message is published if the will flag is set.
Will Message	Will message to be published if will flag is set.
Username and Password	Username and password if the corresponding flags are set.

5. MQTT message format (7/14)

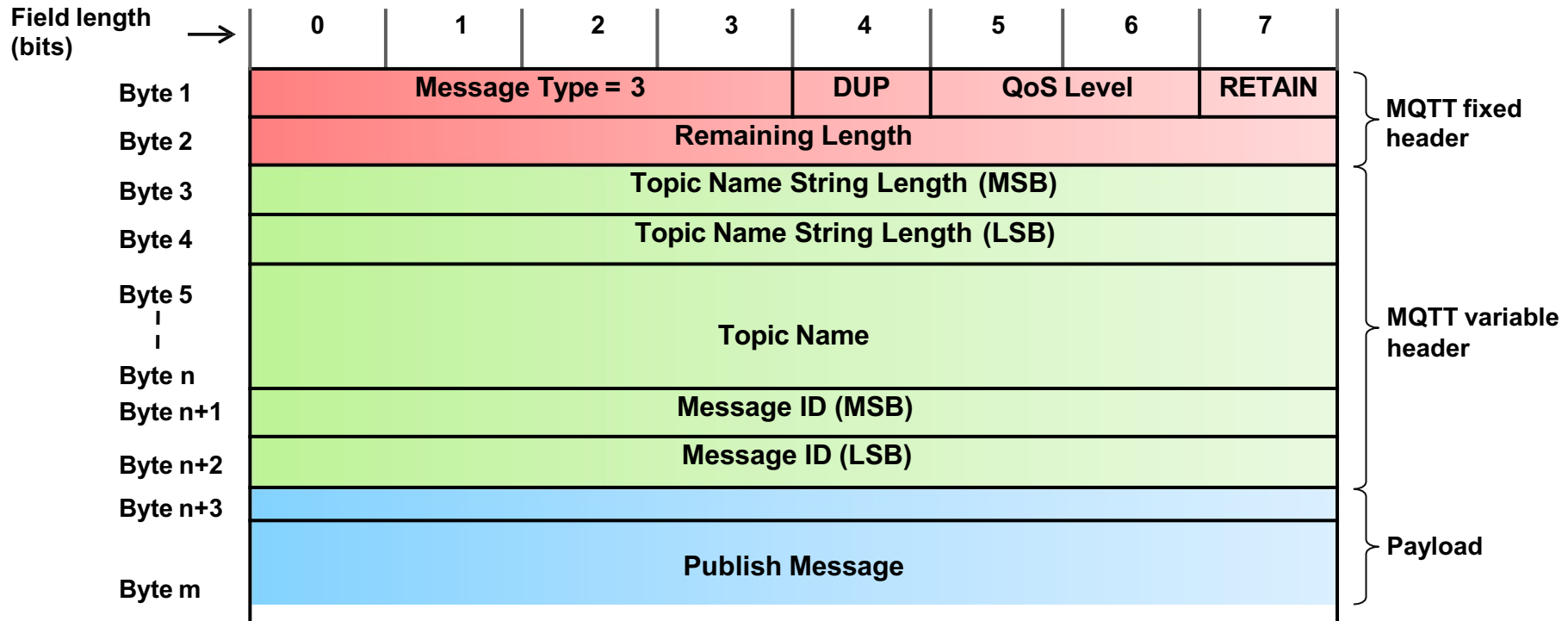
CONNACK message format:



CONNACK message field	Description / Values
Reserved	Reserved field for future use.
Connect Return Code	0: Connection Accepted 1: Connection Refused, reason = unacceptable protocol version 2: Connection Refused, reason = identifier rejected 3: Connection Refused, reason = server unavailable 4: Connection Refused, reason = bad user name or password 5: Connection Refused, reason = not authorized 6-255: Reserved for future use

5. MQTT message format (8/14)

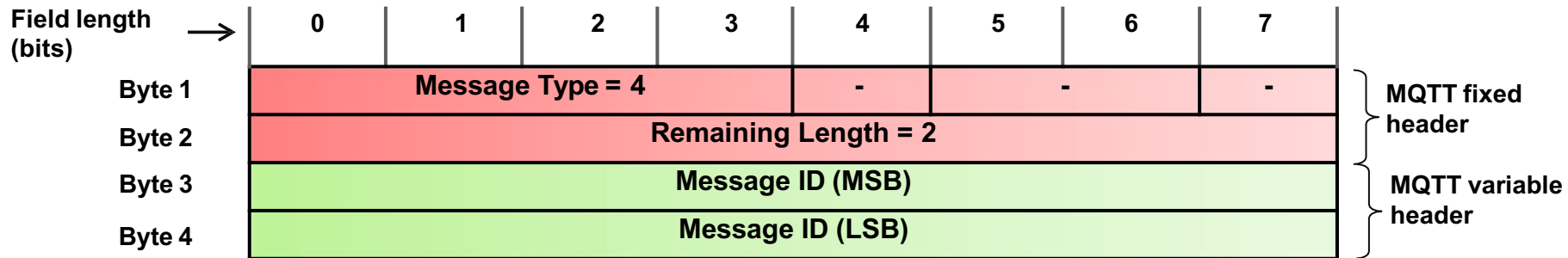
PUBLISH message format:



PUBLISH message field	Description / Values
Topic Name with Topic Name String Length	Name of topic to which the message is published. The first 2 bytes of the topic name field indicate the topic name string length.
Message ID	A message ID is present if QoS is 1 (At-least-once delivery, acknowledged delivery) or 2 (Exactly-once delivery).
Publish Message	Message as an array of bytes. The structure of the publish message is application-specific.

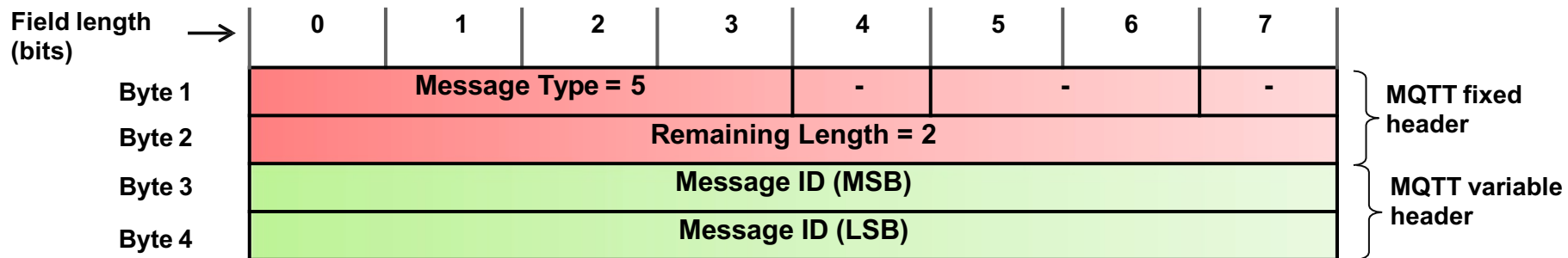
5. MQTT message format (9/14)

PUBACK message format:



PUBACK message field	Description / Values
Message ID	The message ID of the PUBLISH message to be acknowledged.

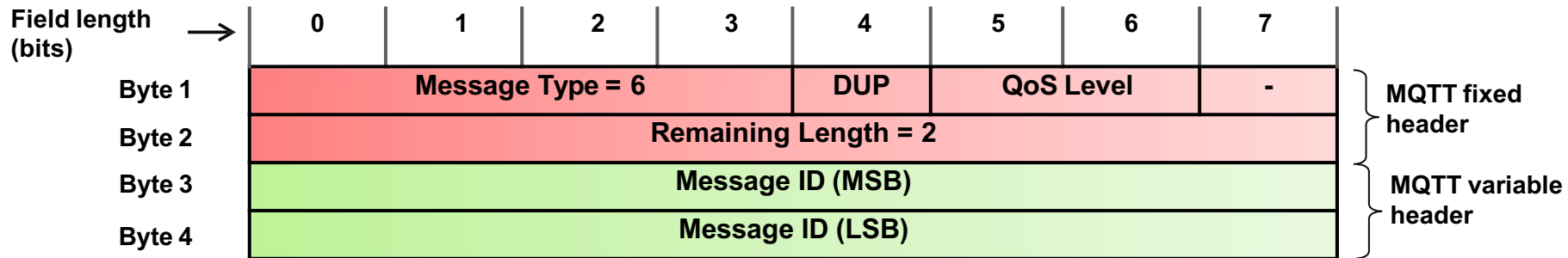
PUBREC message format:



PUBREC message field	Description / Values
Message ID	The message ID of the PUBLISH message to be acknowledged.

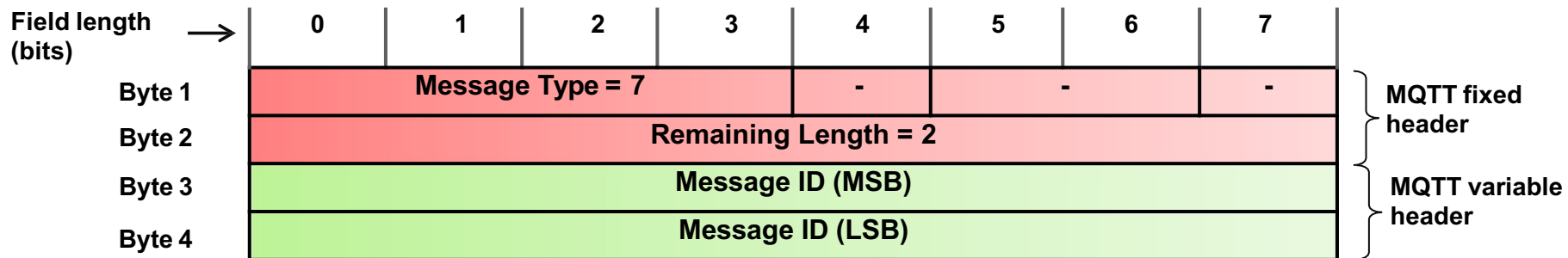
5. MQTT message format (10/14)

PUBREL message format:



PUBREL message field	Description / Values
Message ID	The message ID of the PUBLISH message to be acknowledged.

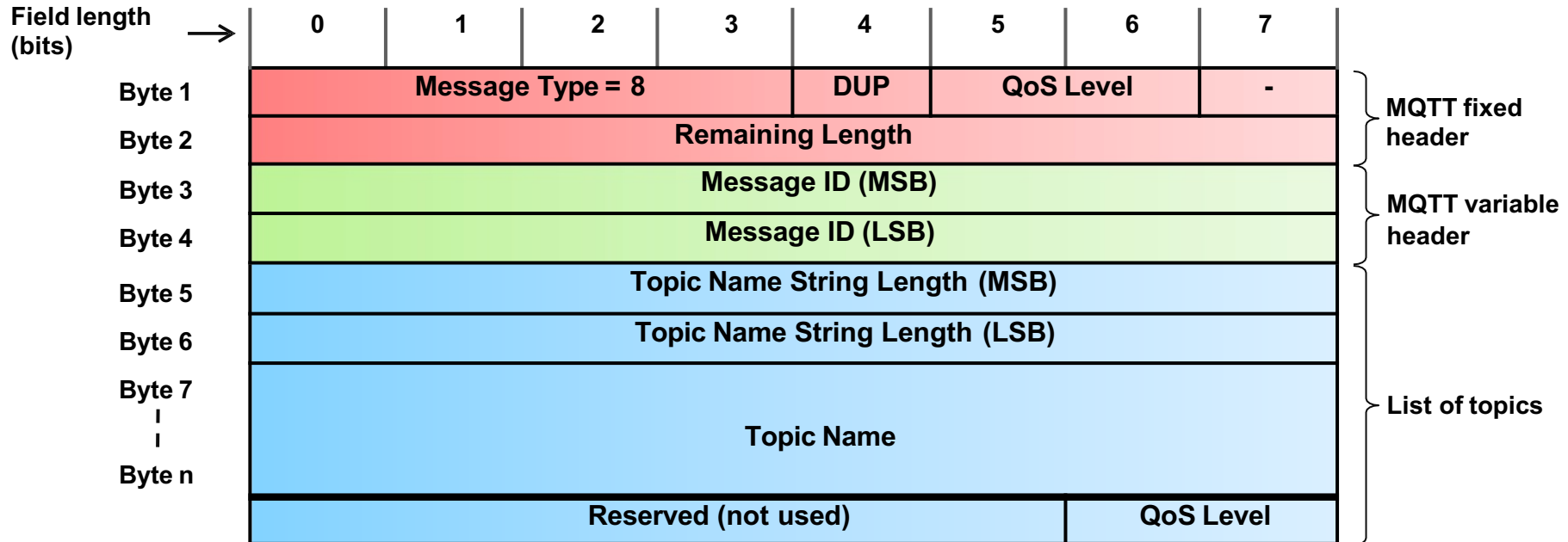
PUBCOMP message format:



PUBCOMP message field	Description / Values
Message ID	The message ID of the PUBLISH message to be acknowledged.

5. MQTT message format (11/14)

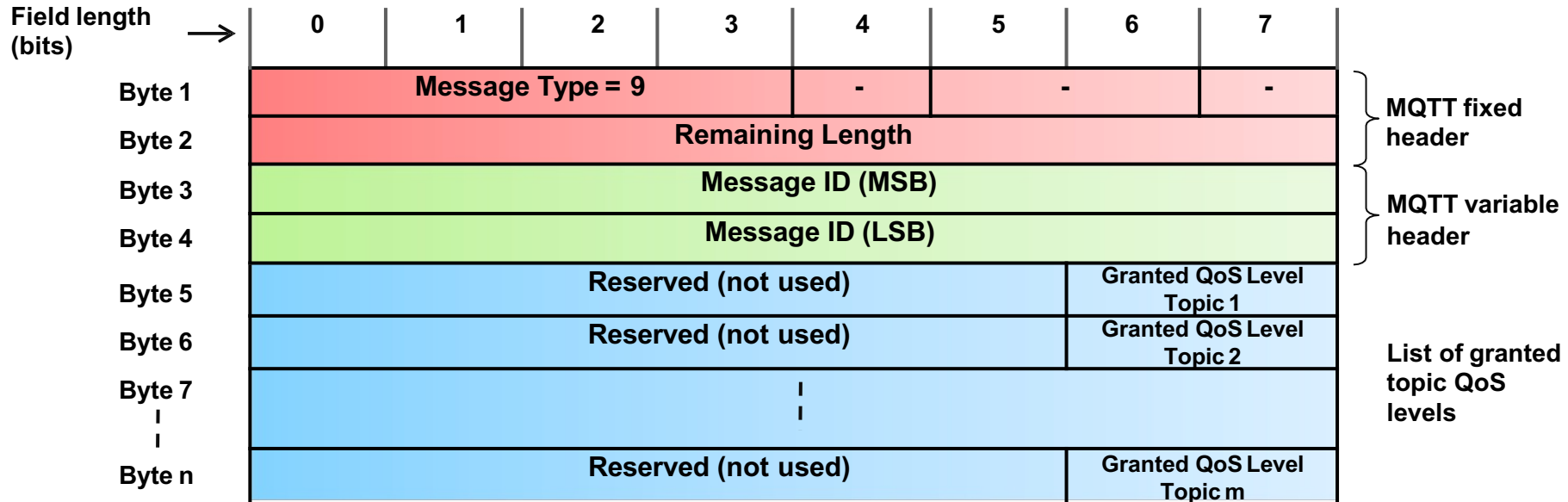
SUBSCRIBE message format:



SUBSCRIBE message field	Description / Values
Message ID	The message ID field is used for acknowledgment of the SUBSCRIBE message since these have a QoS level of 1.
Topic Name with Topic Name String Length	Name of topic to which the client subscribes. The first 2 bytes of the topic name field indicate the topic name string length. Topic name strings can contain wildcard characters as explained under <u>Topic wildcards</u> . Multiple topic names along with their requested QoS level may appear in a SUBSCRIBE message.
QoS Level	QoS level at which the clients wants to receive messages from the given topic.

5. MQTT message format (12/14)

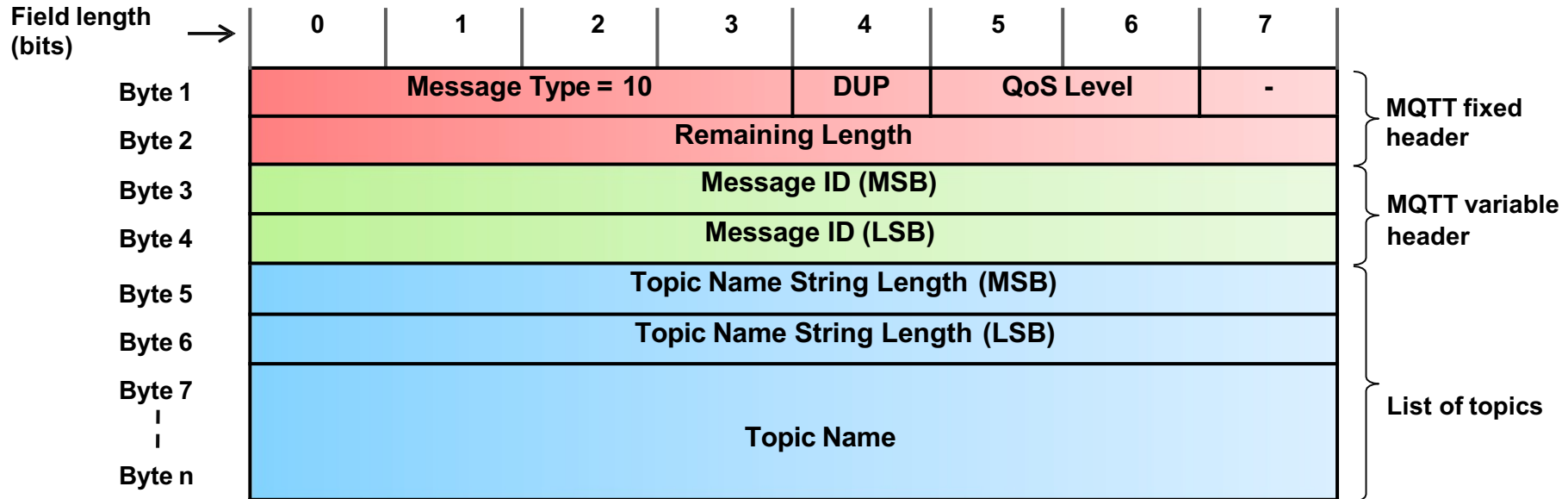
SUBACK message format:



SUBACK message field	Description / Values
Message ID	Message ID of the SUBSCRIBE message to be acknowledged.
Granted QoS Level for Topic	List of granted QoS levels for the topics list from the SUBSCRIBE message.

5. MQTT message format (13/14)

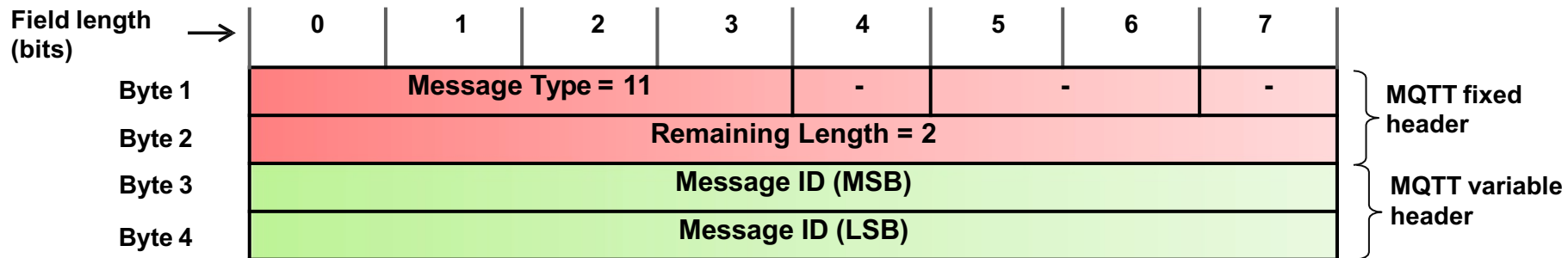
UNSUBSCRIBE message format:



UNSUBSCRIBE message field	Description / Values
Message ID	The message ID field is used for acknowledgment of the UNSUBSCRIBE message (UNSUBSCRIBE messages have a QoS level of 1).
Topic Name with Topic Name String Length	Name of topic from which the client wants to unsubscribe. The first 2 bytes of the topic name field indicate the topic name string length. Topic name strings can contain wildcard characters as explained under <u>Topic wildcards</u> . Multiple topic names may appear in an UNSUBSCRIBE message.

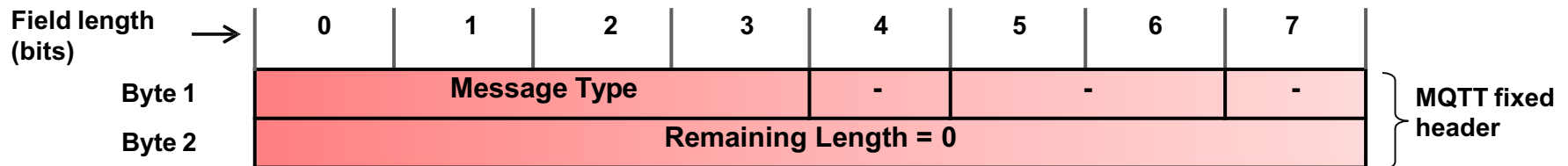
5. MQTT message format (14/14)

UNSUBACK message format:



UNSUBACK message field	Description / Values
Message ID	The message ID of the UNSUBSCRIBE message to be acknowledged.

DISCONNECT, PINGREQ, PINGRESP message formats:



6. MQTT QoS (1/Linux

MQTT provides the typical delivery quality of service (QoS) levels of message oriented middleware.

Even though TCP/IP provides guaranteed data delivery, data loss can still occur if a TCP connection breaks down and messages in transit are lost.

Therefore MQTT adds 3 quality of service levels on top of TCP.



QoS level 0:

At-most-once delivery («best effort»).

Messages are delivered according to the delivery guarantees of the underlying network (TCP/IP).

Example application: Temperature sensor data which is regularly published. Loss of an individual value is not critical since applications (consumers of the data) will anyway integrate the values over time and loss of individual samples is not relevant.

6. MQTT QoS (2/Linux

QoS level 1:

At-least-once delivery. Messages are guaranteed to arrive, but there may be duplicates. Example application: A door sensor senses the door state. It is important that door state changes (closed→open, open→closed) are published losslessly to subscribers (e.g. alarming function). Applications simply discard duplicate messages by evaluating the message ID field.

QoS level 2:

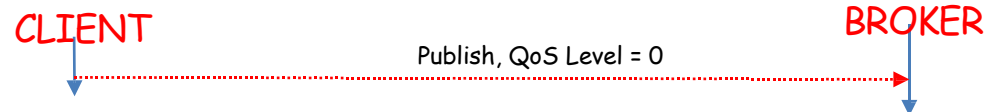
Exactly-once delivery.

This is the highest level that also incurs most overhead in terms of control messages and the need for locally storing the messages.

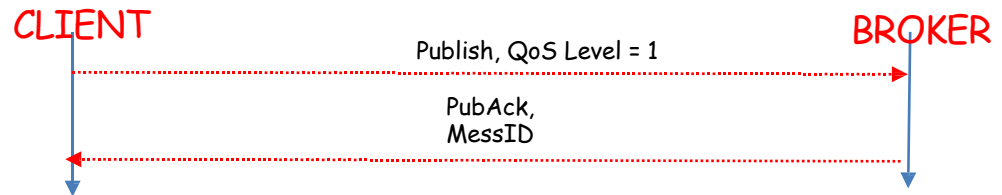
Exactly-once is a combination of at-least-once and at-most-once delivery guarantee.

Example application: Applications where duplicate events could lead to incorrect actions, e.g. sounding an alarm as a reaction to an event received by a message.

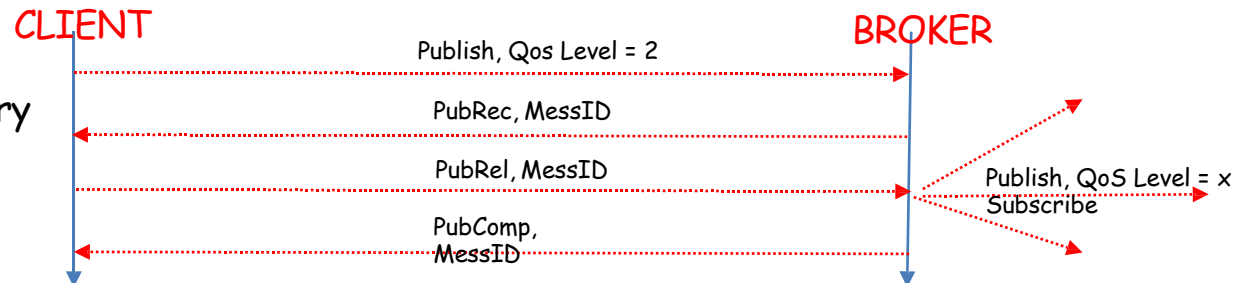
QoS Level = 0 ----- At Most Once Delivery



QoS Level = 1 ----- At Least Once Delivery

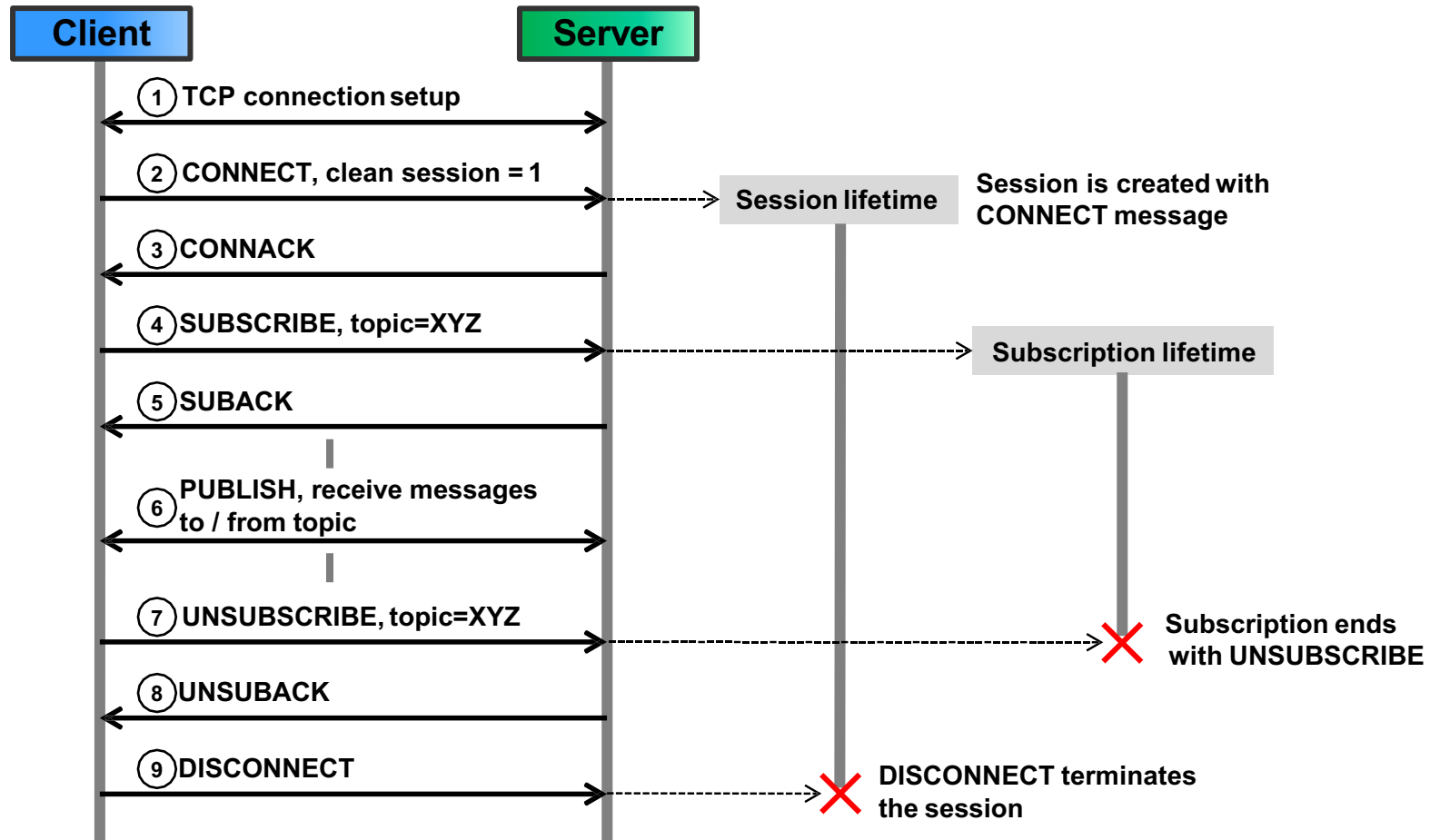


QoS Level = 2 --- Exactly Once Delivery



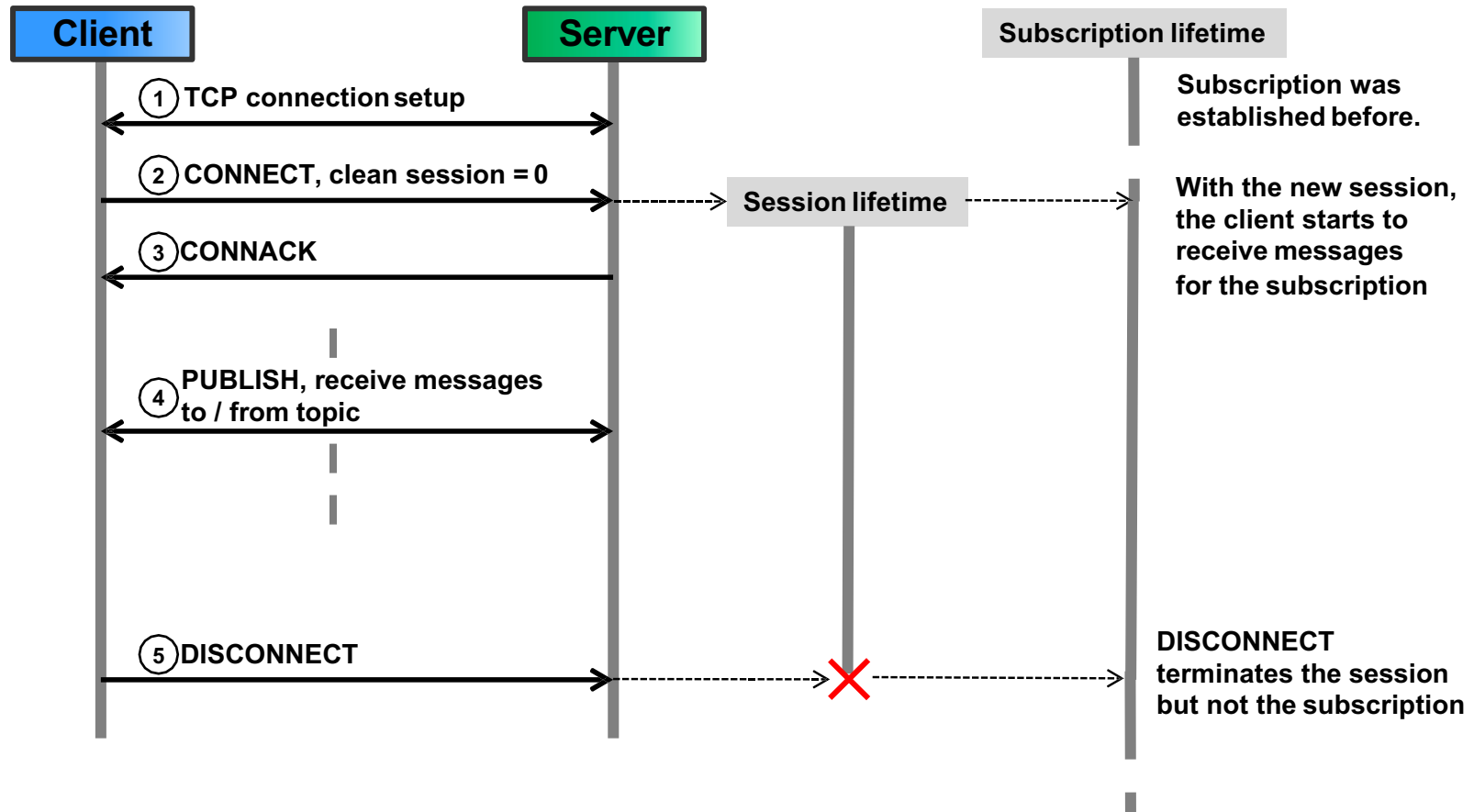
7. CONNECT and SUBSCRIBE message sequence (1/Linux

Case 1: Session and subscription setup with clean session flag = 1 («transient» subscription)



7. CONNECT and SUBSCRIBE message sequence (2/Linux

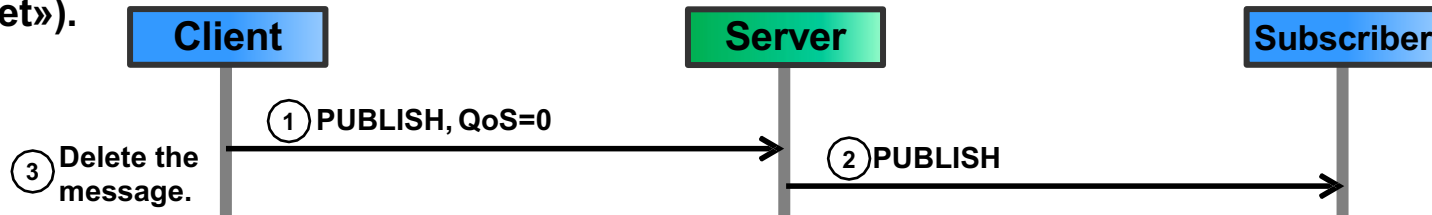
Case 2: Session and subscription setup with clean session flag = 0 («durable» subscription)



8. PUBLISH message flows (1/Linux

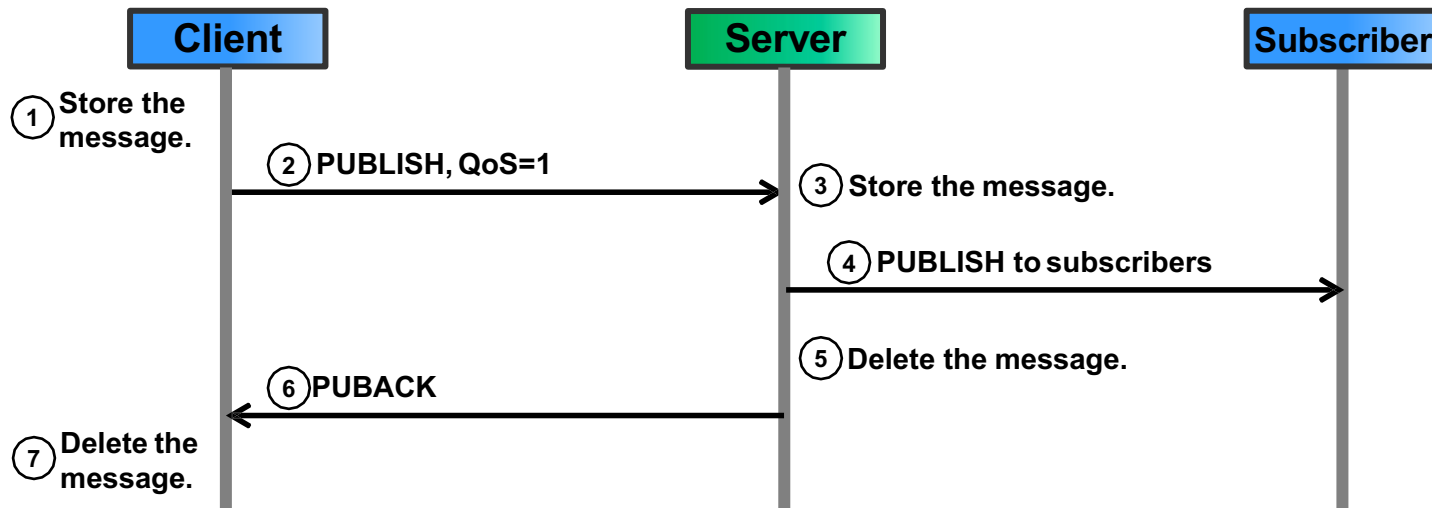
QoS level 0:

With QoS level 0, a message is delivered with **at-most-once** delivery semantics («fire-and-forget»).



QoS level 1:

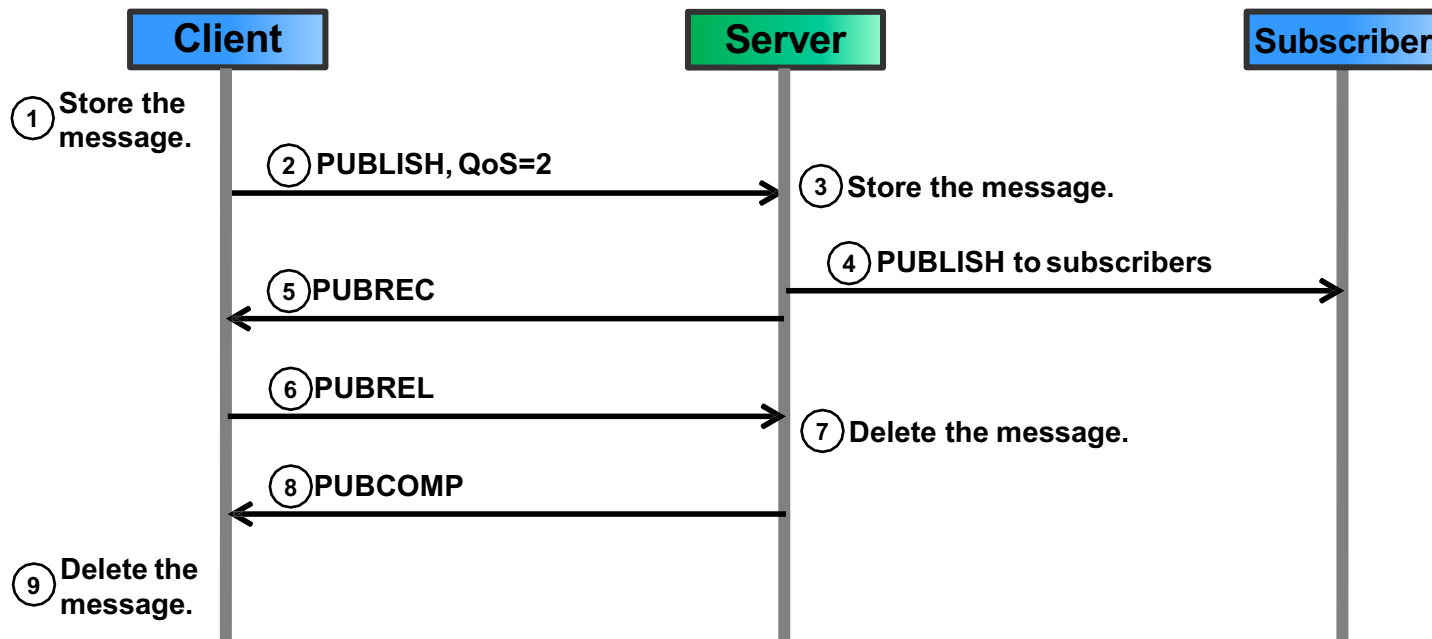
QoS level 1 affords **at-least-once** delivery semantics. If the client does not receive the PUBACK in time, it re-sends the message.



8. PUBLISH message flows (2/Linux

QoS level 2:

QoS level 2 affords the highest quality delivery semantics **exactly-once**, but comes with the cost of additional control messages.



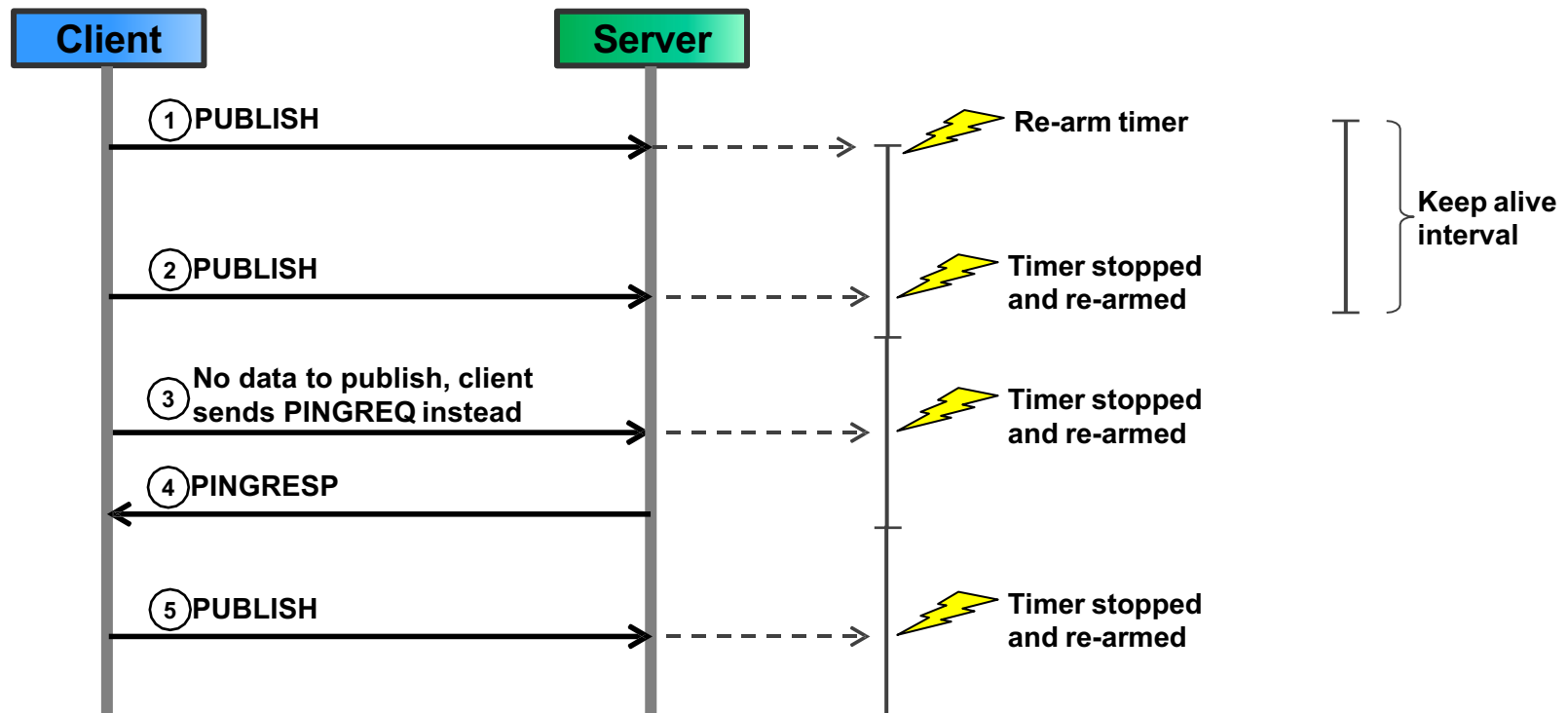
9. Keep alive timer, breath of live with PINGREQ

The keep alive timer defines the maximum allowable time interval between client messages. The timer is used by the server to check client's connection status.

After $1.5 * \text{keepalive-time}$ is elapsed, the server disconnects the client (client is granted a grace period of an additional $0.5 * \text{keepalive-time}$).

In the absence of data to be sent, the client sends a PINGREQ message instead.

Typical value for keepalive timer are a couple of minutes.



10. MQTT will message

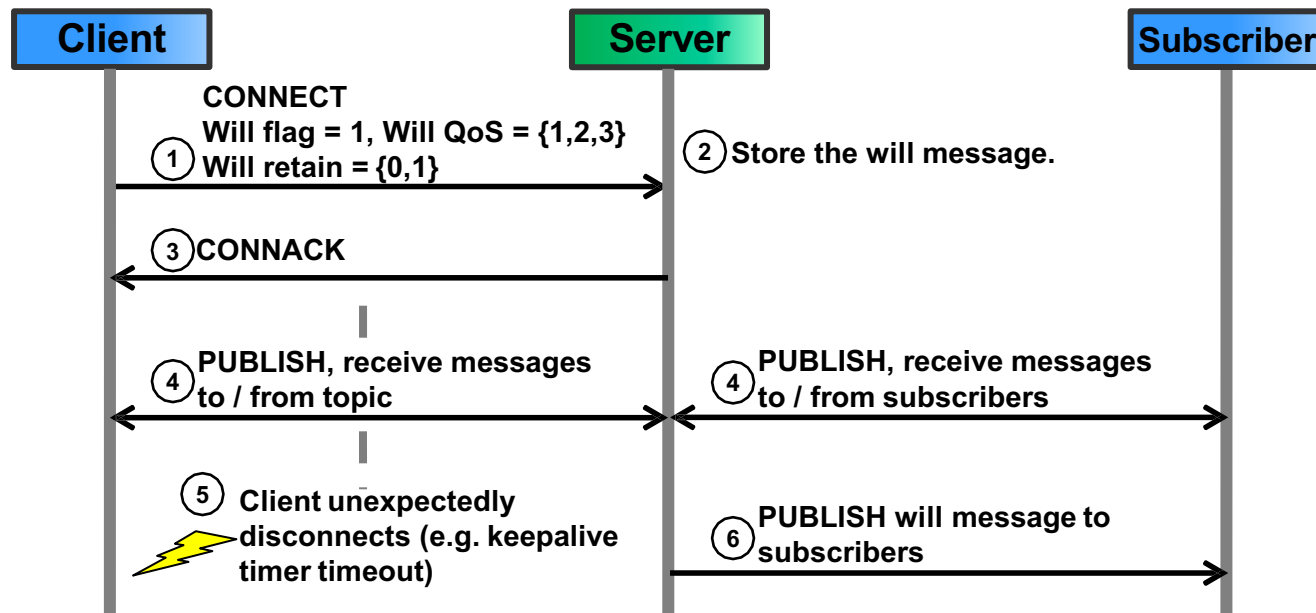
Problem:

In case of an unexpected client disconnect, depending applications (subscribers) do not receive any notification of the client's demise.

MQTT solution:

Client can specify a will message along with a will QoS and will retain flag in the CONNECT message payload.

If the client unexpectedly disconnects, the server sends the will message on behalf of the client to all subscribers («last will»).



11. Topic wildcards

Problem:

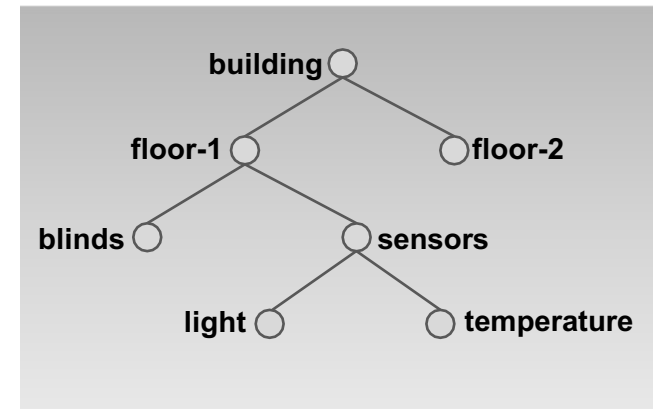
Subscribers are often interested in a great number of topics. Individually subscribing to each named topic is time- and resource-consuming.

MQTT solution:

Topics can be hierarchically organized through wildcards with path-type topic strings and the wildcard characters '+' and '#'.
Subscribers can subscribe for an entire sub-tree of topics thus receiving messages published to any of the sub-tree's nodes.

Topic string special character	Description
/	Topic level separator. Example: <i>building / floor-1 / sensors / temperature</i>
+	Single level wildcard. Matches one topic level. Examples: <i>building / floor-1 / +</i> (matches <i>building / floor-1 / blinds</i> and <i>building / floor-1 / sensors</i>) <i>building / + / sensors</i> (matches <i>building / floor-1 / sensors</i> and <i>building / floor-2 / sensors</i>)
#	Multi level wildcard. Matches multiple topic levels. Examples: <i>building / floor-1 / #</i> (matches all nodes under <i>building / floor-1</i>) <i>building / # / sensors</i> (invalid, '#' must be last character in topic string)

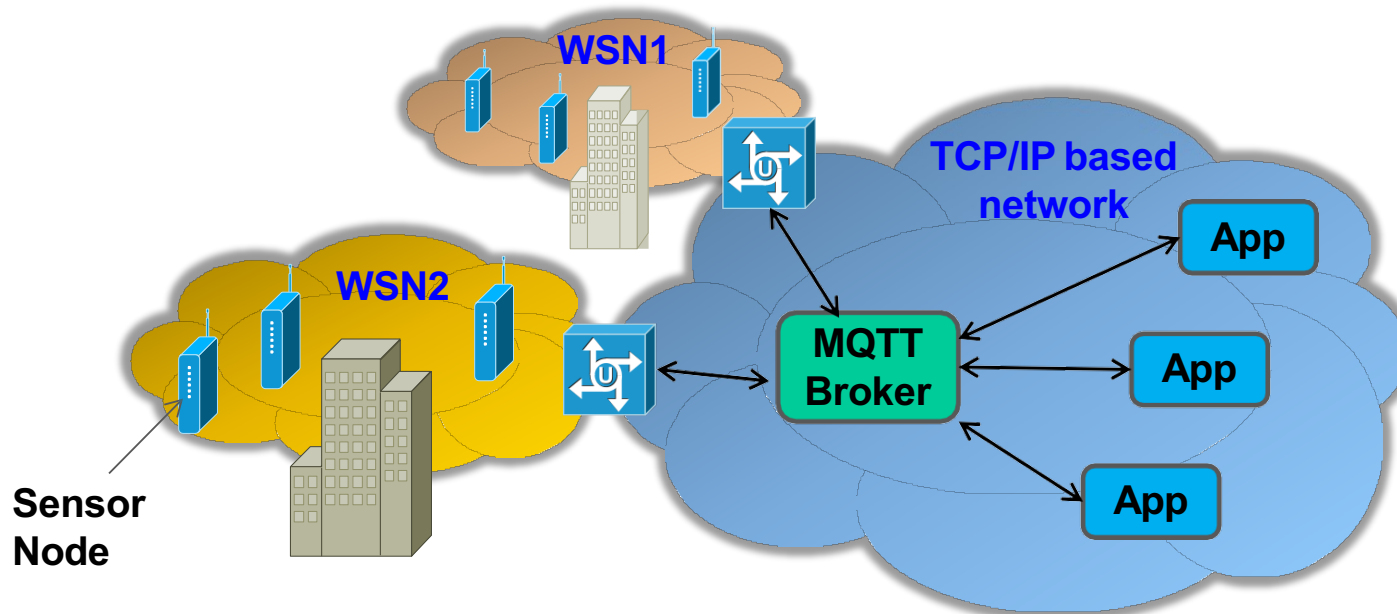
Example topic tree:



12. MQTT-SN (1/Linux – Sensor Networks

WSNs (Wireless Sensor Networks) usually do not have TCP/IP as transport layer. They have their own protocol stack such as ZigBee on top of IEEE 802.15.4 MAC layer. Thus, MQTT which is based on TCP/IP cannot be directly run on WSNs.

WSNs are connected to traditional TCP/IP networks through gateway devices.



MQTT-SN is an extension of MQTT for WSNs.

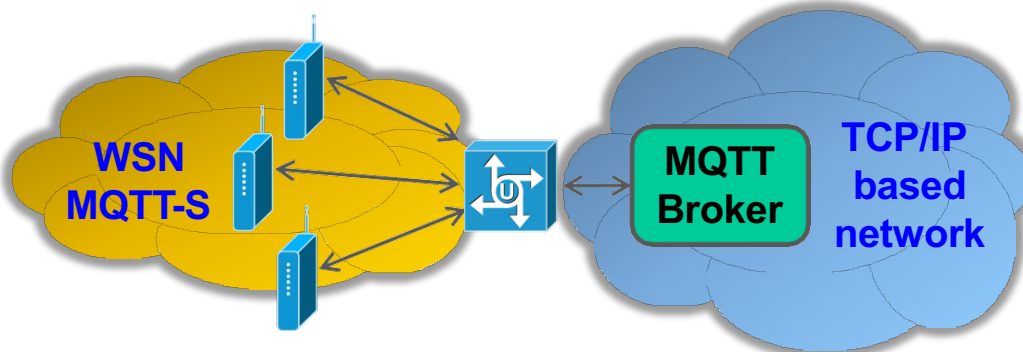
MQTT-SN is aimed at constrained low-end devices, usually running on a battery, such as ZigBee devices.

12. MQTT-SN (2/Linux – Sensor Networks)

MQTT-SN is largely based on MQTT, but implements some important optimizations for wireless networks:

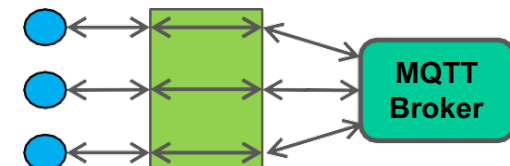
- Topic string replaced by a topic ID (fewer bytes necessary)
- Predefined topic IDs that do not require a registration
- Discovery procedure for clients to find brokers (no need to statically configure broker addresses)
- Persistent will message (in addition to persistent subscriptions)
- Off-line keepalive supporting sleeping clients (will receive buffered messages from the server once they wake up)

MQTT-SN gateways (transparent or aggregating) connect MQTT-SN domains (WSNs) with MQTT domains (traditional TCP/IP based networks).



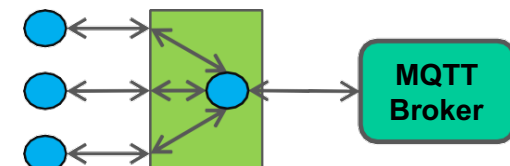
Transparent gateway:

→ 1 connection to broker per client



Aggregating gateway:

→ only 1 connection to the broker



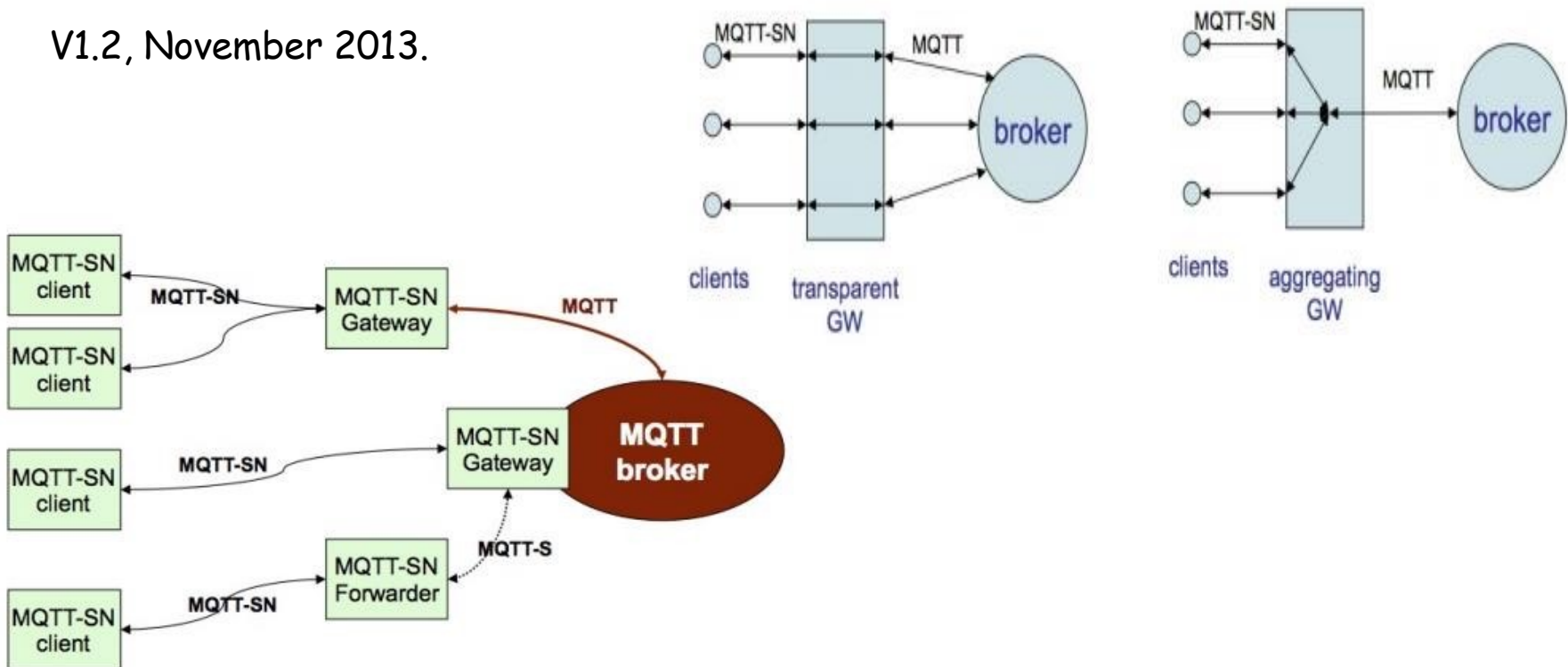
IoT Communications Protocols MQTT

Why MQTT-SN (Sensors Network)?

1. Very Long Packet Size for a 802.15.4 MAC layer
2. TCP as Transport Protocol

MQTT-SN is optimized for low-cost devices implementation, battery-supplied, and with limited computational and processing capabilities.

V1.2, November 2013.



IoT Communications Protocols MQTT

MQTT vs MQTT-SN

MQTT-SN is designed to be as close as possible to MQTT, but is adapted to the peculiarities of a wireless communication environment.

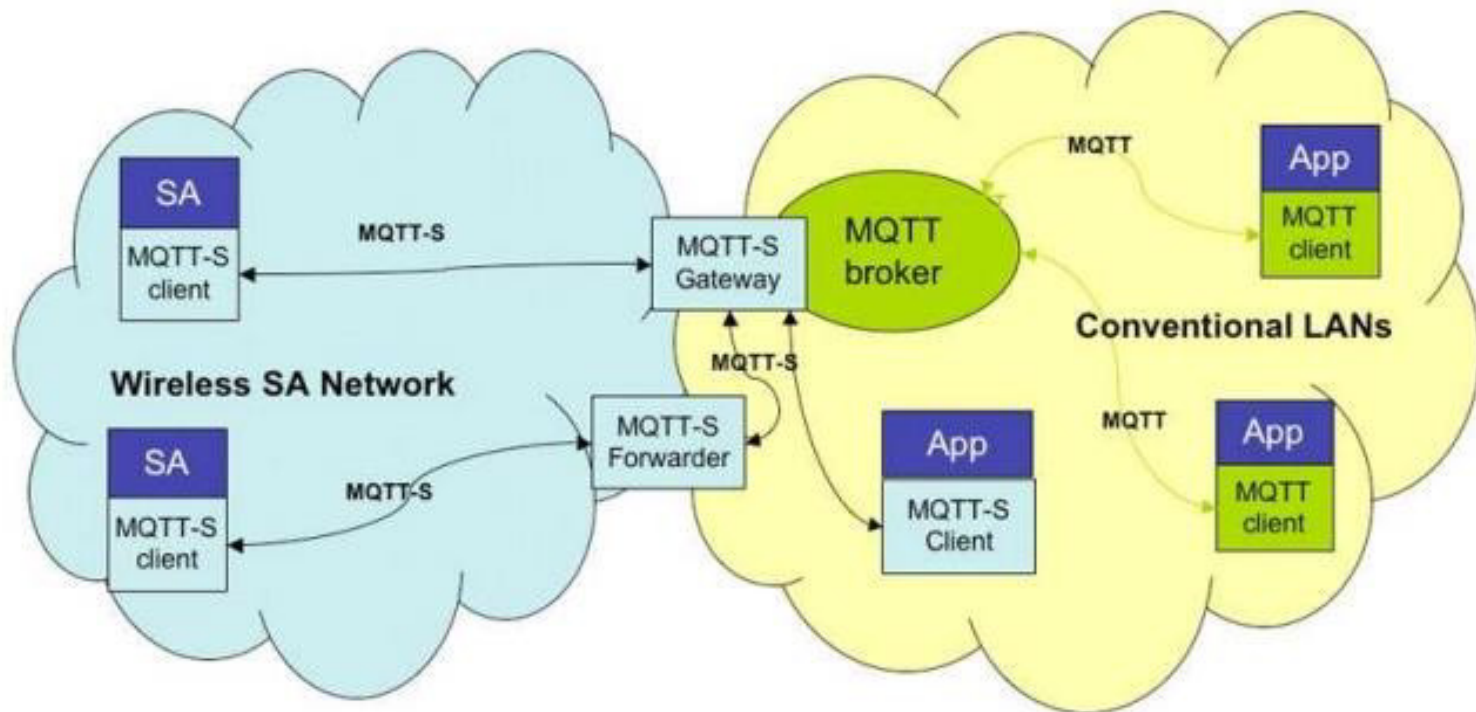
1. **CONNECT** message, divided in three parts (Will Topic - Will Message);
2. Topic Name → Topic ID. Registration Procedure to obtain the ID for a particular Topic Name;
3. Pre-defined Topic ID and Short Topic ID (2bytes-long), for which no registration process is necessary;
4. Discovery Procedure to obtain the MQTT-SN Gateway IP Address;
5. not only client's subscriptions are persistent (RETAIN=1), but also Will topic and Will message.
6. support of sleeping clients: with this procedure, battery-operated devices can go to a sleeping state during which all messages destined to them are buffered at the server/gateway and delivered later to them when they wake up;

	MQTT	MQTT-S
Transport type	Reliable point to point streams	Unreliable datagrams
Communication	TCP/IP	Non-IP or UDP
Networking	Ethernet, WiFi, 3G	ZigBee, Bluetooth, RF
Min message size	2 bytes - PING	1 byte
Max message size	≤ 24MB	< 128 bytes (*)
Battery-operated		✓
Sleeping clients		✓
QoS: -1 "dumb client"		✓
Gateway auto-discovery & fallbacks		✓

ZADATA © 2013

IoT Communications Protocols MQTT

MQTT $\leftrightarrow$ MQTT-SN



IoT Communications Protocols MQTT

MQTT @ OASIS



From March 2013, MQTT is being standardized at OASIS, starting from v3.1 IBM Protocol Specification.

Technical Committee Charter: «The protocol has to support various implementations which run on embedded devices, with limited power, scarce processing and memory requirements, connected to a range of web services or enterprise middleware in constrained environments ».

Targets: refinement of input specifications. Improvements:

1. Message Priority;
2. Payload Typing;
3. Request/Reply Mechanisms;
4. Subscriptions expiration.

Out of Scope:

1. Mapping of the specifications with a particular programming language or middleware;
2. No Reference Implementations for broker entities;
3. No MQTT topic namespace or conventions for topic classification or topic space;
4. No Security Mechanisms will be added, but a Transport Layer Security is assumed.

IoT Communications Protocols MQTT

MQTT: Clients & Brokers

Related Technology Proposals

MQTT Client Implementations:

- WebSphere MQ Telemetry Client (C,Java)
- Eclipse Paho (C, Java, Python, Lua)

Moquette MQTT: creation of a simple and small self-contained Java Implementation of a client broker;

MQTT Server Implementations:

- WebSphere MQ Broker (C, Java);
- Really Small Message Broker, RSMB (C);
- Mosquitto (JMS);
- EMQX

Utility for MQTT:

- Eclipse Paho (Eclipse);
- WMQTT (Java application);

Projects using MQTT

Say It, Sign It (SiSi): helps deaf people by converting speech into British Sign Language, rendered via an MQTT-attached Java avatar. The System uses MQTT and a micro-broker as its messaging infrastructure.

Location Aware Messaging for Accessibility (LAMA): it is a system for making information available in a way that is relevant to their interests and location. The system uses smartphones, MQTT and Websphere Message Broker and some rather clever application software.

Smart Lab: ideated at the University of Southampton, it was a project for monitoring lab experiments in the Chemistry department, and displaying a live dashboard on a Java-enabled cellphone, all using MQTT and the IBM broker technology.

FloodNet: the projects centres upon the development of providing a pervasive, continuous, embedded monitoring presence, by processing and synthesizing collected information over a river and functional floodplain.

IoT Communications Protocols MQTT Libs for iOS

<https://github.com/ckrey/MQTT-Client-Framework>

For iOS MQTT Library there are available several 3rd party libraries.

Using C language libraries or wrapper libraries usually means that there are used POSIX networking calls at some point.

Apple forbids the use of third party networking libraries from using the mobile internet antenna. Thus if one uses C can-only use MQTT, when is connected to a Wi-Fi network.

Therefore, taking into consideration the observations from above and the security constraints - to use native iOS keychain mechanisms instead OpenSSL, there is only one library which is compliant with the requirements - please see the table:

IoT Communications Protocols MQTT Libs for iOS

<https://github.com/ckrey/MQTT-Client-Framework>

Name	Type	Programming Language	Code	Obs/Usage
<u>Paho</u>	<u>Original</u>	<u>C</u>	<u>Open-Source.</u> <u>Eclipse project</u>	<u>Potential no access to GSM because of POSIX calls</u>
<u>IBM</u>	<u>Original</u>	<u>C</u>	<u>Close-Source.</u> <u>IBM SDK</u>	<u>Potential no access to GSM because of POSIX calls</u>
<u>Mosquitto</u>	<u>Original</u>	<u>C</u>	<u>Open-Source.</u> <u>Eclipse project</u>	<u>Potential no access to GSM because of POSIX calls</u>
MQTTKit	Wrapper (Mosquitto)	Objective-C	<u>Open-Source.</u> <u>Github</u>	No longer maintained
Marquette	Wrapper (Mosquitto)	Objective-C	<u>Open-Source.</u> <u>Github</u>	Only for JS iOS not provided
Moscapsule	Wrapper (Mosquitto)	Swift	<u>Open-Source.</u> <u>Github</u>	Security using openssl; not recommended by Apple!
<u>Musqueteer</u>	<u>Wrapper</u> <u>(Mosquitto)</u>	<u>Objective-C</u>	-	<u>No repo available</u>
MQTT-Client-Framework	Native	Objective-C	<u>Open-Source.</u> <u>Github</u>	It is used by an iOS App from Apple store – OwnTracks (https://github.com/owntracks/ios https://itunes.apple.com/us/app/mqttitude/id692424691?mt=8) and has keychain and native security for SSL + no 3rd party usage
MQTTSDK	Native	Objective-C	-	No repo available
CocoaMQTT	Native	Swift	<u>Open-Source.</u> <u>Github</u>	No security

IoT Communications Protocols MQTT

MQTT vs CoAP

MQTT

Many-to-Many Communication Protocol

Decoupling producers and consumers

Data - Centric.

It does best as a communication bus for live data

Clients make a long-lived outgoing TCP connection to a broker

No problem behind NAT

No support for labelling messages. All clients must know the message format up-front to allow the communication.

3 QoS Levels.

CoAP

One-to-One Communication Protocol (for UDP there is multicast)

Transferring State Information between client and server

Document - Centric

Best-suited to a state transfer model, not purely event-based

Clients and servers both send and receive UDP packets.

Tunnelling or Port Forwarding can be used to allow CoAP in NAT environments (IPv4). With IPv6 no problems.

Provides inbuilt support for content negotiation (ACCEPT) and discovery (list), allowing devices to find a way of exchanging data.

Reliability mechanisms is based on NON/CON messages.



1. IoT Communications Protocols CoAP vs. MQTT

MQTT, CoAP, DDS and XMPP are the main competitors for IoT messaging at the Application Layer.

Each one of these has however some **weaknesses**:

- MQTT appears weak in security;
- DDS has problems in terms of scalability and various version dependence;
- XMPP is excessively heavy;
- CoAP not suitable for sending large sums of data and not reliable.

The choice among these is related to the desired **QoS Level**, the **addressing capabilities** and the **particular application**.

QoS is handled by TCP in MQTT, DDS and XMPP, but the mechanism defined there can be heavy in M2M communications. Because it targets device-to-device communications, DDS differs markedly from the other protocols in QoS control, but it is not ideal for device-to-server communications.

In that context MQTT and XMPP are the best-suited, for their discovery procedures.

"The Internet of Things is a big place, with room for many protocols. Choose the one for your application carefully and without prejudice of what you know."

IoT Device Dev Boards and Platforms

IoT Nodes Development Platforms:

- C* (e.g. C-mbed, C-POSIX, C-Arduino)
- Assembly / Firmware (e.g. GNU ARM)
- Lua/Node.js (e.g. NodeMCU vs. Espruino firmware for ESP8266)
- MicroPython
- FreeRTOS vs. Zephyr OS (e.g. on ESP8266/ESP32 RISC-V boards)

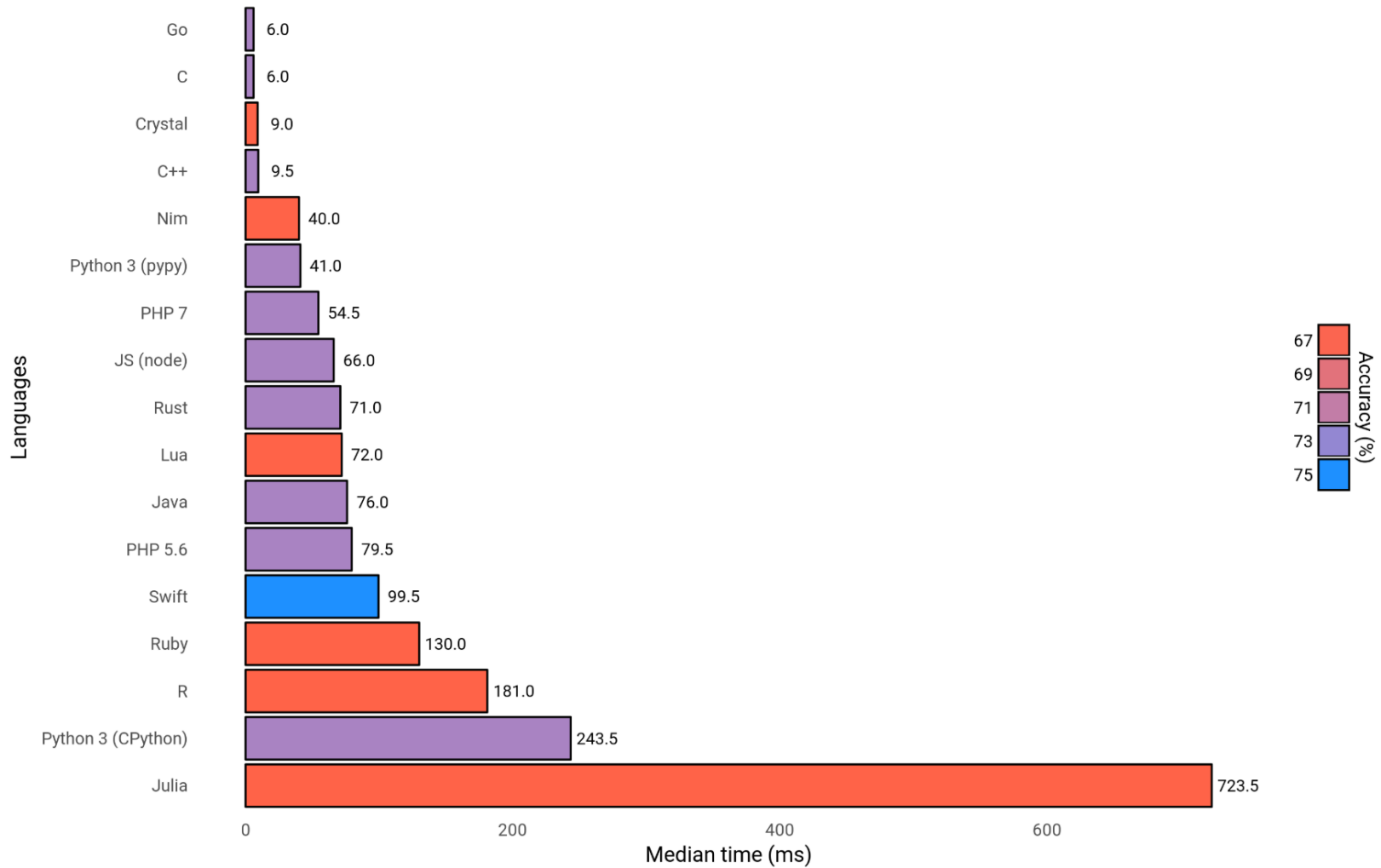
IoT Gateways Development Platforms:

- C* (e.g. C-mbed, C-POSIX, C++, C-Arduino)
- Python 2.x and 3.x
- Java SE-e and Java ME-e
- Swift for ARM
- Node.js – Node-RED (JavaScript)
- C# .Net
- Mobile: iOS-Swift, Android-Java, Windows Mobile/IoT – C# .Net
- Lua
- Ruby/Perl (very rare)
- ... mainly programming languages for Linux Embedded OS (on ARM)

IoT Device Dev Boards and Platforms

Speed comparison of various programming languages

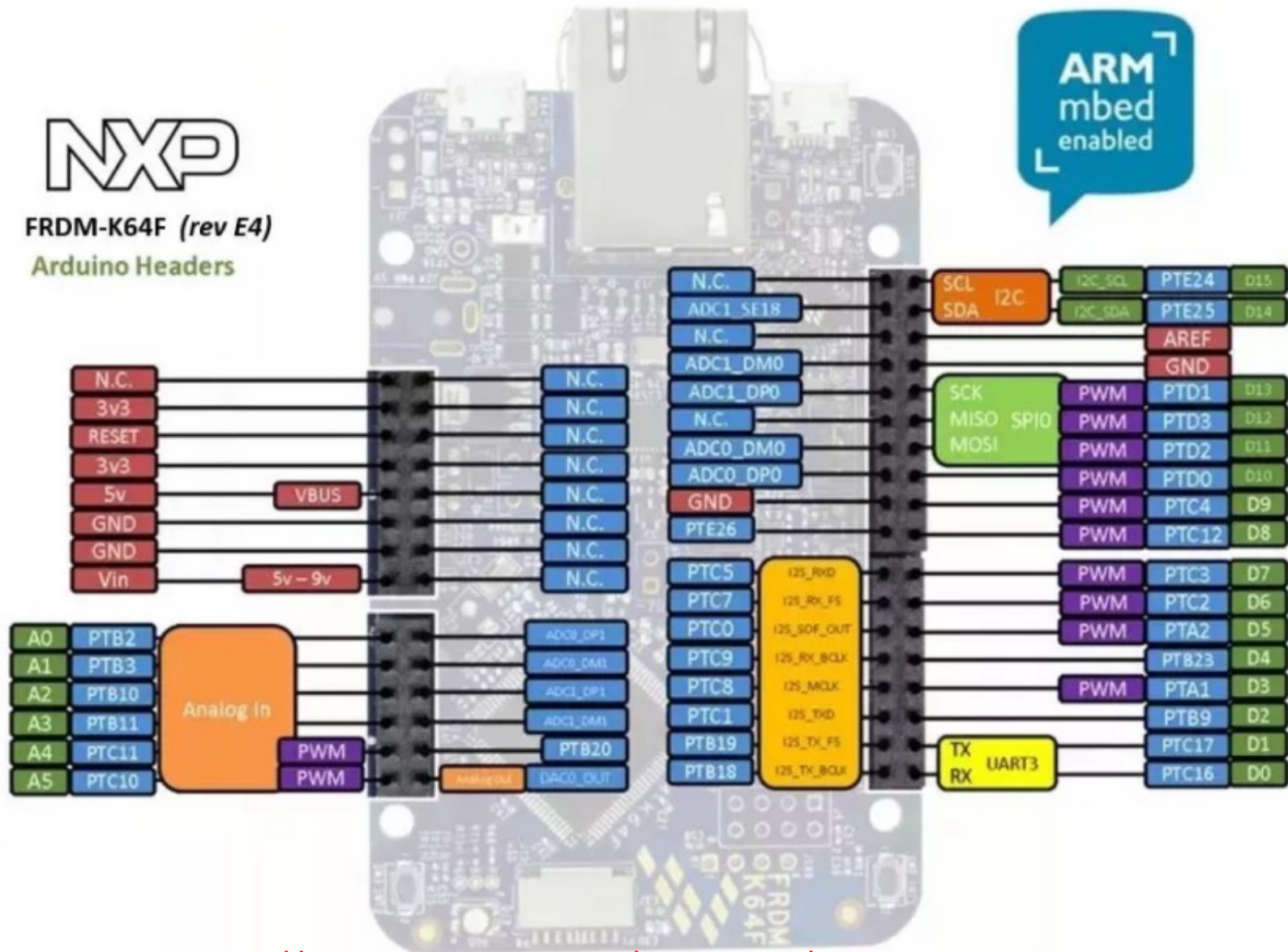
Method: calculating π through the Leibniz formula x times



<https://github.com/niklas-heer/speed-comparison>

IoT Node Device Dev Boards and Platforms

NXP / Freescale FRDM64 (C-mbed on ARM Cortex M*)

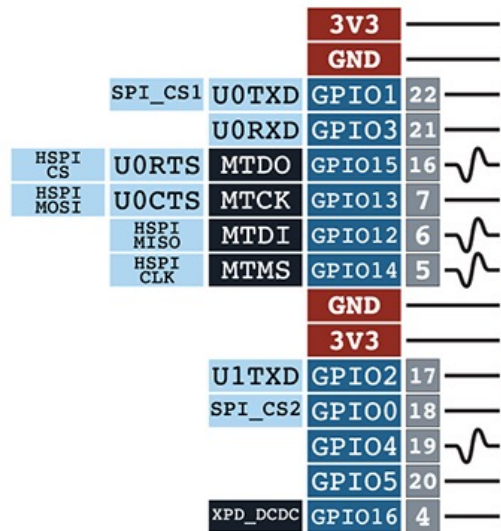


IoT Node Device Dev Boards and Platforms

ESP8266 (AT Commands / Lua – NodeMCU / JS-Node.js – Espruino / MicroPython / Arduino C)

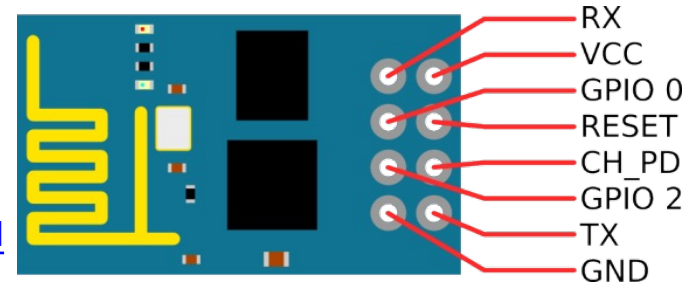
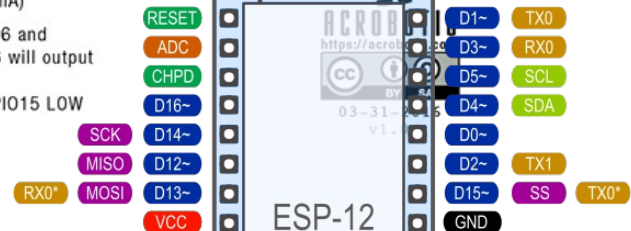
ESP-12E DEVELOPMENT BOARD

PINOUT



NOTES:

- ▲ Typ. pin current 6mA (Max. 12mA)
- ▲ For sleep mode, connect GPIO16 and EXT_RSTB. On wakeup, GPIO16 will output LOW for system reset.
- ▲ On boot/reset/wakeup, keep GPIO15 LOW and GPIO2 HIGH.

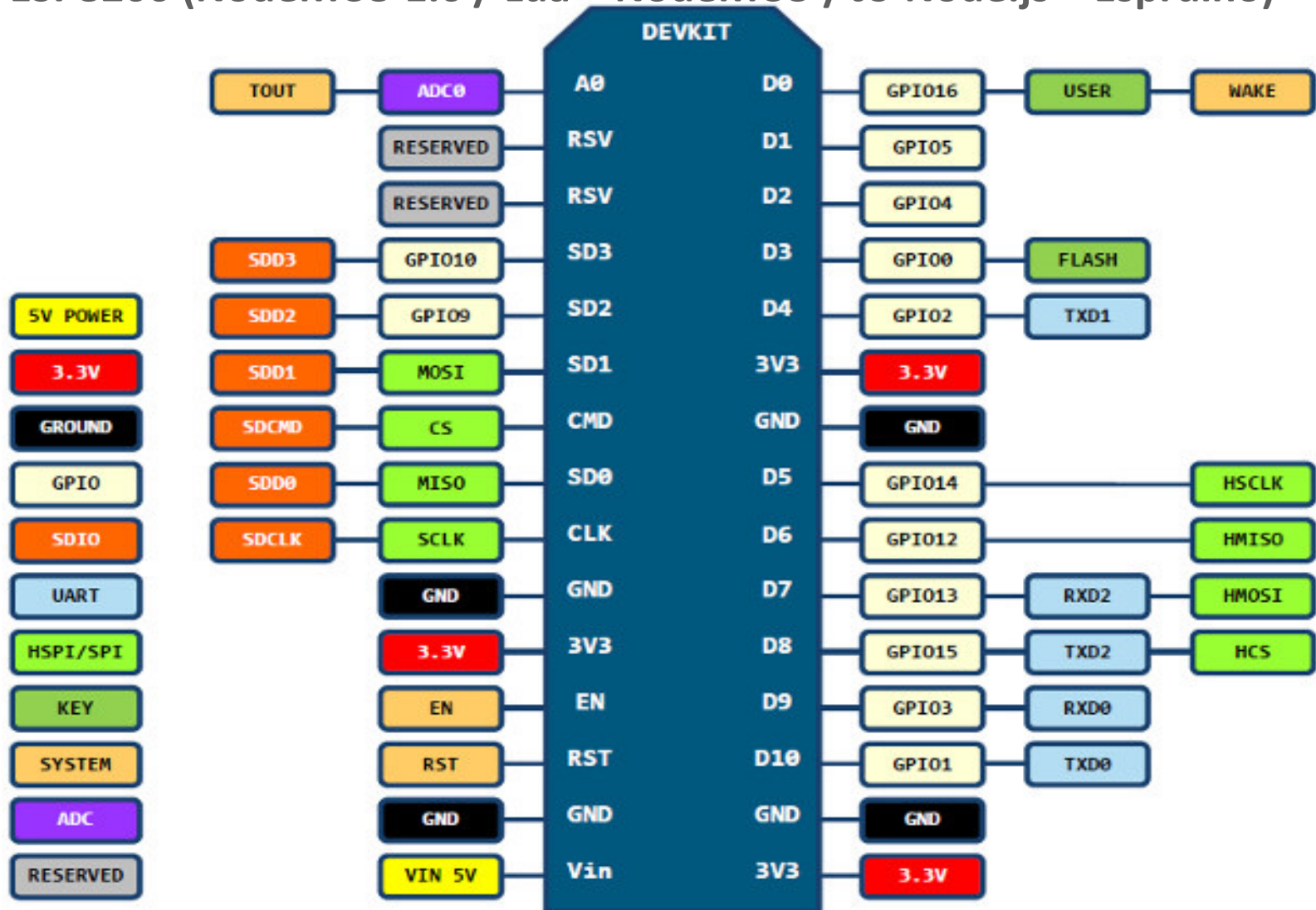


<http://learn.acrobatic.com/tutorials/post/esp8266-getting-started>

<https://acrobatic.com/acr-00018>

IoT Node Device Dev Boards and Platforms

ESP8266 (NodeMCU 1.0 / Lua – NodeMCU / JS-Node.js – Espruino)



D0(GPI016) can only be used as gpio read/write, no interrupt supported, no pwm/i2c/ow supported.

Copyright: <http://www.cnx-software.com/2015/10/29/getting-started-with-nodemcu-board-powered-by-esp8266-wisoc/>

IoT Node Device Dev Boards and Platforms

ESP8266 – NodeMCU – **Lua**: <http://thomaslauer.com/download/luarefv51.pdf>
| <https://www.tutorialspoint.com/lua/>

1. Download the latest firmware on Linux/Raspberry Pi:
@ <https://github.com/nodemcu/nodemcu-firmware/releases> |
<https://github.com/nodemcu/nodemcu-flasher/tree/master/Resources/Binaries> |
https://github.com/nodemcu/nodemcu-firmware/releases/download/0.9.6-dev_20150704/nodemcu_float_0.9.6-dev_20150704.bin
2. Install esptool from Github on Linux/Raspberry Pi:
git clone <https://github.com/themadinventor/esptool.git>
or <https://github.com/espressif/esptool>
3. Erase firmware & Flash the NodeMCU firmware from Linux/Raspberry Pi to ESP8266, after USB connection between boards (also have driver USB to UART – no need in Rpi 3 -

<http://www.silabs.com/products/mcu/pages/usbtouartbridgevcdrivers.aspx>):

Rpi: `sudo esptool.py --port /dev/ttyUSB0 erase_flash`

Mac: `sudo esptool.py --port /dev/tty.wchusbserial14310 erase_flash`

`sudo python ./esptool.py --port /dev/ttyUSB0`

`write_flash 0x00000 ../nodemcu_integer_0.9.6-dev_20150704.bin`

`sudo python ./esptool.py --port /dev/ttyUSB0`

`write_flash 0x00000 ../nodemcu_float_0.9.6-dev_20150704.bin`

Copyright: <http://www.cnx-software.com/2015/10/29/getting-started-with-nodemcu-board-powered-by-esp8266-wisoc/> |

<http://www.esp8266.com/viewtopic.php?p=58281> | <http://www.instructables.com/id/Develop-for-the-ESP8266-on-the-Raspberry-Pi/?ALLSTEPS>

IoT Device Nodes and Gateways Development

Home: <http://esp8266.ru/ESPlorer/>



IoT Node Device Dev Boards and Platforms

ESP8266 - Lua – NodeMCU: Write Code (<http://www.cnx-software.com/2015/10/29/getting-started-with-nodemcu-board-powered-by-esp8266-wisoc/>)

```
Connect to the wireless network
print(wifi.sta.getip()) --nil
wifi.setmode(wifi.STATION)
wifi.sta.config("SSID", "password")
print(wifi.sta.getip()) --192.168.18.110
```

HTTP Client

-- A simple http client

```
conn=net.createConnection(net.TCP, false)
conn:on("receive", function(conn, pl) print(pl) end)
conn:connect(80, "121.41.33.127")
conn:send("GET / HTTP/1.1\r\nHost:
www.nodemcu.com\r\n" .. "Connection: keep-
alive\r\nAccept: */*\r\n\r\n")
```

Arduino like IO access

```
pin = 1
gpio.mode(pin, gpio.OUTPUT)
gpio.write(pin, gpio.HIGH)
gpio.mode(pin, gpio.INPUT)
print(gpio.read(pin))
```

HTTP Server

-- a simple http server

```
srv=net.createServer(net.TCP)
srv:listen(80, function(conn)
  conn:on("receive", function(conn, payload)
    print(payload)
    conn:send("<h1> Hello, NodeMcu.</h1>") end) end)
```

IoT Node Device Dev Boards and Platforms

ESP8266 – Espruino – **JS-Node.js**:

https://www.tutorialspoint.com/nodejs/nodejs_introduction.htm

1. Download the latest firmware on Linux/Raspberry Pi:

@ <http://www.espruino.com/Download> |

http://www.espruino.com/files/espruino_1v89.zip

2. Install esptool from Github on Linux/Raspberry Pi:

git clone <https://github.com/themadinventor/esptool.git>

or <https://github.com/espressif/esptool>

3. Erase firmware & Flash the Espruino firmware from Linux/Raspberry Pi to ESP8266, after USB connection between boards (also have driver USB to UART – no need in Rpi 3 -

<http://www.silabs.com/products/mcu/pages/usbtouartbridgevcdrivers.aspx>):

RPi: `sudo esptool.py --port /dev/ttyUSB0 erase_flash`

Mac: `sudo esptool.py --port /dev/tty.wchusbserial14310 erase_flash`

RPi: `sudo python esptool.py --port /dev/ttyUSB0 -b 115200 write_flash -ff 80m -fm qio -fs 32m 0x0000 "boot_v1.4(b1).bin" 0x1000 espruino_esp8266_user1.bin 0x37E000 blank.bin`

Mac: `sudo python esptool.py --port /dev/tty.wchusbserial14310 -b 115200 write_flash -ff 80m -fm qio -fs 32m 0x0000 "boot_v1.4(b1).bin" 0x1000 espruino_esp8266_user1.bin 0x37E000 blank.bin`

IoT Node Device Dev Boards and Platforms

ESP8266 - Lua – NodeMCU

4. Download Espruino IDE tool as Google Chromium plugin in RPi:

<https://chrome.google.com/webstore/detail/espruino-web-ide/bleoifhkdalbjfbobjackfdifdneehpo?hl=en>

Espruino Web IDE
offered by www.espruino.com
★★★★★ (37) | [Developer Tools](#) | 11,082 users

OVERVIEW | REVIEWS | SUPPORT | RELATED

ABOUT

About the Espruino Web IDE v0.65.0

Credits

- [Web App](#): Google for supporting function, making this editor possible
- [Duke](#): jQuery team for jQuery and jQuery UI, this reduced time for development a lot
- [FileSaver](#): Eli Grey for the library to support local files
- [CodeMirror](#): the team of codemirror for the toolbar behind the editor window
- [Blockly](#): Neil Fraser and the team of Blockly
- [Blockly](#): Code Seven for the Blockly modification plugin
- [Modern UI Icons](#): Austin Andrews of Templarian.com for the Modern UI icons
- [Matt Stratford](#): UI design + Web IDE improvements
- [Juergen Marach](#): Web IDE improvements
- [Gordon Williams](#): Initial Web IDE

Board Information

VERSION	1v60
BUILD_DATE	Apr 4 2014
BUILD_TIME	11:03:28
BOARD	ESP8266
CHIP	STW32P103ECT6
CHIP_FAMILY	STW32P1
FLASH	262144
RAM	49152
SERIAL	34f0b00-41563230-36710443
CONSOLE	US5

PERMISSIONS this app needs:

- * Serial port : for communication with the Espruino board
- * Audio : for communication with an Espruino board via the headphone jack: <http://www.espruino.com/Headphone>
- * Webcam : so that teachers can present live

[Website](#)
[Report Abuse](#)

Additional Information

Version: 0.65.7
Updated: December 7, 2016
Size: 1.48MiB
Language: English

Keep baudrate 115200 for uploading JS-Node.js scripts (Settings-> Communications) from Rpi in ESP8266
| <http://forum.espruino.com/conversations/281522/> | <https://odd-one-out.serek.eu/esp8266-nodemcu-dht22-mqtt-deep-sleep/> | <http://forum.espruino.com/conversations/281507/>

IoT Node Device Dev Boards and Platforms

ESP8266 – JS-Node.js: Write Code

Arduino like IO access

```
var led = new Pin(2);
var toggle=1;
setInterval(
  function() {
    toggle=!toggle;
    digitalWrite(led, toggle);
  }, 500);
```

Arduino like IO access

```
var led = NodeMCU.D1;
var toggle=1;
setInterval(
  function() {
    toggle=!toggle;
    digitalWrite(led, toggle);
  }, 500);
```

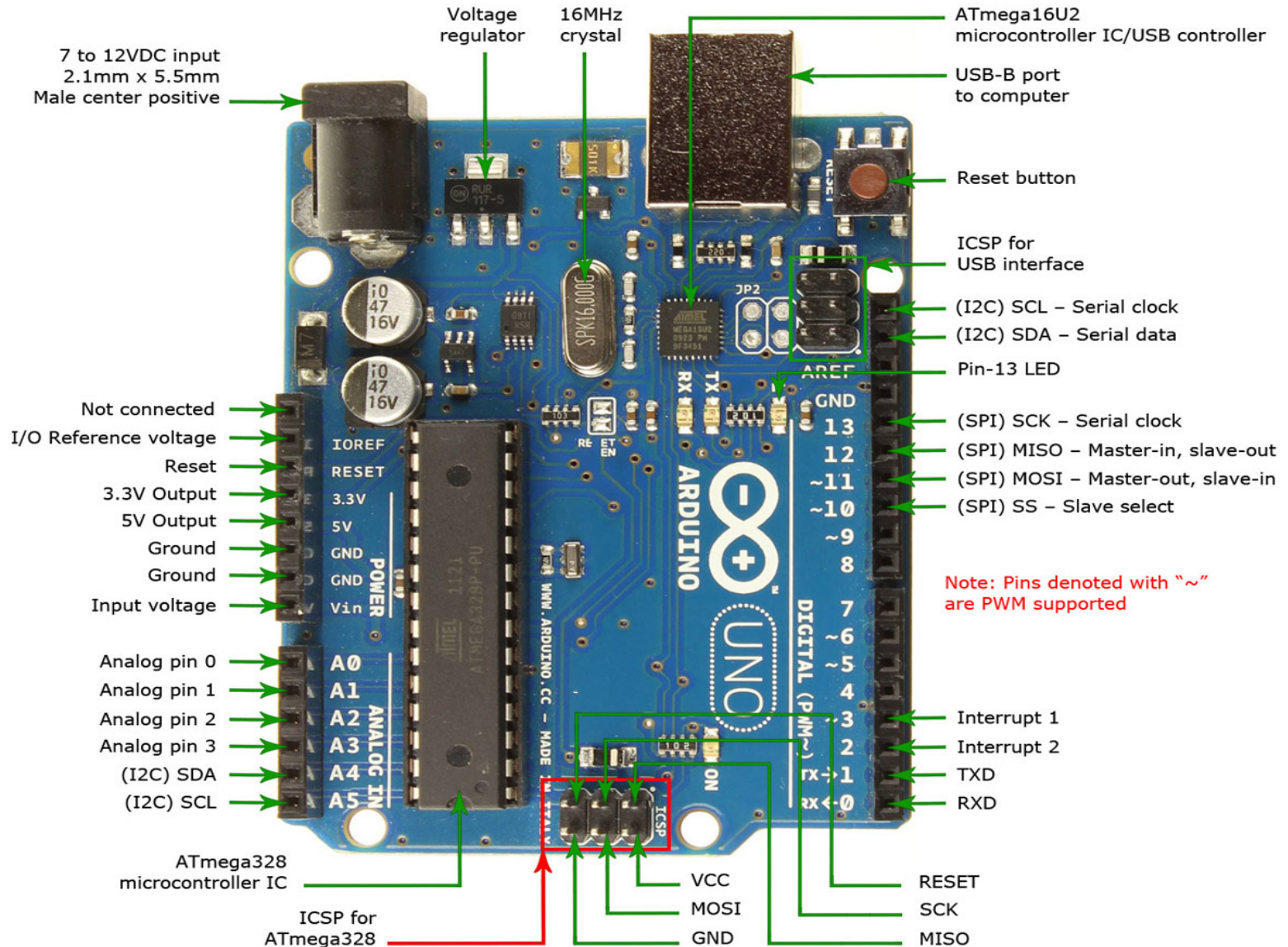
Wi-Fi access

```
var wifi = require("Wifi");
wifi.connect("SSID", {password:"wpa2pass"}, function(err) {
  console.log("connected? err=", err, "info=", wifi.getIP() );
});
wifi.save();
wifi.stopAP();
```

<http://www.espruino.com/Reference#NodeMCU>

IoT Node Device Dev Boards and Platforms

Arduino (C-Arduino on ATmega16)



IoT Node Device Dev Boards and Platforms

Arduino UNO (C-Arduino on ATmega16)

Structure

Arduino programs can be divided in three main parts: **Structure**, **Values** (variables and constants), and **Functions**. In this tutorial, we will learn about the Arduino software program, step by step, and how we can write the program without any syntax or compilation error.

Let us start with the **Structure**. Software structure consist of two main functions –

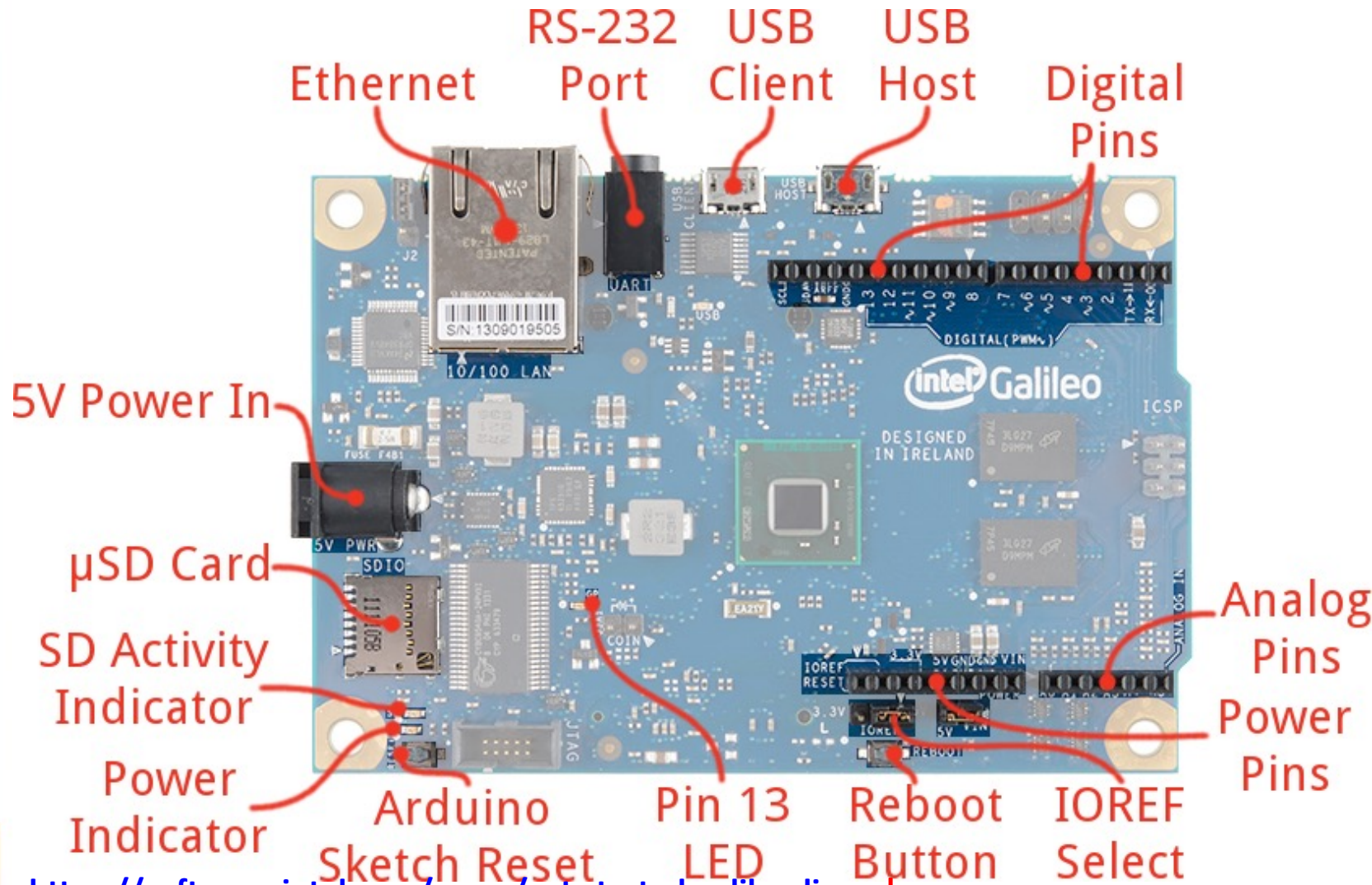
- Setup() function
- Loop() function



<http://www.tutorialspoint.com/arduino/> | <https://www.arduino.cc/en/Tutorial/HomePage>
<https://www.arduino.cc/en/Guide/Libraries> |
<https://github.com/RobTillaart/Arduino/tree/master/libraries/DHTlib>
| <http://playground.arduino.cc/Main/DHTLib>

IoT Node Device Dev Boards and Platforms

Intel Galileo Gen 1&2 (C-Arduino in SoC | C-Posix / Node.js /... in Yocto Linux on Intel Quark SoC)



<https://software.intel.com/en-us/get-started-galileo-linux>

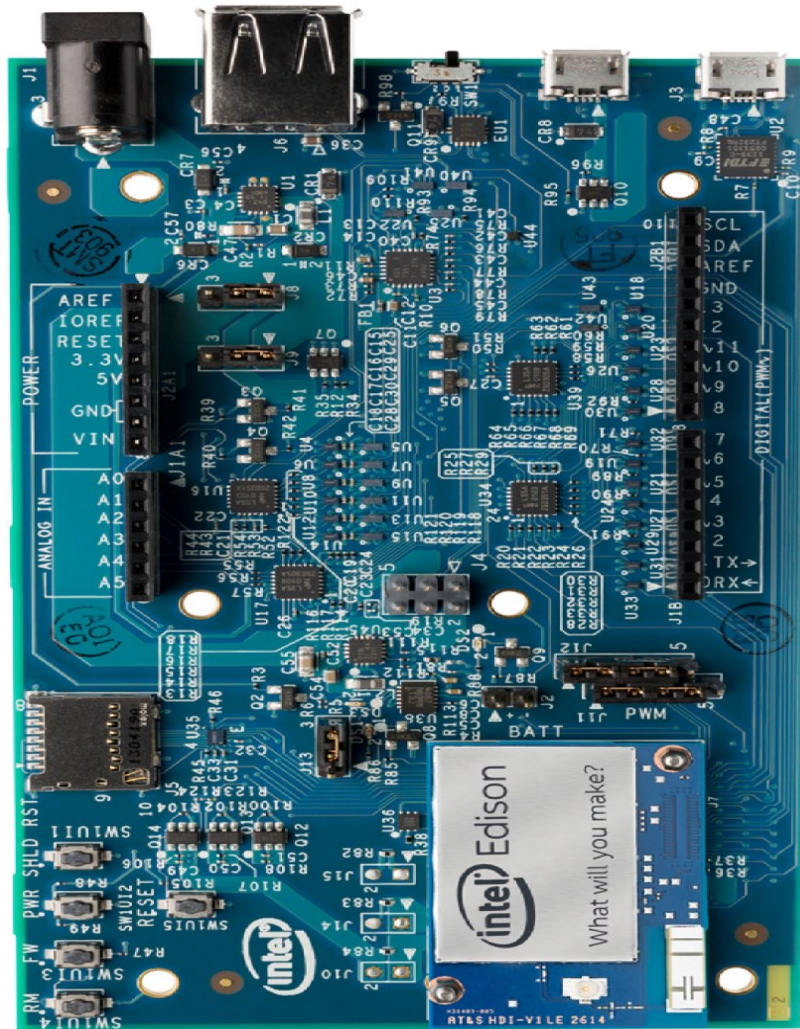
<https://www.arduino.cc/en/Guide/IntelGalileoGen2>

IoT Device Nodes and Gateways Development

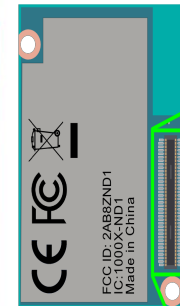
IoT Node & GW Device Dev Boards and Platforms

Intel Edison (C-Arduino in SoC | C-Posix / Node.js /... in Yocto Linux on 2 CPU ATOM cores + 1 SoC Quark)

Real Platforms: Intel NUC Gateway



Intel® Edison Pinout



(Bottom view)

VSYS 2.	1. GND
VSYS 4.	3. USB_ID
VSYS 6.	5. GND
3.3V 8.	7. MSIC_SLP_CLK3
3.3V 10.	9. GND
1.8V 12.	11. GND
DCIN 14.	13. GND
USB_DP 16.	15. GND
USB_DN 18.	17. PWRBTN#
USB_VBUS 20.	19. FAULT
GP134/UART_2_RX 22.	21. PSW
GP44 24.	23. V_VBAT_BKP
GP45 26.	25. GP165
GP46 28.	27. GP135/UART_2_TX
GP47 30.	29. NC
GP48 32.	31. RCVR_MODE
GP49 34.	33. GP13/PWM_1
RESET_OUT# 36.	35. GP12/PWM_0
NC 38.	37. GP182/PWM_2
NC 40.	39. GP183/PWM_3
GP15 42.	41. GP19/I2C_1_SCL
GP84/SD_0_CLK_FB 44.	43. GP20/I2C_1_SDA
GP131/UART_1_TX 46.	45. GP27/I2C_6_SCL
GP14 48.	47. GP28/I2C_6_SDA
GP42/I2S_2_RXD 50.	49. NC
GP40/I2S_2_CLK 52.	51. GP111/SPI_2_FS1
GP41/I2S_2_FS 54.	53. GP110/SPI_2_FS0
GP43/I2S_2_TXD 56.	55. GP109/SPI_2_CLK
GP78/SD_0_CLK 58.	57. GP115/SPI_2_TXD
GP77/SD_0_CD# 60.	59. GP114/SPI_2_RXD
GP79/SD_0_CMD 62.	61. GP130/UART_1_RX
GP82/SD_0_DAT2 64.	63. GP129/UART_1_RTS
GP80/SD_0_DAT0 66.	65. GP128/UART_1_CTS
GP83/SD_0_DAT3 68.	67. OSC_CLK_OUT_0
GP81/SD_0_DAT1 70.	69. FW_RCVR

sparkfun
ELECTRONICS

<https://software.intel.com/en-us/get-started-edison-osx> | <https://software.intel.com/en-us/get-started-edison-linux> | <https://software.intel.com/en-us/iot/tools-ide/ide>

IoT Node/GW Device Dev Boards and Platforms

Intel Edison vs. Galileo

Galileo

- CPU: Intel Quark X1000 400 MHz
- RAM: 256 MB
- Storage: Micro SD Card

Edison

- CPU: A dual core, dual threaded Intel ATOM x86 CPU running at 500 MHz and a 32-bit Intel Quark Micro-controller running at 100 MHz.
- RAM: 1 GB
- Storage: 4 GB ROM + (micro SD card on Arduino board)
- Communication: Wi-Fi and Bluetooth LE.

Summary

- Edison is way more powerful in terms of CPU (ATOM vs Quark) and RAM.

Stackoverflow.com:

Intel(R) Edison is a product-ready, general-purpose compute platform optimized to enable rapid innovation and product development. Intel Edison is ideal for small form factor devices that require a powerful computing system. Some good use cases are robots and quadcopters, 3D fabrication machines, remote asset monitoring, and audio processing.

Intel(R) Galileo is an open source, Arduino-compatible platform that enables educators, students, and makers of all skill levels to quickly and easily engage in projects. It combines the simplicity of the Arduino development environment with the performance of Intel technology and the advanced capabilities of a full Linux software stack.

A really great place to learn more about both platforms is our online community at **maker.intel.com**.

IoT Node/GW Device Dev Boards and Platforms: Intel

<https://software.intel.com/en-us/iot/tools-ide/ide>

<https://software.intel.com/en-us/intel-xdk>

<https://software.intel.com/en-us/get-started-arduino>

<https://software.intel.com/en-us/iot/tools-ide/ide/iss-iot-edition>

<https://software.intel.com/en-us/intel-system-studio-microcontrollers>

<https://github.com/intel-iot-devkit/upm/tree/master/examples/python>

WindRiver: <https://software.intel.com/en-us/iot/hardware/gateways>

The Intel® IoT Developer Kit is programmable using Arduino*, C/C++, JavaScript*, Node.js*, and Python*. Explore the list below to find the best solution for you.



Intel® XDK

Create web interfaces, add sensors to your project, and work with the cloud. This developer kit includes companion templates to get your project up and running quickly.



Arduino*

With readily available code from a variety of manufacturers, quickly add sensors to your project with this intuitive interface.



Intel® System Studio IoT Edition

This Eclipse*-based IDE comes with the built-in capability to easily integrate sensors via UPM and MRAA libraries, which you can develop in C/C++ or Java.



Intel® System Studio for Microcontrollers

Develop for Intel® Quark™ microcontrollers using this Eclipse*-based software suite. Effective debug capabilities, powerful library extensions, and code portability enable innovation for low-power connected devices.



Python*

Even though Intel does not provide an IDE for Python*, the Python programming language comes preinstalled on your board, plus you can find Python support in the sensor library.



Wind River* Intelligent Device Platform XT

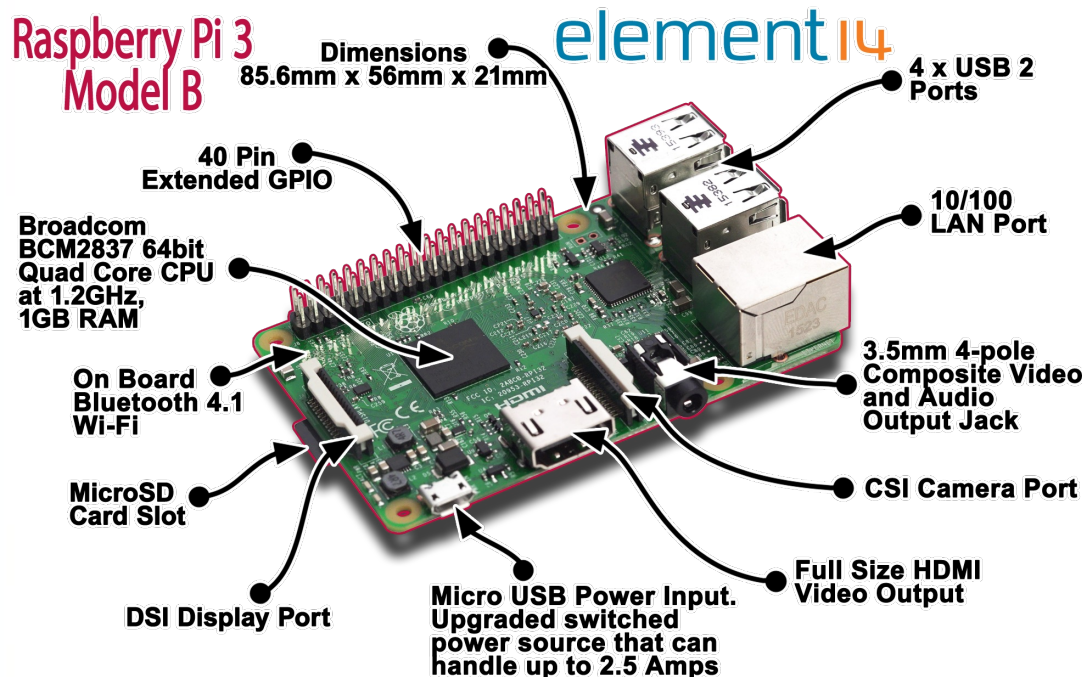
Harness the connectivity, security, and manageability features of the Wind River* Intelligent Device Platform XT.

IoT Gateway Device Dev Boards and Platforms

Raspberry Pi Model 1 / 2 / 3 / 4 with A / B / B+ layout

C POSIX/C++, Java SE-e / Java ME, Python, Node.js/Node-RED, Swift

<http://dev.iachieved.it/iachievedit/swift-3-0-on-raspberry-pi-2-and-3/>



Raspberry Pi 3 GPIO Header

Pin#	NAME	NAME	Pin#
01	3.3v DC Power	DC Power 5v	02
03	GPIO02 (SDA1, I ² C)	DC Power 5v	04
05	GPIO03 (SCL1, I ² C)	Ground	06
07	GPIO04 (GPIO_GCLK)	(TXD0) GPIO14	08
09	Ground	(RXD0) GPIO15	10
11	GPIO17 (GPIO_GEN0)	(GPIO_GEN1) GPIO18	12
13	GPIO27 (GPIO_GEN2)	Ground	14
15	GPIO22 (GPIO_GEN3)	(GPIO_GEN4) GPIO23	16
17	3.3v DC Power	(GPIO_GEN5) GPIO24	18
19	GPIO10 (SPI_MOSI)	Ground	20
21	GPIO09 (SPI_MISO)	(GPIO_GEN6) GPIO25	22
23	GPIO11 (SPI_CLK)	(SPI_CE0_N) GPIO08	24
25	Ground	(SPI_CE1_N) GPIO07	26
27	ID_SD (I ² C ID EEPROM)	(I ² C ID EEPROM) ID_SC	28
29	GPIO05	Ground	30
31	GPIO06	GPIO12	32
33	GPIO13	Ground	34
35	GPIO19	GPIO16	36
37	GPIO26	GPIO20	38
39	Ground	GPIO21	40

IoT Gateway Device Dev Boards and Platforms

Raspberry Pi Model 1 / 2 / 3 with A / B / B+ layout

C POSIX/C++, Java SE-e / Java ME, Python, Node.js/Node-RED, Swift,
ARM - http://elinux.org/RPi_GPIO_Code_Samples

C on ARM: <http://www.valvers.com/open-software/raspberry-pi/> |
<http://www.valvers.com/open-software/raspberry-pi/creating-a-bootable-sd-card/> |
<http://www.valvers.com/open-software/raspberry-pi/step01-bare-metal-programming-in-cpt1/> | <http://www.valvers.com/open-software/raspberry-pi/step02-bare-metal-programming-in-c-pt2/> ... <http://www.valvers.com/open-software/raspberry-pi/step02-bare-metal-programming-in-c-pt5/>

Java SE-e & OpenJDK DIO: <http://openjdk.java.net/projects/dio/> |
<https://wiki.openjdk.java.net/display/dio/Getting+Started> |
<https://www.tutorialspoint.com/java/> | <https://docs.oracle.com/javase/tutorial/>
| <http://docs.oracle.com/javame/8.0/api/dio/api/index.html>

Java ME & OpenJDK DIO:

http://www.oracle.com/webfolder/technetwork/tutorials/obe/java/RaspberryPi_Setup/RaspberryPi_Setup.html |
<http://www.oracle.com/technetwork/java/embedded/javame/embed-me/downloads/java-embedded-java-me-download-2162242.html> | <http://docs.oracle.com/javame/8.1/get-started-rpi/toc.htm> | <http://www.oracle.com/technetwork/articles/java/cruz-gpio-2295970.html> | <http://docs.oracle.com/javame/8.0/api/dio/api/index.html>

IoT Gateway Device Dev Boards and Platforms

Raspberry Pi Model 1 / 2 / 3 / 4 with A / B / B+ layout

C POSIX/C++, Java SE-e / Java ME, Python, Node.js/Node-RED, Swift,
ARM - http://elinux.org/RPi_GPIO_Code_Samples

ASM ARM: <http://thinkingeek.com/arm-assembler-raspberry-pi/> |
<https://www.cl.cam.ac.uk/projects/raspberrypi/tutorials/os/>

Python : <https://www.tutorialspoint.com/python/> |
<https://www.tutorialspoint.com/python3/> |
<https://www.python.org/about/gettingstarted/> |
<https://docs.python.org/3/tutorial/> |
<https://docs.python.org/2/tutorial/index.html>

JS-Node.js: <http://www.tutorialspoint.com/nodejs/> |
<https://www.airpair.com/javascript/node-js-tutorial> |
<https://nodejs.org/en/docs/> | <http://eloquentjavascript.net/>

Node-RED: <http://noderedguide.com/> |
<http://nodered.org/docs/hardware/raspberrypi> | <http://noderedguide.com/nr-lecture-1/> ...

Swift: <http://dev.iachieved.it/iachievedit/swift-3-0-on-raspberry-pi-2-and-3/> |
<http://dev.iachieved.it/iachievedit/more-swift-on-linux/> |
<http://www.tutorialspoint.com/swift/>